Object Classification, Detection and State Estimation using YOLO v3 Deep Neural Network and Sensor Fusion of Stereo Camera and LiDAR

by

KAMALKUMAR MEHTA

Presented to the Faculty of the Graduate School of
The University of Texas at Arlington in Partial Fulfillment
of the Requirements
for the Degree of

MASTER OF SCIENCE IN AEROSPACE ENGINEERING

THE UNIVERSITY OF TEXAS AT ARLINGTON

August 2021

Copyright © by Kamalkumar Mehta 2021

All Rights Reserved

I would like to dedicate all my work to my mother Lt. Indiraben Mehta, my father Bharatkumar Mehta and my all sisters Rajeshree, Rasmika, Vaishali, and Sejal for providing continuous support and made me believe in myself.

Acknowledgements

First and foremost, I am extremely grateful to my master dissertation supervisor Dr. Kamesh Subbarao for his invaluable guidance and support throughout the process of this dissertation development. I am thankful to him for his belief in me to pursue research on one of the emerging field deep learning of which I have never been introduced and for providing me a research assistant position at Aerospace System Laboratory (ASL). I am honored to have such a great opportunity to upgrade my technical skill simultaneously with performing experiments.

I would like to express my gratitude to the members of my dissertation committee Dr. Animesh Chakravarthy and Dr. Alan Bowling for taking time out to provide their inputs to make this work possible. Additionally, I am thankful to the administrative staff in the Mechanical and Aerospace Engineering department for all guidance related to the documentation. I would also like to acknowledge the staff at the Office of International Education for their guidance related to maintaining my international student status throughout the development process of my dissertation and organizing the webinar of all aspects that the international students have to take care of.

Secondly, I am thankful to my parents who financially and morally supported me throughout my educational life and providing me the freedom to pursue study in the field of my interest. I am blessed to be the child of one of the best parents in this world. Moreover, I would like to thank all my friends for their generous help wherever required and for helping in making my baby steps in at the starting of the research work.

Finally, I would like to thank Professor Andrew Ng for launching a great Massive Open Online Courses (MOOCs) on Coursera where I have been first introduced to deep learning. Additionally, I am thankful to Mathworks Inc. and its technical support team for help me debugging the program and introducing a tool-set related to the work I was doing.

June 01,2021

Abstract

Object Classification, Detection and State Estimation using YOLO v3 Deep Neural Network and Sensor Fusion of Stereo Camera and LiDAR

Kamalkumar Mehta, M.S.

The University of Texas at Arlington, 2021

Supervising Professor: Dr. Kamesh Subbarao

Real-time object classification, localization, and detection with region-based convolution neural network (R-CNN) require high computational power, or it consumes a tremendous amount of time with the use of the available onboard computer system. Therefore, either of those ways is not practical in real-time object detection. In contrast, the YOLO-v3 network, which stands for you only look once, that uses the YOLO algorithm seems to be practical in live object classification, localization, and detection, since the YOLO algorithm seems to work faster than the sliding window algorithm used by R-CNN. Sensor fusion in this context requires estimation and association of information coming from more than one optical sensor that improves the accuracy in locating the object in the real world.

The goal of this research is to implement a real-time object detection and tracking algorithm for a collision avoidance application in an autonomous ground vehicle. Transfer learning was performed to train the pre-trained artificial neural network (YOLO-v3) that uses a supervised learning algorithm, and a labeled dataset was created to train the deep neural network. Hyper-parameters such as mini-batch size, L2

regularization parameter, number of epochs, and learning rate were tuned to study the trade-off between the bias and variance of a classifier to improve the performance of the neural network. Moreover, the intrinsic calibration of the stereo camera and LiDAR was performed to find standard measurement error by re-projecting the measurement data on ground truth data. Similarly, the extrinsic calibration was done to derive standard transformation error between these two optical sensors. Finally, the object's states were fed to the centralized sensor fusion framework, which uses a linear Kalman filter, to estimate the states of the objects.

The complete framework was developed in MATLAB 2020b by use of deep learning, computer vision and image processing, LiDAR, and sensor fusion and tracking toolboxes. The experimental platform was equipped with two single pin-hole cameras with Sony IMX 179 optical sensor, Intel Realsense L515 LiDAR camera, and Intel NUC as an on-board processing unit. Four 3d shapes (cylinder, cone, pyramid, and rectangular parallelopiped) were used to train the neural network for object detection. The driving scenario designer tool from MATLAB 2020b was used to simulate a driving scene and further use to test the sensor fusion framework.

Table of Contents

Acknowledgements iv

Abstract vi

List of Illustrations xii

List of Tables xv

Chapter Page Chapter

1. Introduction 1

    1.1 Background and Motivation 1

        1.1.1 Object Detection using Deep Learning 1

        1.1.2 Sensor Calibration and Fusion 4

    1.2 Contributions 6

2. Optical Sensors and Object Detection 8

    2.1 Introduction 8

    2.2 Optical Sensor: Camera 8

        2.2.1 Single Pin-hole Camera 9

        2.2.2 Stereo Vision 11

        2.2.3 Sony IMX 179 sensor features 14

    2.3 Optical Sensor: 3-D LiDAR Camera 15

        2.3.1 Intel Realsense L515 16

    2.4 Sensor Calibration 17

        2.4.1 Stereo Camera Intrinsic Calibration 17

        2.4.2 LiDAR Intrinsic Calibration 18

        2.4.3 Stereo camera and LiDAR Extrinsic Calibration 19

2.5 Object Detection 21
2.5.1 Detect Object in Stereo Camera using Deep Learning 22
2.5.2 Detect Object in LiDAR using Image Label 23
3. The Artificial Neural Network: YOLO-v3 24
3.1 Introduction 24
3.2 YOLO-v3 Architecture 25
3.3 Layers in Neural Network Architecture 27
3.3.1 Input Layer 27
3.3.2 Convolutional Layer 28
3.3.3 Non-linear Activation Layer 31
3.3.4 Pooling or Down-sampling Layer 34
3.3.5 Up-sample Layer 35
3.3.6 Fully Connected Layer 37
4. Supervised Learning 38
4.1 Introduction 38
4.2 Types of Supervised Learning 39
4.2.1 Classification 39
4.2.2 Regression 39
4.3 Stochastic Gradient Descent with Momentum (SGDM) 40
4.3.1 Loss and Cost Function 41
4.3.2 Training 43
4.3.3 Forward and Backward Propagation 45
4.4 YOLO algorithm 46
4.5 Anchor Box Estimation 48
5. Hyper-parameters Tuning 50
5.1 Introduction 50

5.2 Approaches for Tuning Hyperparameters 51
5.2.1 Trial & Error 51
5.2.2 Grid Search 51
5.2.3 Random Search 52
5.2.4 Bayesian Optimization 52
5.3 Bias and Variance Trade-off 54
5.4 List of Tunable Hyperparameters 56
5.4.1 L2 Regularization 56
5.4.2 Data Augmentation 57
5.4.3 Mini-batch Size 57
5.4.4 Number of Epoch 58
5.4.5 Momentum 59
5.4.6 Learning Rate 60
6. Multi-sensor Data Fusion 61
6.1 Introduction 61
6.2 Types of Multi-sensor Data Fusion 61
6.2.1 Feature Level Fusion 62
6.2.2 Position Level Fusion 62
6.2.3 Attribute Level Fusion 62
6.3 Frameworks of Position Level Fusion 63
6.3.1 Centralized Fusion Architecture 63
6.3.2 Distributed Fusion Architecture 64
6.3.3 Compound Fusion Architecture 66
6.4 Linear Kalman Filtering 66
6.5 Multi-Sensor Data Fusion 68
7. Results, Discussion and Future Scope 70

7.1 Experimental Setup 70
7.2 YOLO-v3 Supervised Training 70
7.3 Neural Network Testing 74
7.4 Real-time Object Detection Results and Position Estimation 79
7.5 Simulation Results of Sensor Fusion 80
7.6 Sensor Fusion Implementation on Hardware 87
7.7 Conclusion 94
7.8 Future Work 95
References 96
Biographical Statement 101

List of Illustrations

Figure Page

1.1 Data Pipeline of Research Work 6

2.1 Single Pin-hole Camera Model [1] 10

2.2 Radial Distortion [1] 12

2.3 Tangential Distortion [2] 13

2.4 Triangulation [1] 14

2.5 Sony IMX 179 Image Sensor with Non-Distortion Lens 15

2.6 Intel Realsense LiDAR Camera L515 16

2.7 Camera Intrinsic Calibration to find Intrinsic parameters 18

2.8 Camera Intrinsic Calibration to find standard measurement error . . . 19

2.9 LiDAR Intrinsic Calibration to find standard measurement error . . . 20

2.10 Stereo Camera and LiDAR Extrinsic Calibration 20

2.11 Performance of YOLO-v3 compare to Other Neural Networks [3] . . . 22

2.12 Data Pipeline to Generate LiDAR Label using Camera Data [4] 23

3.1 Artificial Neural Network Architecture 25

3.2 Stacks of Layers in YOLO-v3 Network [5] 26

3.3 Convolution operation over (a) a single filter and (b) multi-filters . . . 29

3.4 Stride and Padding in the Conv Layer 31

3.5 The shape of the classifier of the neural network with and without non-linear activation [6] 32

3.6 Non-linear Activation functions (a) Sigmoid, (b) Hyperbolic tangent, (c) Relu, and (d) Leaky Relu function [6] 33

3.7 The Pooling or Down-sampling Layer [7] 35
3.8 Up-sampling of inputs from various layers [6] 36
3.9 Fully Connected layers 37
4.1 Cost Function (a) Non-convex (b) Convex [8] 42
4.2 Exponentially Weighted Average of Noisy Data [9] 44
4.3 Robustness of Stochastic Gradient Descent with Momentum 44
4.4 Flowchart of Forward and Backward Propagation 45
4.5 YOLO Object Detection Pipeline 47
4.6 Anchor Boxes Estimation using IoU Distance Matrix 48
4.7 Trade off Between Number of Anchor Boxes and mean-IoU [10] 49
5.1 Loop of Hyperparameter Tuning Process [6] 50
5.2 Configuration Space Layout of Grid Search and Random Search Approaches [11] 52
5.3 Bayesian Optimization Process for (a) 2 Trained Model, and (b) 8 Trained Models [12] 53
5.4 Classifier Behaviour for Various Bias and Variance Conditions [6] . . . 54
5.5 Relationship between Bias and Variance [13] 55
5.6 Training Data Augmentation 58
5.7 Exponentially Weighted Averages for Different Values of β [9] 59
6.1 Attributes Identification Flow Chart [14] 63
6.2 Centralized Fusion Architecture [14] 64
6.3 Distributed Fusion Architecture [14] 65
6.4 Compound Fusion Architecture [14] 65
6.5 State and Co-variance Propagation in Prediction and Correction Steps [15] 67
6.6 Flowchart of Multi-sensor Data Fusion for Asynchronous Case 69
7.1 Wheeled Mobile Robot equipped with LiDAR and Stereo Camera . . . 71

7.2 Learning Rate Variation During Training 71
7.3 Loss Reduction During Training 72
7.4 Pie Chart of Test Results (a) % of Faulty Detection and (b) % of Faulty Detection due to Various Reasons 74
7.5 Confusion Chart of Classification on Test Batch 75
7.6 Overall Precision of Test Batch 76
7.7 True Positive Detections of Test Batch 77
7.8 Faulty Detections of Test Batch (a),(b), and (c) Multiple Detection for Single Object, (d) and (f) No Detection, and (e) No Detection due to Motion Blur 78
7.9 Real time Object Detection with Position Estimation 79
7.10 Driving Scene in Simulation (a) Bird's Eye View of Scene (b) Way points Definition 81
7.11 Position Estimation of Object in Simulation 82
7.12 Velocity Estimation of Object in Simulation 83
7.13 Acceleration Estimation of Object in Simulation 84
7.14 Trajectory Estimation of a Object w.r.t Inertial Frame 85
7.15 Sensor Measurement and Estimation Error Plot 86
7.16 Real time Position Estimation of a Object 'Box' 89
7.17 Real time Velocity Estimation of a Object 'Box' 90
7.18 Real time Position Estimation of a Object 'Traffic cone' 91
7.19 Real time Velocity Estimation of a Object 'Traffic cone' 92
7.20 Real time Object Detection and Position Estimation of Various Object 93

List of Tables

Table		Page
2.1	Measurement Error of Intrinsic Calibration	18
2.2	Re-projection Error of Extrinsic Calibration	20
5.1	Effect of Variation in Train and Test Error on Bias and Variance	55
7.1	Optimal Value of Hyperparameter Set Considered in Training	73
7.2	Performance Parameters of Testing	76
7.3	Performance Parameters of Detections in Fig.7.7	76
7.4	Driving Scene Parameter Definitions	81
7.5	Sensor Measurement and Residue Error in Driving Scene	85

Chapter 1

Introduction

1.1 Background and Motivation

1.1.1 Object Detection using Deep Learning

From ancient times, it has been a long desire by the inventors to make the machine that can think [13]. In the last two decades, computer science pioneers are researching on development of human-like intelligence in machines. Artificial intelligence (AI) becomes an active research field worldwide in recent years, as it has many practical applications such as autonomous navigation, speech recognition, online advertisement, etc. The availability of advanced computing hardware and state-of-the-art algorithms has attracted more scientists and researchers to involve in this field.

Autonomous navigation is one of the prime application of AI, which is widely used to accomplish real-time tasks such as plan a route of commercial aircraft, detect traffic signals, signs, pedestrian, and other vehicles to avoid a collision, simultaneous localization and mapping in a mobile robot, and remote medical surgery by a robotic arm. Similar to the human visual intellectual system, the perception of surroundings can be made by the use of exteroceptive optical sensors such as LiDAR and a camera. Moreover, the sensory data are required to pre-process to extract 3D environment information like classes of object, and relative 3D position of objects from the vehicle. Computer vision techniques were used in the early days to extract such information from images. However, it was difficult to pull out all useful info from hard-coded

object recognition and detection programs. Thus, such approaches are not efficient and robust to use in real-time applications.

The challenges faced due to the hard-coded knowledge-based system suggest the use of the intelligent system that can acquire own knowledge from patterns and features of raw data. This idea has introduced machine learning techniques. Deep learning is a subsidiary field of machine learning that allows computers to perform complex tasks with achieving nearly human-level understanding. The reason behind learning the high-level representation of features of raw data is the depth of the deep neural network. The greater depth of the network can able to execute a complex sequence of instructions serially and/or parallelly. The supervised training of a deep learning model can generate a map from training data to their labels and further store it in the model as a model parameters weight and bias.

Convolutional Neural Network (CNN) is a neural network that use both computer vision and machine learning techniques to accomplish object recognition and detection task. At the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012, Krizhevsky et al. [16] proposed AlexNet a Deep Convolutional Neural Network (DCNN) which has broken the ground in the field of deep learning by showing its capability in image classification over 1000 of different categories of image dataset with high accuracy. Since then, research in the domain of object detection [17], [18], [19], [20], [21] focuses in developing method that uses combine computer vision and deep learning techniques. In 2014, Girshick et al. [17] proposed a deep learning-based object detector the Region-based Convolutional Neural Network (R-CNN) which integrates AlexNet [16] and region-based selective search [22]. Apart from achieving high quality in object detection, R-CNN has noticeable drawbacks [19] as follow:

- It is trained on a multistage training pipeline which is difficult to optimize since each stage is required to train separately.
- The regression training is expensive in the context of system memory and time.
- Test time is slow for object proposal in each test image.

The list of drawbacks has leads innovators to propose numerous object detection frameworks such as SPPNet [18], Fast RCNN [19], Faster RCNN [23] [24] etc. In 2014 He et al. came up with Spatial Pyramid Pooling (SPP) with CNN architecture, in which they have added an SPP layer next after the last convolutional layer to extract the features from the object proposal. Since this framework requires passing test images once to get fixed-length features for object proposal, SPPNet proved to be significantly faster than RCNN. However, there is little improvement in the speed of detector training. To overcome the disadvantages of RCNN and SPPNet, Girshick et al. [19] in 2015 introduced Fast RCNN, an improved version of RCNN, that enables an end-to-end supervised training process that learns a softmax classifier together with bounding box regression. Fast RCNN has considerably improved in terms of efficiency for instance the training speed is 3 times and testing speed is 10 times faster than the initial model. The Faster RCNN model implemented by Ren et al. in 2017 provides efficiency and accuracy in Region Proposal Network (RPN) to generate precise region proposals. Nevertheless, the region-based object detection frameworks are computationally expensive specifically for mobile devices that have limited computational capability, therefore the trend in research shifted to develop a unified framework for detectors.

Redmon et al. [25] in 2016 proposed You Only Look Once (YOLO) a unified object detection framework that assigns object detection as a regression problem from objects image pixels to individual bounding boxes for each object and probability of their respective object class. Unlike the RCNN family, YOLO uses entire image

features to detect objects globally. Specifically, an input image is divided into a course grid each to predict the precision, bounding box position, and a class of the object. YOLO is proven to be faster in the context of training time and running on real-time data at 45 fps [25]. However, as discussed in [25], it may fail to detect small size objects and prone to make more localization error compared with Fast RNN [19]. Redmon and Farhadi in 2017 and Sung et al. in 2018 proposed YOLO9000 [10] and YOLOv2 [26] the improved versions of YOLO respectively, in which they replaced the GoogLeNet [27] with DarkNet19 [10], added batch normalization, removed fully connected layers, introduced anchor boxes training by using a k-mean clustering algorithm, and multi-scale anchor boxes training. The YOLO9000 network can detect over 9000 classes of objects in real-time by implementing a joint optimization technique to train together on the ImageNet classification dataset with the COCO detection dataset. However, the supervised detection of YOLO9000 is not reliable, i.e. it detects the classes of the object without bounding box annotations. Redmon and Farhadi [3] in 2018 advanced YOLOv3 to overcome the problems with old versions. It demonstrated three times the fast speed with equal accuracy as SSD at 320×320.

1.1.2 Sensor Calibration and Fusion

The multi-sensor data fusion has become an active research topic among the researcher as the significant improvement in information technology. Sensor technologies such as cameras, stereo cameras, Light Detection and Ranging (LiDAR), and radar have been widely used for the perception of the environment in intelligent vehicles [28]. Especially in autonomous navigation, object detection must have enough accuracy and speed to allow guidance, navigation, and control system to react quickly. The current literature suggests accurate object detection systems that use vision and range sensors, although few issues need to be addressed in the context of

autonomous navigation such as detect the object distance from the vehicle [29]. There are numerous approach has been proposed for object distance estimation based on the modality of sensors such as LiDAR, radar, and camera. LiDAR and stereo camera fusion technique has been introduced in [29] for object detection depends upon the different levels of information fusion.

The sensor fusion compensates for weaknesses of each modality of sensors, for instance, the camera has a limited field of view and cannot able to estimate object distance accurately as LiDAR whereas LiDAR cannot detect color information of environment as a camera; however, their environment perception ability become a complement for each other by fusing them. The authors of [30] proposed a cooperative fusion framework based on feature-level fusion by use of stereo vision and laser scanner. In Another article [31], fuzzy logic based sensor fusion method was introduced in which image and point cloud data parses individually for object distance detection and further fuse them to accurately estimate the distance. In this research work, they have used low-level fusion to robust the object detection process and it was experimentally proven by on-road experiments.

The first essential step of low-level data fusion is extrinsic calibration between the camera and LiDAR. By extrinsic calibration of two sensors, the relative geometric position and orientation of each sensor are determined, which further use to define the correspondence between the 3D point cloud and 2D image data [32–34]. In many study on sensor calibration, the authors have used external object to find correspondence for example trihedral rig [35–37], circles, board patterns [38,39], checkerboard [40], etc. Recently, Geiger et al. [41] proposed an automatic sensor calibration method, in which the sensor automatically detects the target objects. The automatic calibration technique uses the Random Sample Consensus (RANSAC) algorithm to

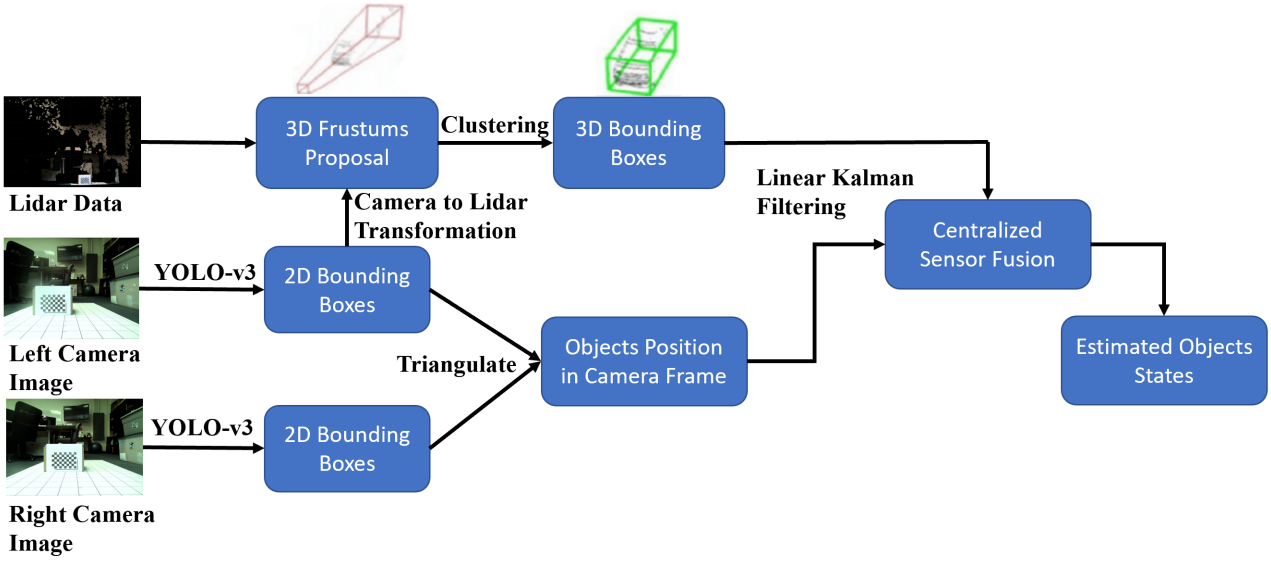


Figure 1.1. Data Pipeline of Research Work.

extract the target object plane and the Iterative closest Point (ICP) algorithm with non-linear optimization to accurately define an external parameter for sensor fusion.

1.2 Contributions

The list of contributions in this thesis are listed below.

- Performed a transfer learning on pre-trained Convolutional neural network YOLOv3 by using Stochastic Gradient Descent Method (SGDM) for object detection and classification.
- Iteratively performed supervised learning by tuning hyper-parameters such as L2 regularization parameter, Learning rate, mini-batch size, numbers of epochs, and learning momentum to improve the performance of the detector.
- Accomplished evaluation of the trained model in real-time using the pre-labeled test dataset.

- Perform an intrinsic and extrinsic calibration of optical sensors stereo camera and LiDAR to find their standard measurement errors, co-variance, and euclidean transformation between two sensor frames.
- Implemented a sensor fusion framework for constant acceleration motion model by using linear Kalman filter in a simulation environment.
- Implemented a sensor fusion on hardware for real-time application after successful testing in a simulated environment.

Chapter 2

Optical Sensors and Object Detection

2.1 Introduction

The detection of light from the light sources intuits the idea of vision, based on what the human eyes are working on. When light strikes on any object, some part of it is absorbed by the object, and the rest is reflected into the environment. This reflected part makes its way to the human eyes and accumulates on the retina. The light collected in the retina generates a picture of the object from where it is reflected. This kind of natural phenomenon has immense importance in the development of photosensitive instruments [1] and such devices are widely used in the computer vision field. It is a multidisciplinary scientific field that primarily deals with a high-level understanding of the images, which is used to extract numerical and symbolic information of the scene. Further, this information is used to make a useful decision in autonomous robotic applications.

Object detection is a computer vision technique that uses to locate an object within an image or video frame. The primary goal of object detection is to replicate human intelligence using a computer. Object detection algorithms use many machine learning techniques such as convolutional neural networks, image thresholding, RGB color channel segmentation, support vector machines, and aggregate channel features.

2.2 Optical Sensor: Camera

A camera is a small sealed box with light-sensitive film in it. A small hole on the box is positioned in such a way that it allows limited light rays that concentrate

at a single point. This point is known as an optical center, and the distance between the optical center and photographic film is known as focal length. Such a camera cannot be used for practical purposes since its light detection capacity is very small. Therefore, it is equipped with an external lens to increase the light gathering capacity. Two types of camera sensor arrays are used in an autonomous robotic application: 1. Single pinhole camera and 2. Stereo camera. These will be discussed in detail further in the following section.

2.2.1 Single Pin-hole Camera

A simple geometric arrangement of a single pin-hole camera is illustrated in Fig. 2.1. A reflected light ray from the object Q located at (X, Y, Z) passes through the image plane and heads towards the center of projection or optical center, and $\mathrm{q}(x, y, f)$ is the object's projection on the image plane. The object's position is defined in the world frame, which is a Cartesian coordinate system. While, the object's projection is defined in the image frame, which is a homogeneous coordinate system. The simple projective transformation from world frame to image frame by use of similar triangles is as follows.

$$x = \frac{f \cdot X}{Z}, y = \frac{f \cdot Y}{Z} \tag{2.1}$$

Where, f is the focal length of the camera.

Three reference frames used to illustrate the camera model are: (1) World Frame, (2) Camera Frame, and (3) Image Frame. The parameters used to describe the transformation from world coordinates to camera coordinates are called extrinsic. And those from camera coordinates to image coordinates are called intrinsic. Extrinsic parameters represent the 3-D Euclidean transformation between the world frame and camera frame. And intrinsic parameters represent projective or homogeneous

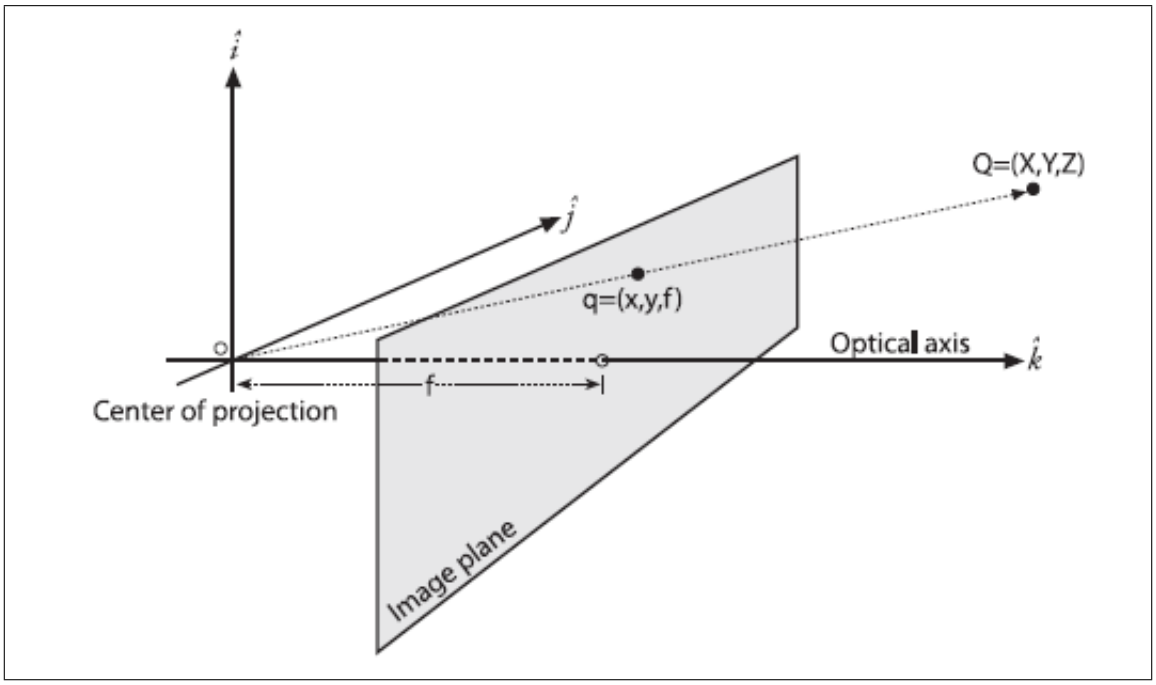


Figure 2.1. Single Pin-hole Camera Model [1].

transformation between 3-D camera frame and 2-D image frame. The camera model derived using intrinsic and extrinsic parameters is mentioned below.

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = K \times \begin{bmatrix} R & t \end{bmatrix} \times \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{2.2}$$

$$K = \begin{bmatrix} f_x & 0 & 0 \\ s & f_y & 0 \\ c_x & c_y & 1 \end{bmatrix} \tag{2.3}$$

Where, K is a intrinsic matrix, R and t are rotation and translation matrices, (f_x, f_y) is a focal length in pixel, (c_x, c_y) is optical center coordinates in pixel, and s is skew coefficient.

2.2.2 Stereo Vision

A stereo camera is composed of two cameras situated in a tandem way with a small distance between them. It is mainly used to simulate human binocular vision. By looking at the single object from two different positions, the object's location appears different concerning two individual camera's image frames. This difference is known as disparity which can be used to compute 3D world coordinates of the object by using triangulation. Stereo imaging requires four steps to accurately estimate the depth data: 1. Undistortion, 2. Rectification, 3. Correspondence Formation, and 4. Triangulation.

1. Undistortion: The cameras are used with external wide view lenses that result in image distortion. There are two types of distortions: Radial and tangential. The image can be undistorted by using radial and tangential distortion models. Light rays at the edge of a lens bend much more compare to rays close to the lens optical center which results in radial distortion. Due to radial distortion, the straight edge of any object appears curved in the image as seen in Fig. 2.2. Radial distortion is modeled using radial distortion coefficients of the lens as shown in Eq. (2.4).

$$\begin{aligned} x_{\text{distorted}} &= x\left(1 + k_1 \cdot r^2 + k_2 \cdot r^4 + k_3 \cdot r^6\right) \\ y_{\text{distorted}} &= y\left(1 + k_1 \cdot r^2 + k_2 \cdot r^4 + k_3 \cdot r^6\right) \end{aligned} \tag{2.4}$$

Where, x and y are undistorted normalized pixel coordinates, k_1, k_2, and k_3 are radial distortion coefficients, and r is the radial distance from the optical center.

Tangential distortion occurs when the lens plane is not completely parallel with the image plane as seen in Fig. 2.3. This may happen due to manufacturing defect or use of cheap glue to stick the imager in the camera. It is modelled using tangential distortion coefficients of the lens as shown in Eq. (2.5).

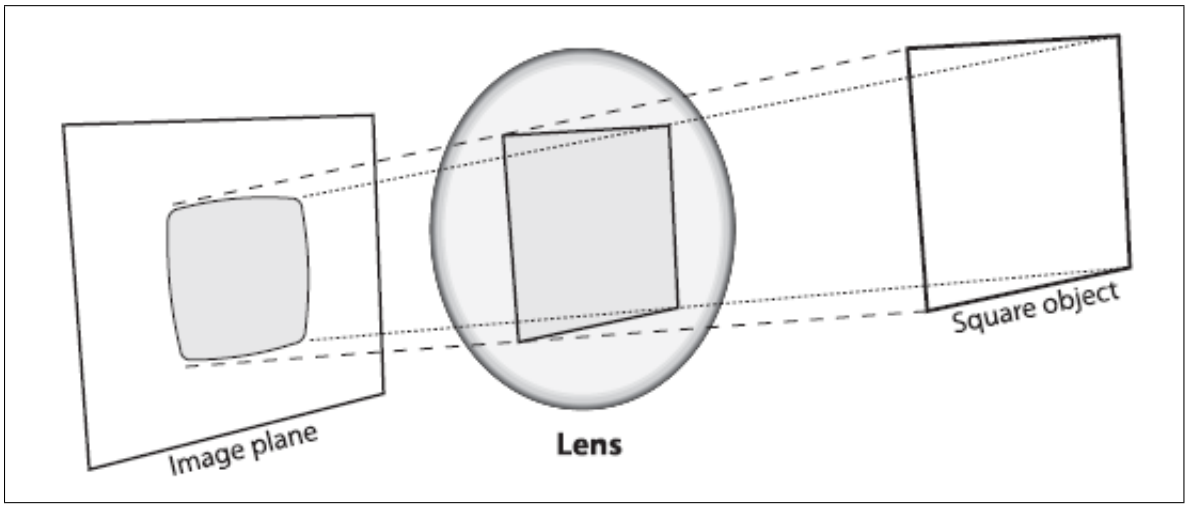


Figure 2.2. Radial Distortion [1].

$$\begin{aligned} x_{\text{distorted}} &= x + [2 \cdot p_1 \cdot x \cdot y + p_2 \cdot (r^2 + 2 \cdot x^2)] \\ y_{\text{distorted}} &= y + [p_1 \cdot (r^2 + 2 \cdot y^2) + 2 \cdot p_2 \cdot x \cdot y] \end{aligned} \tag{2.5}$$

Where, x and y are undistorted normalized pixel coordinates, p_1,and p_2 are tangential distortion coefficients, and r is the radial distance from the optical center.

2. Rectification: Rectification is the process in which images from two camera sensors are rotated and twisted to make the difference between their y coordinates to zero. Firstly, any point of interest is multiplied with an essential or fundamental metric which is derived in stereo camera intrinsic calibration to find its corresponding epipolar line in another view. These two epipolar lines from both the views are then made parallel to the horizontal axis of the stereo camera. Image rectification is performed when two cameras are not aligned and their image planes are not co-planner. This is usually the case for all manufactured stereo cameras because it is very difficult to maintain perfect coplanarity between two cameras.

3. Correspondence Formation: In this step, a search algorithm is used that finds similar features in both images. This is a computationally expensive process.

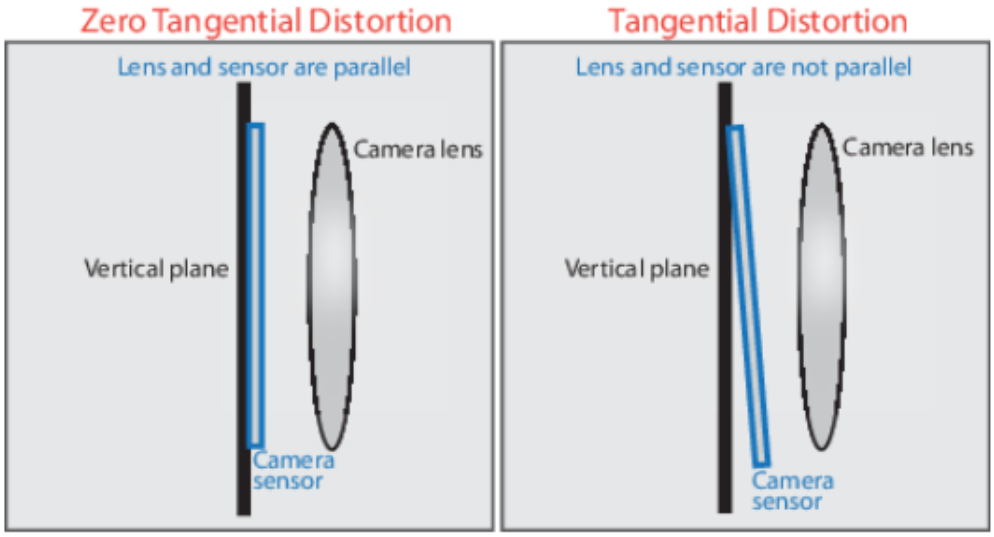


Figure 2.3. Tangential Distortion [2].

However, prior knowledge of the object geometry can be helpful to reduce the search space. The result of this step is a disparity map, where the value of disparity is a difference in x - coordinates of corresponding points in the images from left and right cameras.

4. Triangulation: Triangulation is a technique where the depth information is derived by the use of prior knowledge of disparity and distance between two cameras. As shown in Fig. 2.4, the depth information is easily computed by the use of a similar triangle rule. The relationship between depth and disparity is shown in Eq. (2.6).

$$Z = \frac{f \cdot T}{x^l - x^r} \tag{2.6}$$

Where, Z is a depth of object w.r.t stereo camera, f is a focal length, T is a horizontal distance between two cameras, and x^l and x^r are the projection of object P on the left camera image plane and right camera image plane respectively.

It can be inferred from the above equation that the depth is inversely proportional to the disparity. The error in-depth calculation increases as the disparity value

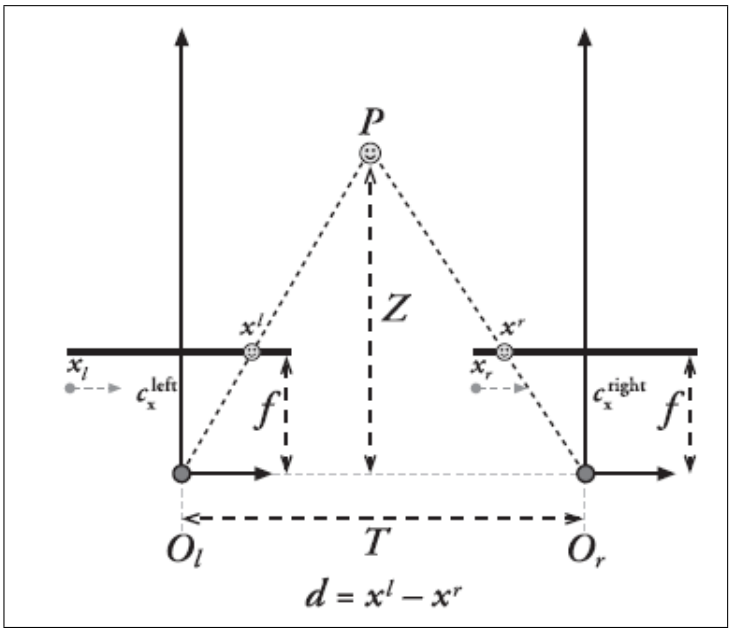


Figure 2.4. Triangulation [1].

get closer to zero. Therefore, the depth resolution is precise only for the objects which are relatively near to stereo camera than distant objects.

2.2.3 Sony IMX 179 sensor features

This work uses two Sony IMX 179 image sensors for stereo vision. This sensor is accompanied by a non-distortion lens which has a 75^o field of view in a horizontal direction. The features of this sensor are mention below.

- CMOS active pixel type dots
- CSI2 serial data output
- RGB primary color pigment mosaic filters on chip
- Max. 30 fps in all pixel scan mode
- 8 MP high definition resolution
- Image Resolutions available: 3264 X 2448, 3200 X 2400, 1024 X 768

Figure 2.5. Sony IMX 179 Image Sensor with Non-Distortion Lens.

2.3 Optical Sensor: 3-D LiDAR Camera

LiDAR stands for Light Detection And Ranging. It is one of the remote sensing methods that uses a pulsing laser to measure the range of an object. LiDAR instruments were initially used for terrain mapping, surveying and are fitted on satellites and aircraft. However, in recent years it is widely used in autonomous driving applications. It works on the principle of time of flight. The range resolution of LiDAR can be defined by Eq. (2.7). LiDAR generates the object's data in form of a point cloud which is composed of X, Y, Z range channels, Intensity, and RGB channels. A LiDAR scanner is a type of instrument that rotates around its axis to scan the surroundings. However, in this research work, a LiDAR camera is used, which has 70^o of the horizontal field of view, to get 3D point cloud data.

$$R = \frac{c}{2B} \tag{2.7}$$

Figure 2.6. Intel Realsense LiDAR Camera L515.

Where, R is a range, c is a speed of light in the corresponding medium, and B is the system bandwidth.

2.3.1 Intel Realsense L515

The Intel Realsense LiDAR camera L515 is the first camera manufactured by Intel that enables high-resolution accurate depth sensing. It is designed specifically to be used in a remote-controlled mobile robotics application. Some features of this LiDAR instrument are mention below.

- Depth capture at 95 percent reflectively of surface is 0.25 m to 9 m
- Diameter is 61 mm and height is 26 mm
- Approximate Weight is 100 g
- Power consumption is 3.5 W
- A short exposure time of laser is less than 100 ns per depth point

- A number of depth points per second is 9.2 M at 640 X 480 and 23.6 M at 1024 X 768 image resolution
- Depth accuracy is 5 mm to 14 mm

2.4 Sensor Calibration

The formal definition of calibration is "Operation that, under specified conditions, in a first step, establishes a relation between the quantity values with measurement uncertainties provided by measurement standards and corresponding indications with associated measurement uncertainties (of the calibrated instrument or secondary standard) and, in a second step, uses this information to establish a relation for obtaining a measurement result from an indication" [42].

It is clearly stated in the definition that the calibration process is completely based on the comparison. In the calibration process, two datasets are used which are the ground truth and sensor measurement dataset. Ground truth dataset is precisely generated using standard measurement instrument while measurement data is collected from the sensor which is going to calibrate. By re-projecting, the measurement data on ground truth data, the standard error of the respective sensor can be derived.

No sensor is 100 percent accurate. Thus, the calibration of sensors is an essential part of the system. The calibration is done to find standard measurement error is called intrinsic calibration whereas the calibration is done to find standard transformation error between two measurement sensors is called extrinsic calibration.

2.4.1 Stereo Camera Intrinsic Calibration

Intrinsic camera parameters are calculated by first taking forty images of checkered board patterns at different positions w.r.t to the camera. Five images out of forty were ignored because of the high reprojection error. Further, the calibration

Description	Stereo Camera	LiDAR
X Error	0.08 mm	0.10 mm
Y Error	-0.09 mm	-0.04 mm
Z Error	-0.03 mm	-0.19 mm
Overall Error	-4.5 mm	-21.89 mm

Table 2.1. Measurement Error of Intrinsic Calibration

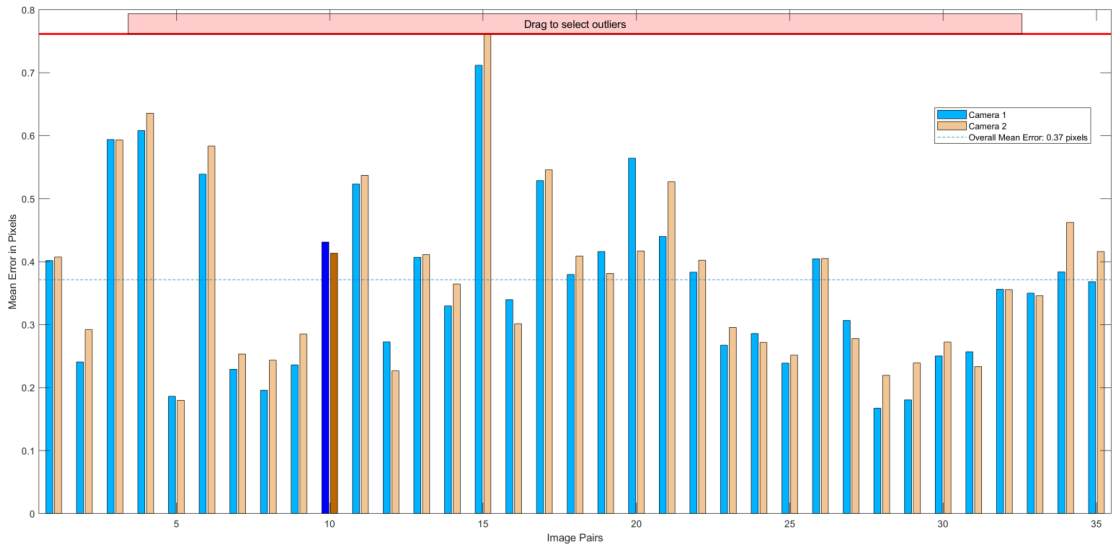


Figure 2.7. Camera Intrinsic Calibration to find Intrinsic parameters.

was performed which has 0.37 pixels of re-projection error as shown in Fig. 2.7. The histogram of standard measurement error of stereo camera is shown in Fig. 2.8 and standard measurement error is also mentioned in table 2.1.

2.4.2 LiDAR Intrinsic Calibration

Intrinsic calibration of LiDAR was performed by using twenty-five images of checkered board pattern by positioning checkered board at various location w.r.t LiDAR. Similar to the stereo camera, the ground truth and measurement data of LiDAR were generated. Finally, these two datasets were used to find the standard

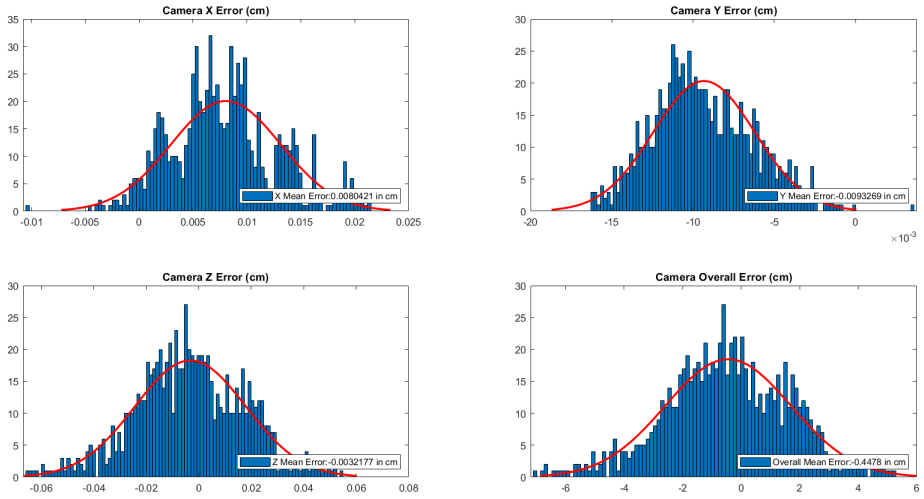


Figure 2.8. Camera Intrinsic Calibration to find standard measurement error.

measurement error of LiDAR. The histogram of LiDAR calibration errors is pictured in Fig. 2.9.

2.4.3 Stereo camera and LiDAR Extrinsic Calibration

In the Extrinsic calibration for both sensors, the same data as in intrinsic calibration was used to get a 3-D rigid transformation matrix. The data set collected manually is considered as ground truth data. The 3-D transformation matrix is derived from both the ground truth data. There is a total of two transformation matrices that can be derived, for Camera to LiDAR transformation and LiDAR to camera transformation. After, the measurement data set was generated using transformation matrices. Further ground truth and measurement data set were used to derive standard measurement error for 3-D rigid transformation. The transformation error is mentioned in table 2.2 and the histogram is shown in Fig. 2.10.

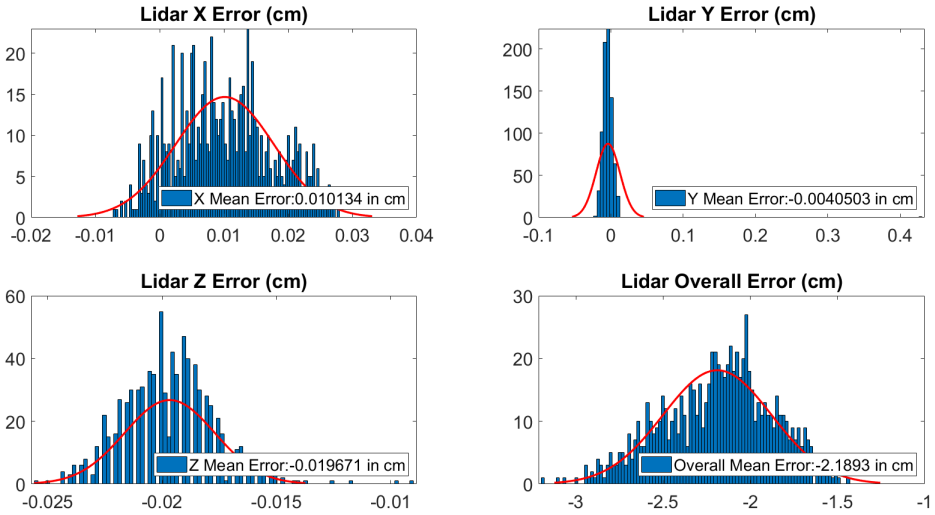


Figure 2.9. LiDAR Intrinsic Calibration to find standard measurement error.

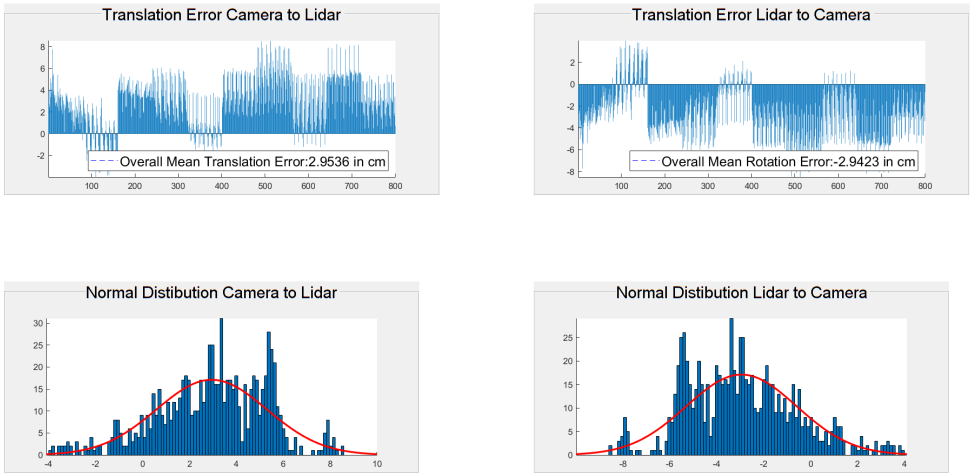


Figure 2.10. Stereo Camera and LiDAR Extrinsic Calibration.

Description	Camera to LiDAR	LiDAR to Camera
Re-projection Error	29.5 mm	-29.42 mm

Table 2.2. Re-projection Error of Extrinsic Calibration

2.5 Object Detection

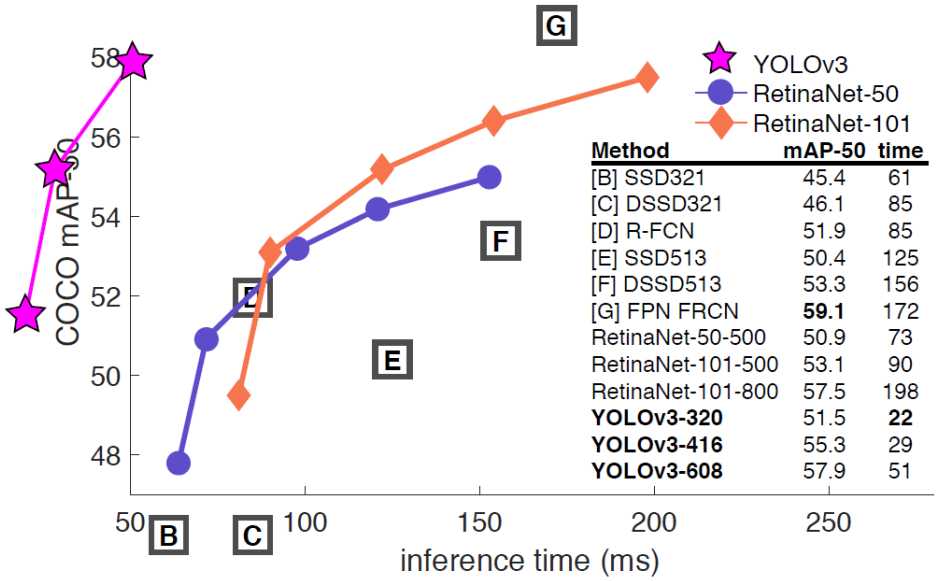
Any object in this world has special features that make it stand different from others, three edges for a triangle for instance. Such features can be used to classify different objects. However, object detection is not just a task of classifying the object, it also includes object classification, localization, and mapping w.r.t body frame. There are many computer vision techniques for object detection. However, such an algorithm might require a controlled environment. For instance, the image thresholding technique needs an indoor environment without sunlight. Therefore, This helps to reduce real-time challenges faced in the object detection algorithms.

1. Occlusion: This problem occurs when one object obstructs the view of another object that is behind it. In such a condition, the computer program does not completely understand the features of the object or fail to recognize the occluded object and gives the combined detection result which is not acceptable in a real-world application.

2. Change of viewpoint: Non-homogeneous 3D objects with complex shapes may not be recognized when viewed from another direction than the one for which the algorithm developed for. For example, for a case of detecting a person, if the detector's viewpoint is changed from front to right then the algorithm may detect fewer features of the person, and or it may fail to detect the person.

3. Variation in size: The algorithms are developed to work in real-time object detection applications. Thus, images with small sizes are used in such object detection tasks to overcome the computational load. Therefore, this makes recognition very difficult for the objects occupying few pixels in the image. In such a case, the object detection algorithm is effective only for a limited range.

4. Motion Blur: When it is a case of detecting a moving object such as a car or any vehicle, it is usually the issue that the camera image is blurred because of the



Method	mAP-50	time
[B] SSD321	45.4	61
[C] DSSD321	46.1	85
[D] R-FCN	51.9	85
[E] SSD513	50.4	125
[F] DSSD513	53.3	156
[G] FPN FRCN	**59.1**	172
RetinaNet-50-500	50.9	73
RetinaNet-101-500	53.1	90
RetinaNet-101-800	57.5	198
YOLOv3-320	51.5	**22**
YOLOv3-416	55.3	29
YOLOv3-608	57.9	51

Figure 2.11. Performance of YOLO-v3 compare to Other Neural Networks [3].

slow shutter speed. In such a scenario, robust object detectors are recommended to use.

2.5.1 Detect Object in Stereo Camera using Deep Learning

Object detection using deep learning techniques work faster than other machine learning algorithm and provide an accurate prediction of object class and location within the image frame. Neural network learns features of objects while training and store them as weights and biases of the network. In this research work, the YOLO (you only look once) neural network was used to detect four classes of geometric shapes like pyramid, square, cone, and cylinder. This network was trained on self-generated training data set of 11000 images of mentioned classes of objects.

YOLO-v3 neural network is proven to be faster than other deep learning techniques as seen in Fig. 2.11. This is because unlike a traditional search algorithm it divides the input image into grids to perform a search of objects. The incred-

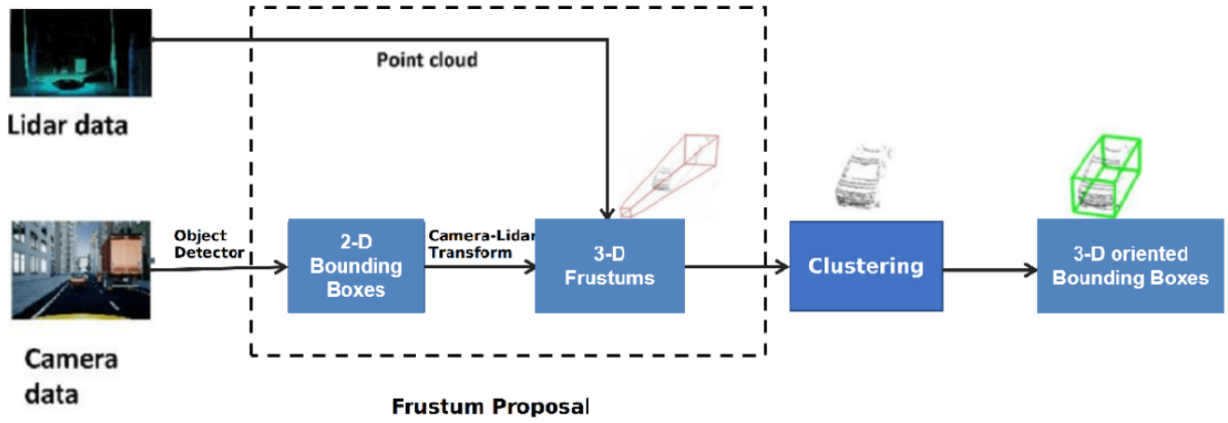


Figure 2.12. Data Pipeline to Generate LiDAR Label using Camera Data [4].

ible processing speed of YOLO has made it qualify for real-time object detection applications.

2.5.2 Detect Object in LiDAR using Image Label

To use an individual neural network for object detection from LiDAR data consumes a lot of processing time. Instead, using the labels generated from image data to predict a 3-D bounding box in LiDAR data costs less time. A schematic of the data pipeline to predict a 3-D bounding box in LiDAR data using Image label is given in Fig. 2.12.

Chapter 3

The Artificial Neural Network: YOLO-v3

3.1 Introduction

The artificially generated multi-layer perceptron architecture is inspired by the biological neural structure of the human brain. It is identified as an artificial neural network (ANN), as it is structured with millions of connected nodes or called artificial neurons in a hierarchical pattern. In a neural network, millions of units are grouped in different layers to create a deep neural network to perform complex human-level functions. The ANN is an organized structure of different types of layers connected in a serial or parallel manner. To achieve a human-like performance, hundreds of layers are connected in a complex pattern to generate a deep neural network, and the empirical results summarized in [13] prove that the ANN has the capability to perform tasks that are impossible by conventional algorithms.

In the fully connected layer of any neural network, all nodes of the preceding layer have connections with all the nodes in the next layer. The connection between nodes of two successive layers is identified as a signal. Each signal contains two parameters: weight and bias. The values of these parameters state the influence of one node on another. The weight and bias are the learn-able parameters that can be learned while training. These two quantities implicitly control the training evaluation that will be discussed in detail later in this chapter.

Mainly, the artificial neural network comprising an input layer, multiple hidden layers, and an output layer. As shown in Fig. 3.1 the input layer contains the training dataset such as images for the classification problem, audio clips for speech recogni-

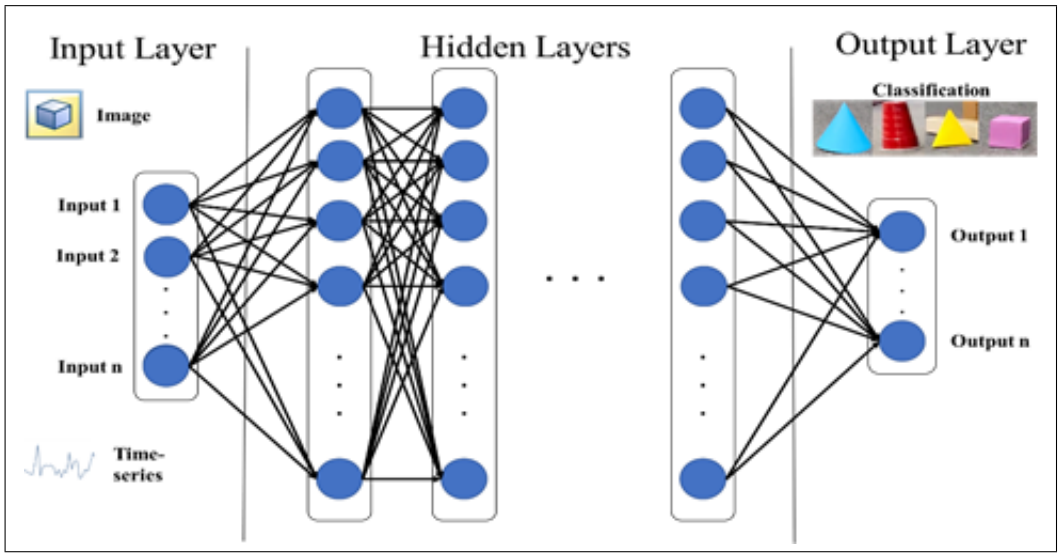


Figure 3.1. Artificial Neural Network Architecture.

tion, and labeled image datasets for object detection problems. The convolutional neural network can only accept image data in the input layer whereas the recurrent neural network only takes time-series data. The hidden layers contain multiple groups of layers such as a fully connected layer, convolutional layer, normalization layer, Relu activation layer, sigmoid activation layer, pooling layer, and many more. Finally, the output layer is a classification layer based on the function of ANN. Therefore, it can be a SoftMax layer for multi-class classification or a binary classification layer for identifying a single class object.

3.2 YOLO-v3 Architecture

The YOLO-v3 network is a concatenation of a convolutional neural network (CNN) and multiple detection heads [3]. In this architecture, CNN is used for feature detection and image classification, and multiple detection heads are used to localize and detect an object in a frame. More than one detection heads are used to detect the various size of objects. The bigger sized head can detect a tiny object or the

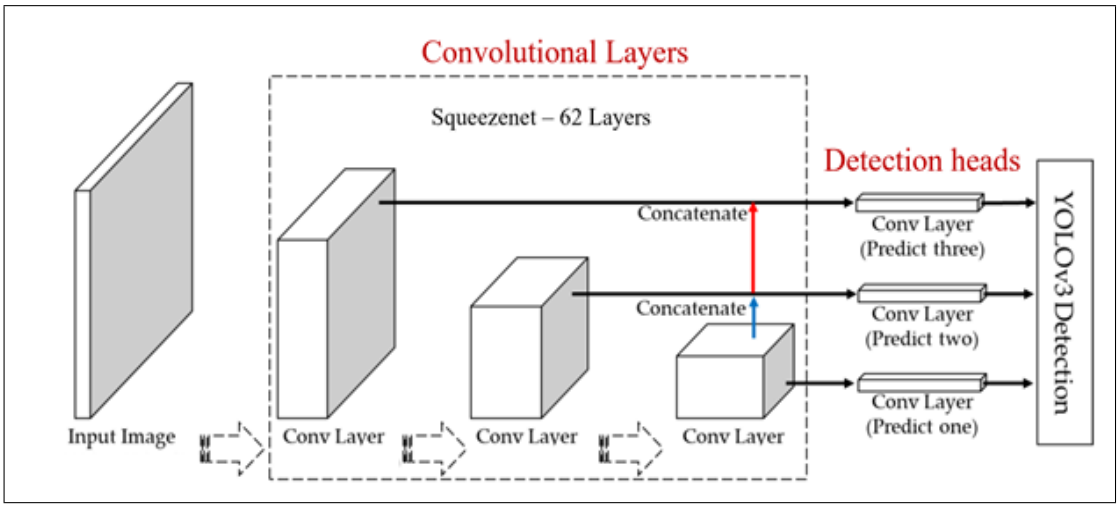


Figure 3.2. Stacks of Layers in YOLO-v3 Network [5].

object positioned far from the camera and vice versa. This type of structure can be created by using any efficient convolutional neural network with any number of detection heads by considering the computational cost of time delay in response. This architecture uses anchor box estimation of labeled training data to accurately predict the bounding box.

This network uses YOLO (you only looks once) algorithm to localize and detect the object. Unlike the conventional detection algorithm (the sliding window algorithm), the YOLO algorithm divides the input image into grids and localizes objects within a frame. In terms of computational speed, the YOLO proves to be faster than its counterparts such as R-CNN, Fast-RCNN, and Faster-RCNN networks [3]. These networks use the variances of the sliding window algorithm. The YOLO detection algorithm will be explained in detail later in this chapter.

The YOLO-v3 architecture is pictured in Fig. 3.2. The input layer contains the training data as a labeled image dataset. The input layer image size is defined as 227×227×3. Therefore, this network accepts RGB images of size 227×227. In the input layer before passing the image dataset to the successive layer, all images are

resized to column vectors of size 154587×1. The column vectors of a dataset of a total of 8600 images are then concatenated in a large matrix of size 154587×8600. The hidden layers comprise convolutional layers grouped with non-linear activation layers such as Relu, sigmoid, and hyperbolic tangent layer. The size of the convolutional layer becomes smaller as moving deeper into the network. Three different sizes of activations from the convolution layer taken as input for the detection heads. The three-detection heads, of various sizes in the given case, are used to detect the different sizes of objects. The size of the detection head is pre-determined on what size of the object the network requires to detect. At the output layer, the detection head gives a class of objects, bounding box coordinates, and precision in output for each detected object.

3.3 Layers in Neural Network Architecture

The artificial neural network, as mentioned earlier in this chapter, is a stack of an organized group of various types of layers. All these layers are grouped systematically to accomplish the purpose of the network. Besides, the non-linear activation functions are grouped to pose non-linearities in the results, which is important when data does not follow the uniform distribution. The types of layers in the neural networks are explained in detail below.

3.3.1 Input Layer

The input layer takes grayscaled or RGB images to a network for object classification or detection problems. In this layer, the size of the input image is pre-defined at the time of neural network design and it can not be modified in the case of transfer learning [6]. The primary purpose of the input layer is to filter the images that match

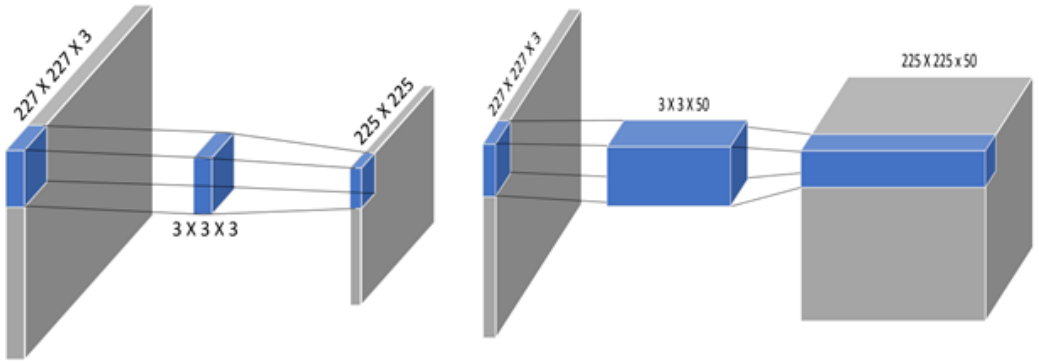
the size with the input layer. This layer does not perform any computation other than feeding the dataset to the hidden layers.

3.3.2 Convolutional Layer

The convolutional layer is the main building block of the convolutional neural network. In this layer, the convolutional operations are performed over the set of images to detect features such as edges, textures, patterns, parts, and objects. To perform the convolution operation, the size parameters of the convolutional filter such as height, width, and depth are defined as per the requirements. The depth of the filter is the same as the number of channels of the input image in the majority of cases and it trains to detect a single feature of the input dataset. In the backpropagation in supervised learning, the weight parameters of the filter are learned unless required accuracy can be acquired and then stored in the respective layers of the network for the further application.

In the convolution operation, each weight parameter of the filter dot multiplied with each respective pixel of the image, and then all results were summed together. By sliding the filter horizontally and vertically simultaneously and applying dot multiplication to respective entries of the filter and input image to generate an activation that passes to the upcoming layer. The convolution operation is graphically represented in Fig. 3.3, where the input image size is 227×227×3, and the filter size is 3×3×3. After the convolutional operation, the output size became 225×225×3. Therefore, it seems that at each convolution layer, the size of the image gets reduce as it passes deeper into the network.

Each layer of the convolutional neural network is predefined with the number of features its needs to detect. Since each convolution filter can detect a single feature from the input image, it needs to define the same number of filters as the number



(a) (b)

Figure 3.3. Convolution operation over (a) a single filter and (b) multi-filters.

of features needs to detect. The outcomes from each filter are concatenated into a multi-channel activation. For instance, an image of size 227×227×3 is passing from a Conv layer having 50 filters as depicted in 3.3(b), the output will be of the size 225×225× 50.

The size of activation from the Conv layer can be controlled by three hyper-parameters: 1. Number and size of filters, 2. Padding size, and 3. Stride length.

1. Filter size and numbers: The filter width and height are selected by considering the proportion of data association requires between the conv filter and the input image. The larger size in width and height of the filter represents the greater amount of data association, however, the outcome became shorter. The relationship between input size, filter size, and the output size is defined as follows.

$$n^{[L]} = n^{[L-1]} - f + 1 \tag{3.1}$$

where, $n^{[L]}$S is output size, $n^{[L-1]}$ is input size, f is filter size and L is the layer number.

As per the Eq. (3.1), the depth of the outputs is governed by the number of filters used in the conv layer and Input image size. Therefore, the size of the depth

is similar to the number of filters. For instance, if the Conv layer of n_f filters having the size $f \times f \times n_c^{[L-1]}$ and the size of an input image is $n_w^{[L-1]} \times n_H^{[L-1]} \times n_c^{[L-1]}$, the size of the output becomes $(n_w^{[L-1]} - f + 1) \times (n_H^{[L-1]} - f + 1) \times n_f$.

2. Padding: There are two disadvantages of the Conv network. The first is at each Conv layer, the size of the image decreases, and the second is the pixel values at the edge of the image are not associated with multiple values in the activation, because of this the important information at the edge is ignored. For the deep neural network that has a hundred or more than a hundred Conv layers, the reduction in the size of the image is not acceptable.

To overcome these disadvantages, the padding is applied at each edge of the input image as illustrated in Fig. 3.4. The border of the input image is padded with zeros such that the output size becomes the same as the original input size. Such type of convolution operation is known as "the same convolution", and without padding is identified as "the valid convolution". The relation between output and input size is as follows.

$$n^{[L]} = n^{[L-1]} + 2p - f + 1 \tag{3.2}$$

where, p is padding, and f is filter's size

3. Stride: The stride length is defined as how many pixels need to skip while sliding the filter over the input image. In general, the stride length is 1, which means the filter slides over each pixel in the image. As represented in Fig. 3.4, in the Conv layer with stride length 2, the filter jumps to 2 pixels and it skips the pixel at the middle position. If the value of the stride length for any Conv layer is more than one, the relation of output and input is defined as follows.

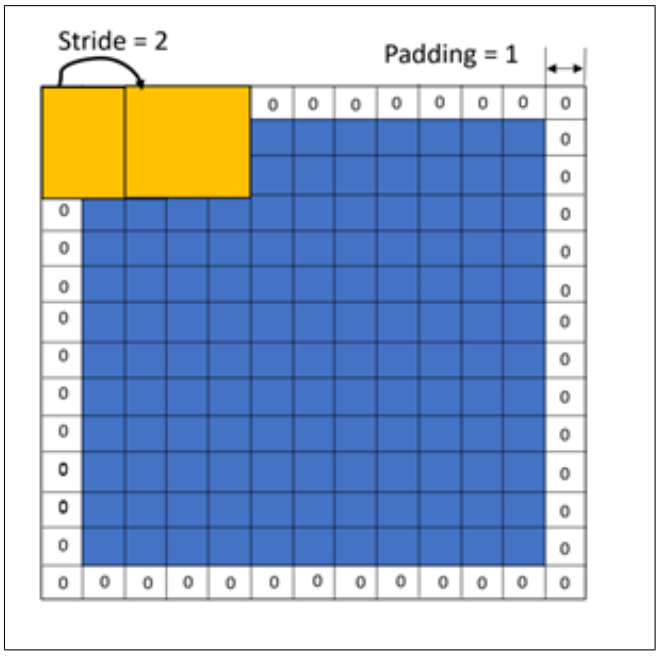


Figure 3.4. Stride and Padding in the Conv Layer.

$$n^{[L]} = \frac{n^{[L-1]} + 2p - f}{S} + 1 \tag{3.3}$$

To combine all hyper-parameters, if the filter size is $f^{[L]}$, padding of size $P^{[L]}$, stride length $s^{[L]}$, number of filters $n_C^{[L]}$, input image size $n_H^{[L-1]} \times n_w^{[L-1]} \times n_c^{[L-1]}$, and the output size is $n_H^{[L]} \times n_w^{[L]} \times n_c^{[L]}$

- Each filter dimension is $f^{[L]} \times f^{[L]} \times n_c^{[L-1]}$.
- Activation size is $n_H^{[L]} \times n_w^{[L]} \times n_c^{[L]}$.
- The total number of weights are $f^{[L]} \times f^{[L]} \times n_c^{[L-1]} \times n_c^{[L]}$.
- The total number of bais is $n_c^{[L]}$.

3.3.3 Non-linear Activation Layer

The non-linear activation layer is used just after the Conv layer. The feature map generated by the conv layer is taken as input and generates an activation map

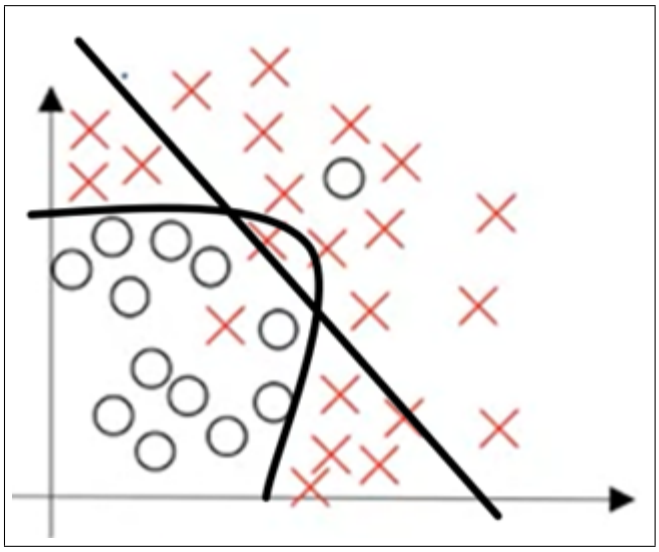


Figure 3.5. The shape of the classifier of the neural network with and without non-linear activation [6].

in output. Since the operation in this layer is performed element-wise, the output dimensions are similar to input dimensions.

The function of this layer is to impose low-level non-linearity into the feature map created by the conv layer. Since the conv layer operation is a linear function, the network without any nonlinear activation layer generates a linear classifier that has a high bias value. The trained neural network with high bias is not acceptable in any practical application. Fig. 3.5 illustrates the difference between linear and non-linear classifiers on a sample dataset.

There are many types of activation functions used in the non-linear layer such as sigmoid, hyperbolic tangent, Relu (rectified linear unit), and leaky Relu. The illustration of these activation functions is shown in Fig. 3.6.

The sigmoid and hyperbolic tangent functions are the exponential functions. The sigmoid function varies from 0 to 1, and the hyperbolic tangent function varies from -1 to 1. As illustrated in Fig. 3.6 (a) and (b), the mean of data of the sigmoid

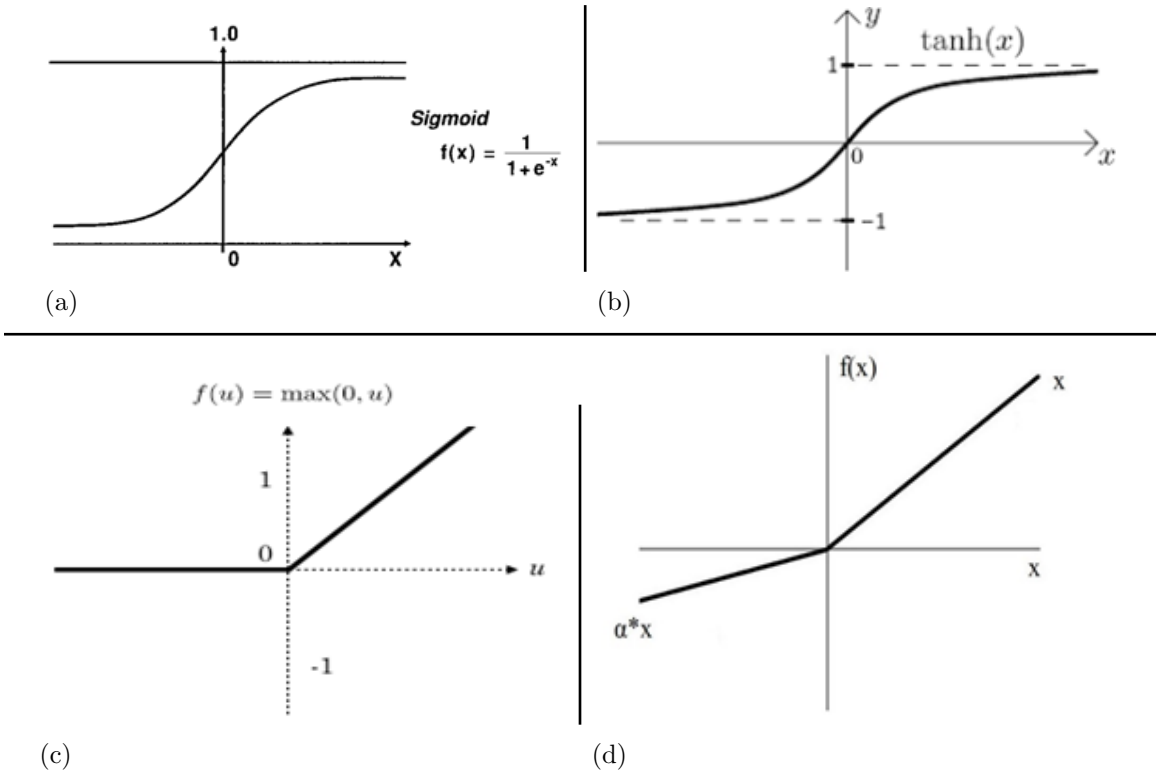


Figure 3.6. Non-linear Activation functions (a) Sigmoid, (b) Hyperbolic tangent, (c) Relu, and (d) Leaky Relu function [6].

activation function is 0.5, and for the hyperbolic tangent function is 0. Therefore, the tangent activation function is superior to sigmoid activation and preferable for the hidden layers. However, the sigmoid activation is preferable when 0 or 1 require at the output in the case of binary classification. Both functions have a little change in the slop at the larger value of the input, and because of this, the learning process slows down. To overcome this issue, the rectified linear unit function is used. The rectified linear unit function has a slop value of 1 for 0 to 1 and for the rest of the part, it is 0 as seen in Fig. 3.6 (c). This activation function is also identified as Relu activation. The Relu has a non-zero value at 0, and it is a very small value such as 0.00000001. Since it has a constant slope, comparatively it is faster than the sigmoid and tangent activation. The improved version of Relu is leaky Relu activation that

has a non-zero and little slope for the negative inputs as shown in Fig. 3.6 (d). In comparison, the Relu activation is preferable because of its faster learning speed. However, other activation functions are also used based on the purpose of the neural network.

3.3.4 Pooling or Down-sampling Layer

The Pooling or Down-sampling layer extracts the most significant number by the predefined size of the window. Similar to the conv layer, the filter slides over each channel of the input image and reduces the size of the input. Therefore, the pooling layer is used after several stages of conv layers or non-linearity layers to reduce the computational load, increase the speed of the training process, and prevent overfitting. The essential intuition behind the pooling layer is to discard the less significant data with preserving the main features of the image (for image recognition task) to reduce the cost due to the spatial resolution.

The hyperparameters of this layer are filter size, stride length, and padding, however, the filter has no parameters that need to learn while training. The shape of the filter should not be required to be square, it can be parameterized to the size of the rectangle. However, it is rarely be used in the practice. Since the pooling process is performed in each channel, the size reduction is happening in width and height, and the depth size is always similar to the input size as demonstrated in Fig. 3.7. The most common type of pooling layer has a filter size of 2 X 2 and a stride length of 2. Such a type of pooling layer does not have any overlapping nature of the filter, so it filters the unique set of the data in the output. In contrast, if the stride length is less than the filter size, data in the middle of the channel of the image filter multiple times. This type of pooling layer is also known as an overlapping pooling layer. It is highly preferable to use an overlapping pool layer since it is proven to be used to

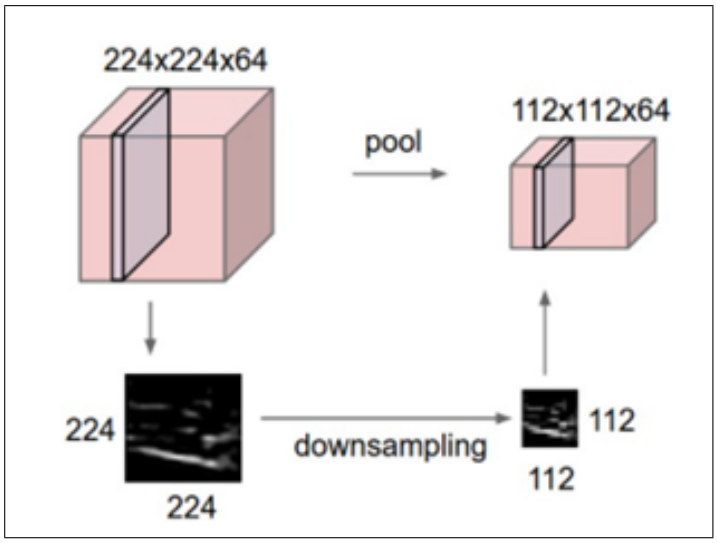


Figure 3.7. The Pooling or Down-sampling Layer [7].

prevent overfitting of the network. The larger filter size in the pooling layer has a destructive nature and degrades the performance of the neural network.

There are two most common methods for the pooling layer.

1. Max Pooling
2. Average Pooling

In the max pooling, it extracts the maximum value from the predefined sized window region of the input image. On other hand, the average pooling finds the mean value from the window region. In terms of performance, the max-pooling seems faster and has better convergence compare to average pooling.

3.3.5 Up-sample Layer

The Up-sampling layer is used when it needs to recover the input image or a part of the input image somewhere between the few Conv layer from the beginning and the output layer. The Up-sample layer converts the low-resolution image into a high-resolution image. This process can be done by learnable parameters or interpola-

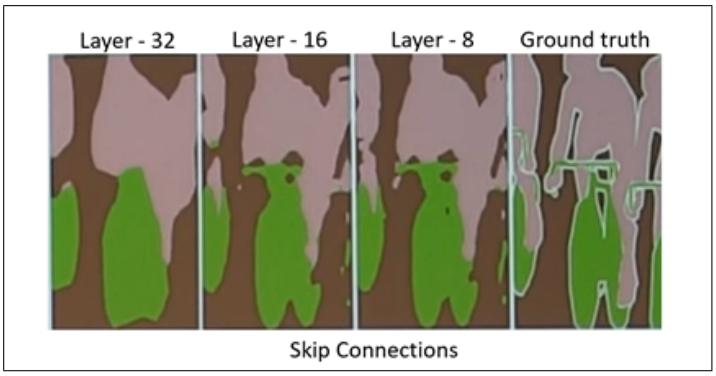


Figure 3.8. Up-sampling of inputs from various layers [6].

tion. Therefore, this layer can be defined with learnable parameters or non-learnable operations. In the interpolation, the elements in the small-sized input interpolated in multiple stages or row and column are copied many times. The learnable parameter is defined in the form of the elements of a predefined sized filter similar to the conv layer. However, unlike the conv network, the parameters get learned that generates an output that closely represents the ground truth image. The up-sampling can also be done by using "skip-connections". It means that instead of taking input from the very last layer, it can be imported from any sallow layer that has a comparatively higher resolution. As a result, the output accurately represents the ground truth image. In The up-sampling layer is used in a task when a high resolution of the image is required, such as object localization and detection. For object localization, detection, and track tasks require high-resolution input to accurately predict the position of any object. However, the bigger size image slows down the processing time in the live detection problem. Therefore, to trade off between the processing time and accuracy, the input for the up-sample layer should be selected with enough resolution by using skip connections. The comparison of up-sampling outcomes from inputs from various layers is shown in figure 3.8.

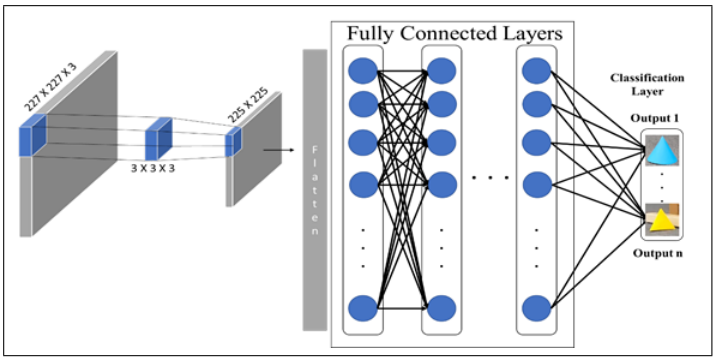


Figure 3.9. Fully Connected layers.

3.3.6 Fully Connected Layer

The fully connected layer is usually located between the convolution layer and the output layer. The purpose of this layer is to combine all the features detected by the conv layers and generate a classification score for each class. The activation from the conv layer first flattens before considering it as input to the fully connected layer. All the elements of the activation from the conv layer are connected with each unit of the fully connected layer. Since each connection between units of fully connected layers posses weight property, the number of trainable parameters in this layer is much more than other conv layers in the convolutional neural network. However, the fully connected layer is not used in the YOLO-v3 architecture to increase the robustness in real-time object detection problems, as it contains many trainable parameters that reduce the overall performance.

Chapter 4

Supervised Learning

4.1 Introduction

Supervised learning is a process of training a machine learning model, in which an algorithm is used to learn the mapping function $Y = F(X)$ from input to output. By and large, the desired output Y might be hard to gather and consequently should be given by a human "Supervisor", yet the term applies in any event when the preparation set was gathered naturally. Most supervised learning algorithms depend on assessing a likelihood appropriation $p(y|x)$. We can do this basically by utilizing the most extreme probability assessment to track down the best boundary vector θ for a parametric spread model $p(y|x;\theta)$.

While preparing the model, information is generally parted in the proportion of $80:20$ for example 80% as training data and the rest as testing data. The model gains from the training dataset as it were. We utilize distinctive AI algorithms, which we will talk about in detail in the following articles, to construct the trained model. By learning, it implies that the model will assemble some rationale of its own. At the test time, the data is taken from the rest 20% of the dataset that the model has never seen, the model predicts the value and compares it with the original value to figure out the accuracy.

4.2 Types of Supervised Learning

4.2.1 Classification

It is a Supervised Learning task where outputs have characterized labels. For instance, Output – Purchase has characterized marks as $0 or 1$; 1 states the client will buy and 0 implies that the client will not buy. The objective here is to foresee discrete qualities having a place with a specific class and assess based on precision. It tends to be either paired or multi-class order. In paired order, the model predicts either $0 or 1$; however if there should be an occurrence of multi-class grouping, the model predicts more than one class.

Examples of classification learning algorithm

- Stochastic Gradient Descent
- K-Nearest Neighbours
- Random Forest
- Support Vector Machine

4.2.2 Regression

It is a Supervised Learning task where outputs have constant labels; for example Output – wind speed does not have any discrete worth, however, is constant in the specific reach. The objective here is to foresee a worth as close to real yield as possible and afterward assessment is finished by figuring out an error. The more modest the loss the more noteworthy the exactness of the regression model.

$$A \cdot x = b \tag{4.1}$$

Curve fitting is the most essential of regression method, with polynomial and dramatic fitting bring about arrangements that come from settling the straight frame-

work as in Eq. (4.1). At the point when the model is not endorsed, at that point improvement techniques are utilized to choose the best model. There are different situations in a reality where we need some future forecasts like climate condition, deals expectation, promoting patterns, and so on, for such cases, we need some innovation which can make forecasts more precisely. So for such a case, we need Regression testing which is a measurable technique and utilized in AI and information science. The following are some applications of the Regression technique:

- Regression measures the connection between the objective and the autonomous variable.
- It is utilized to discover patterns in information.
- It assists with foreseeing genuine qualities.
- By tuning regression progress, we can unquestionably decide the main factor, the most insignificant factor, and the dependence of a parameter on other variables.

Examples of classification learning algorithm

- Linear Regression
- Logistic Regression
- Polynomial Regression
- Support Vector Regression
- Decision Tree Regression
- Random Forest Regression

4.3 Stochastic Gradient Descent with Momentum (SGDM)

Training of a neural network by gradient descent method is similar to the training of any machine learning model; however, the neural network has a non-linearity that causes most of the loss function to become non-convex. Therefore, it is usually

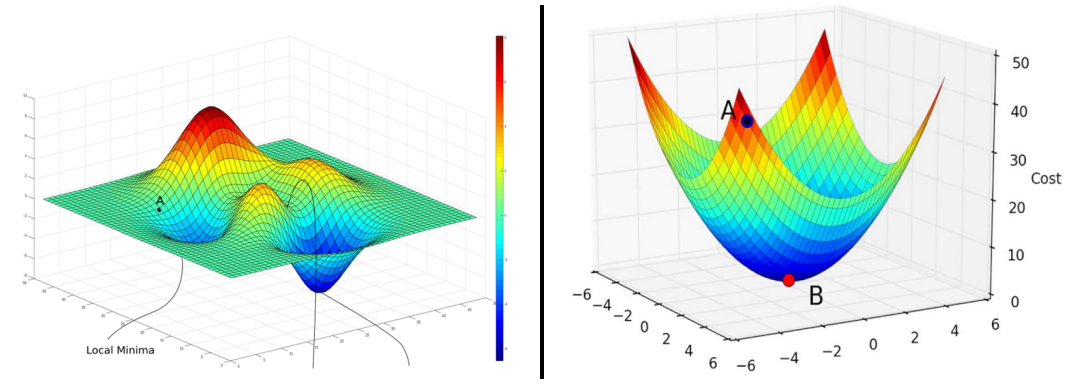
difficult to train a neural network that drives the loss to a minimum value. Additionally, stochastic gradient descent on non-convex loss function does not guarantee the convergence unlike in linear logistic regression case. Nevertheless, iterative gradient-descent-based optimization of appropriate convex function with proper initialization of parameters leads the gradient to converge to the minimum value. The stochastic gradient descent method is implemented in three steps as follows: 1. Loss and Cost Function, 2.Training, and 3.Forward and Backward propagation.

4.3.1 Loss and Cost Function

Neural network training is highly sensitive to the definition of the loss function. Intuitively, it seems that pass the training example from the network to get the output number and subtract it from the desire output number and square the result to find the loss. To exemplify, let us consider a single node neural network with the sigmoid activation function. The activation of this network is $\hat{y} = \sigma\left(w^T x + b\right)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$, x is an input, w and b are training parameters, σ is a sigmoid activation function. The network output $\hat{y}$ should be close to the desired output y for minimum loss. Therefore, as discussed earlier, the loss function is mention in Eq.(4.2).

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2 \tag{4.2}$$

The above loss function is generally used in linear regression problems, in contrast for a non-linear model the quadratic form of loss function becomes non-convex. Since there are multiple local minima as shown in Fig. 4.1(a), gradient descent can not converge to a global minimum. In this case, a convex function that plays a similar role as an error function is used to minimize the discrepancy at the output. The convex loss function is defined as follows.



(a) (b)

Figure 4.1. Cost Function (a) Non-convex (b) Convex [8].

$$L(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})) \tag{4.3}$$

The justification for choosing the logarithmic loss function as shown in Eq. (4.3) is as follows. When the desire output y is 1, the loss function become $L(\hat{y}, y) = -\log \hat{y}$. Therefore, the loss function tries to make $\hat{y}$ as large as possible, and the largest value that can be achieved of the $\hat{y}$ is 1 because the highest possible value of the sigmoid activation function is 1. On the other hand, when the desired output y is 0, the loss function is a change to $L(\hat{y}, y) = -\log(1 - \hat{y})$ such that loss function pushes the $\hat{y}$ as small as possible.

The loss functions mentioned in Eq. (4.2) & (4.3) are defined w.r.t single training example. However, the neural networks are trained on millions of training datasets, therefore the loss function is required to extend to use on multiple training data. The extension of the loss function is known as the cost function which is mentioned in Eq. (4.4). The typical surface plot of this cost function is pictured in Fig. 4.1(b).

$$J(\omega, b) = \frac{1}{m}\sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)} \log \hat{y}^{(i)} + \left(1 - y^{(i)}\right) \log\left(1 - \hat{y}^{(i)}\right)] \quad (4.4)$$

Where, m is the number of training examples in dataset, and i is an index of the training example.

4.3.2 Training

The stochastic gradient descent with momentum is one of the state-of-the-art algorithm in training neural networks. It works faster and better than the conventional gradient descent algorithm. To speed up the convergence, it uses a momentum. The momentum or velocity is defined by exponentially weighted averages of neural network parameters. It finds the moving averages of noisy data and bring them closer to the original values. As illustrated in Fig. 4.2, the yellow line is seems to be closer to the original function. The momentum of neural network parameters w and b are defined in Eq. (4.5). The momentum parameter β represent the number of values average at each iteration which can be derive by as follows $\frac{1}{1-\beta}$. Typically in learning the value of momentum parameter would be choose to 0.9, which tends it is averaging over last 10 values.

$$\begin{aligned} V_{dW} &= \beta v_{dW} + (1 - \beta) dW \\ V_{db} &= \beta v_{db} + (1 - \beta) db \end{aligned} \quad (4.5)$$

Where, V_{dW} & V_{db} are the momentum of weight and bias respectively, and β is a momentum parameter.

However, the bias is considerably high initially as shown in Fig. 4.2. The high bias value can be reduce by using bias correction term which is define as follow $\frac{V_{dW}}{1-\beta^t}$. From the equation, it is inferred that up to certain numbers of iteration the bias correction reduces the bias because the denominator is smaller and as the number of

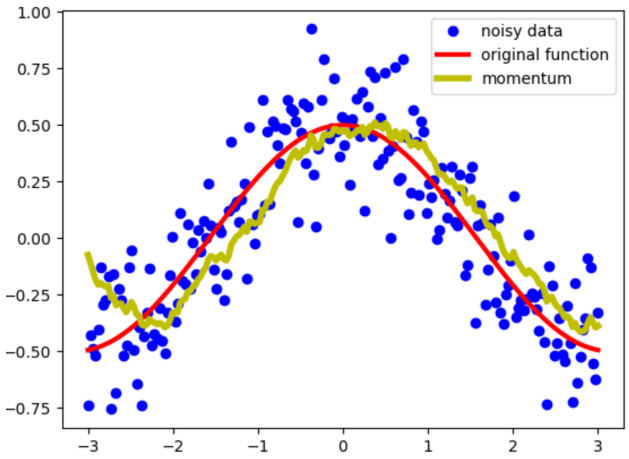


Figure 4.2. Exponentially Weighted Average of Noisy Data [9].

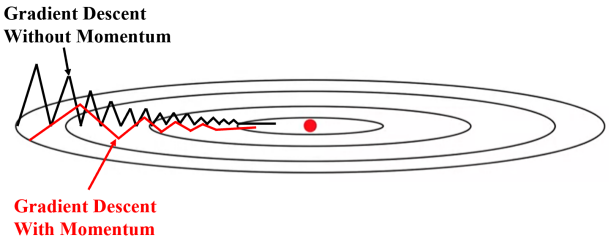


Figure 4.3. Robustness of Stochastic Gradient Descent with Momentum.

iteration increases the denominator moves closer to 1 that makes a little change in momentum.

As mention earlier, the SGDM is robust compare to other gradient descent methods which can be justified from Fig. 4.3. The black line, which represent the steps for simple gradient descent, is highly oscillatory; however, it average out the positive and negative values in oscillation which results in damping out the oscillation in vertical direction and pushes learning path towards the minima as represented by red line. The neural network parameters update after calculating momentum and bias correction is defined in Eq. (4.6).

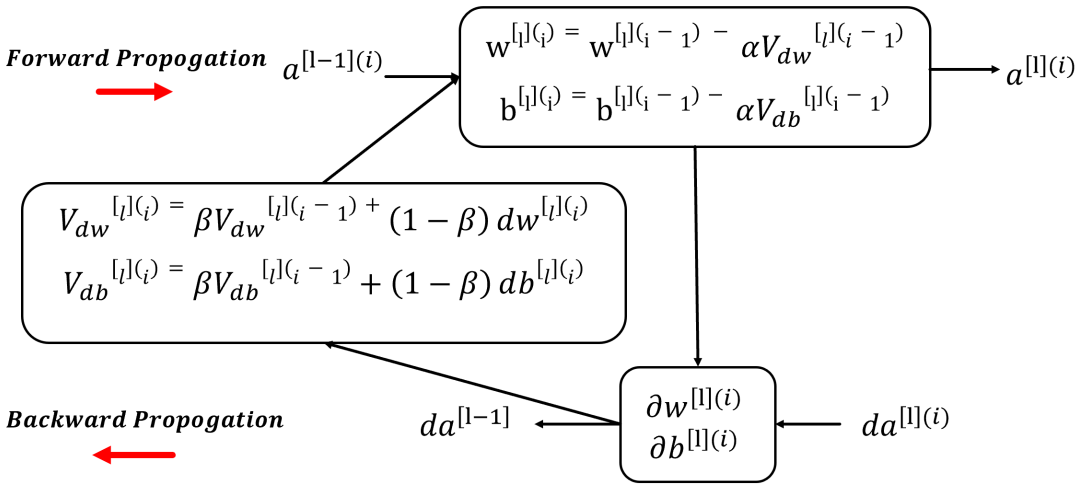


Figure 4.4. Flowchart of Forward and Backward Propagation.

$$W = W - \alpha \cdot V_{dW}$$
$$b = b - \alpha \cdot V_{db} \tag{4.6}$$

Where, α is a learning rate of training which states the size of the step.

4.3.3 Forward and Backward Propagation

In context to deep learning training, the back-propagation is a practice of fine-tuning neural network parameters based on the error from the previous iteration and forward propagation is associated with deriving the amount of error after perform tuning the neural network parameters. In the regression training, the parameters weight and bias are initialized and propagated forward to find the amount of loss at the output layer of the network. Further, the gradient of loss function w.r.t weights and biases are calculated at each layer in the backward direction. The flowchart of forward and backward propagation in stochastic gradient descent is illustrated in Fig. 4.4.

4.4 YOLO algorithm

YOLO algorithm predicts the class and bounding box of a detected object in a single run of the algorithm based on the regression method. The schematic of object detection in YOLO is shown in Fig. 4.5. It uses features from CNN architecture to classify and detect objects. As illustrated in Fig. 4.5, the output from the last convolutional layer of the base network passes through the soft-max layer to classify object categories with their precision. The same output is used for object detection.

In the detection head, the predefined grid established over the image as shown in the schematic. The object localization is performed to locate objects in each grid cells, which predicts multiple anchor boxes per grid. The number of anchor boxes that should be predicted per grid is predefined at the time of network design. The numbers of predicted anchor boxes for a cone are 4 as shown in Fig. 4.6, for example. These all predicted anchor boxes are associated with unique IoU (Intersection Over Union) values. From the non-max suppression layer, the anchor boxes with lower IoU filtered and highest IoU anchor boxes are assigned as the bounding box for corresponding objects. Finally, the coordinates of the centroid of the bounding boxes w.r.t image frame (b_x, b_y), bounding boxes height b_h, and width b_w and encoded in the output with the precision of object classification.

The YOLO object detector cannot learn the features of all bounding boxes from training data because there are millions of different sizes bounding boxes in the ground truth dataset which requires either a huge computational time or maybe impossible to train. Instead, the bounding boxes from the training dataset are grouped based on their IoU distance metric using the k-mean clustering algorithm. The estimation of anchor boxes is discussed in the following section.

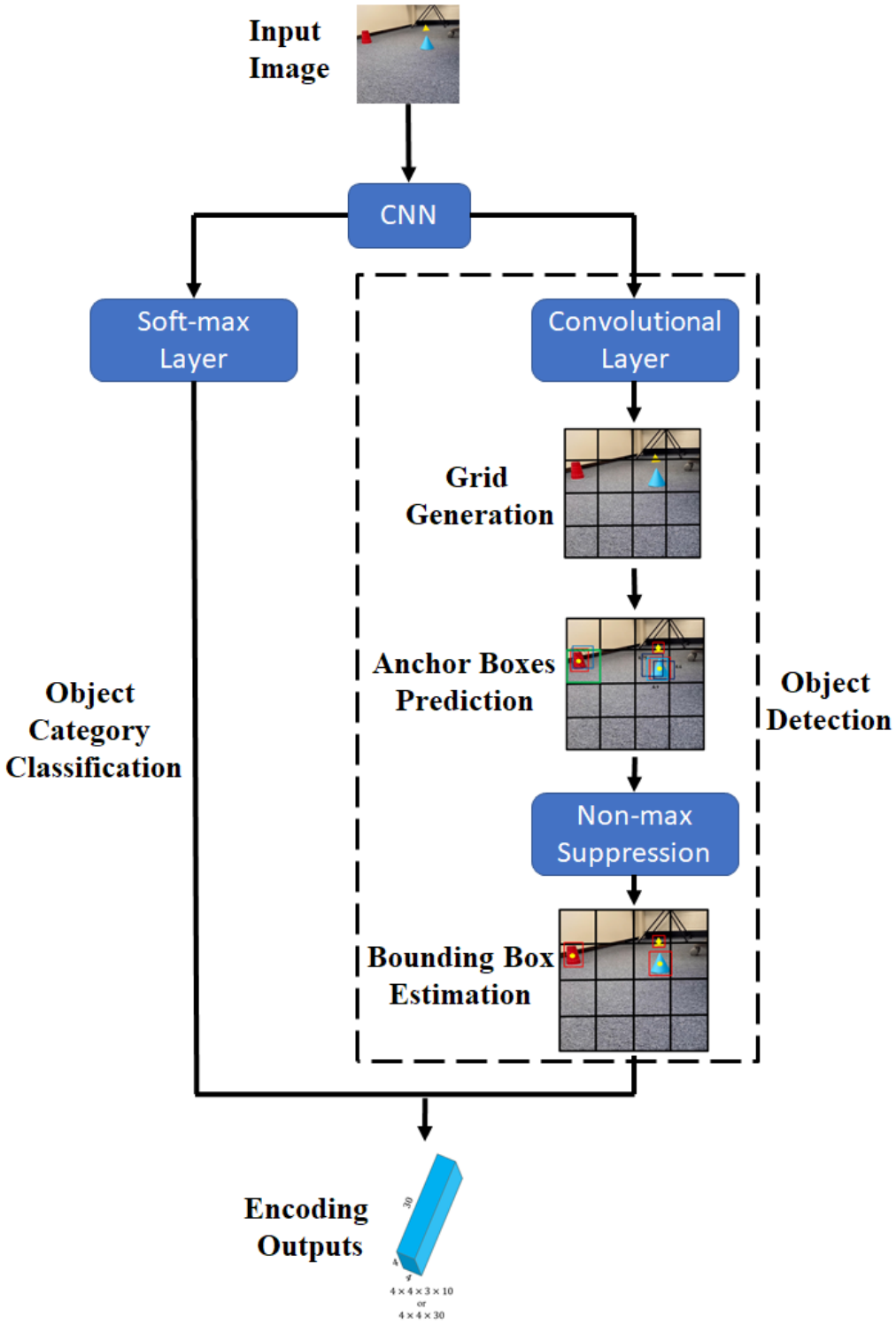


Figure 4.5. YOLO Object Detection Pipeline.

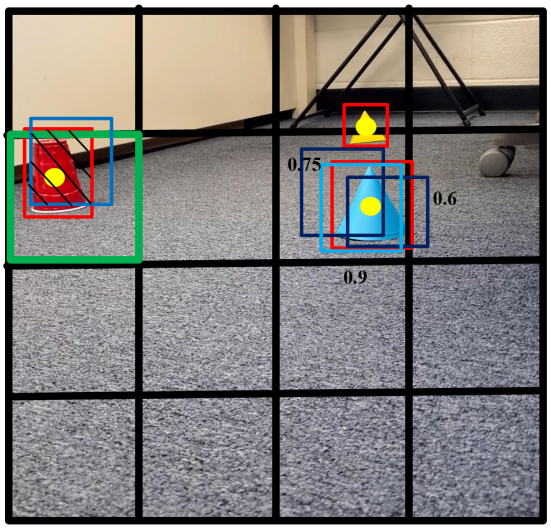


Figure 4.6. Anchor Boxes Estimation using IoU Distance Matrix.

4.5 Anchor Box Estimation

Anchor box is an important entity in deep learning, particularly for object detection applications. The number, scale, and shape of the anchor box are hyper-parameters for neural network training which are tuned iteratively to achieve high accuracy. The anchor boxes are predefined from the ground truth dataset by performing anchor box estimation using the k-mean clustering technique. The network is trained to predict the probability and refinement of the corresponding predefined anchor box and the final feature map at the output represents the refined bounding box for each classified object. Since the use of the anchor boxes allows to detection of multiple objects in a single run, the object detector can be used for real-time applications.

The anchor box estimation from the training dataset is performed by use of the distance metric based on IoU because Euclidean distance metric-based estimation produces a larger error as the size of the boxes increases [10]. Additionally, the IoU

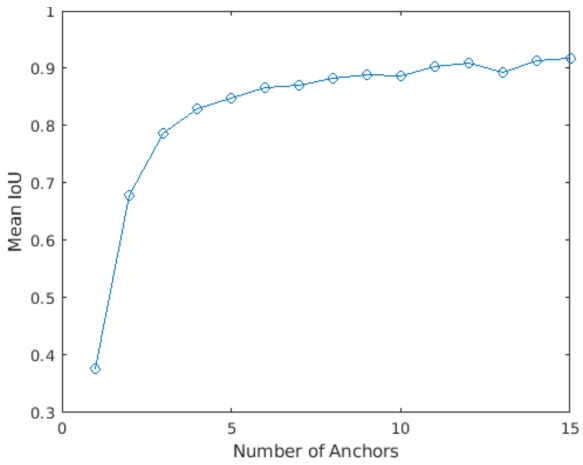


Figure 4.7. Trade off Between Number of Anchor Boxes and mean-IoU [10].

based anchor boxes estimation clusters together with the similar aspect ratio boxes that fit the ground truth data.

As mentioned earlier, the number and mean-IoU required for the estimated anchor boxes are the hyper-parameter of the neural network. From the imperial result analysis, it has been seen that the number of anchors boxes is proportional to the mean-IoU of the boxes as represented in Fig. 4.7. The high value of mean-IoU improves the precision in the bounding box prediction and larger mean-IoU achieved by increasing the number of anchor boxes, however, a large number of anchor boxes leads to an increase in computation cost, and some cases may result in over-fitting.

Chapter 5

Hyper-parameters Tuning

5.1 Introduction

In deep learning, hyperparameter tuning or optimization is an iterative process of searching a set of optimal values of hyperparameters to minimize the loss of the model training. Hyperparameters are the parameters that implicitly govern the learning process by controlling the training parameters' weight and bias. Unlike training parameters, hyperparameters are not trained during the learning process, but they are predefined manually by empirical data.

However, there are few approaches for hyperparameters optimization such as trial & error, grid search, random search, and Bayesian optimization. Such an approach finds the set of hyperparameters that yields modal optimality and minimizes the loss function. The optimization of hyperparameter is a highly iterative process that is constrained by time consumption, the number of training data available, system requirements, and available resources.

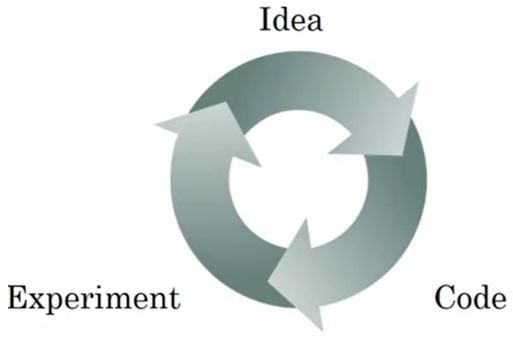


Figure 5.1. Loop of Hyperparameter Tuning Process [6].

5.2 Approaches for Tuning Hyperparameters

All four approaches are briefly explained as follows.

5.2.1 Trial & Error

This approach is completely manual in which any value of hyperparameter is either selected by random guess or based on empirical results. If the selected values do not considerably minimize the training and testing loss, another value will be guessed based on the empirical relation between the corresponding hyperparameter and loss function. Such an approach is generally adopted by students and researchers because of the limited availability of industry-standard equipment to perform complex and time-consuming experiments. The iterative loop of this approach is pictured in Fig. 5.1. As per the loop, firstly the best guess for the hyperparameters is made, implement and finally assess the results and reiterate the process until the results are not convincing.

5.2.2 Grid Search

It is a naive approach to try out every possible composition of hyperparameters. Initially, the n dimension grid is defined, where the n dimension equals the number of hyperparameters that need to be tuned. For each dimension, define all possible values that need to have experimented with, perform training on all possible configurations and repeat until the best match is established. This technique consumes a tremendous amount of time if the size of the grid is large. However, the time consumption can be reduced if multiple computational resources are available. Usually, it is adopted when the dimension of the grid is less than or equal to 4, and as the dimension increases the time complexity increase exponentially.

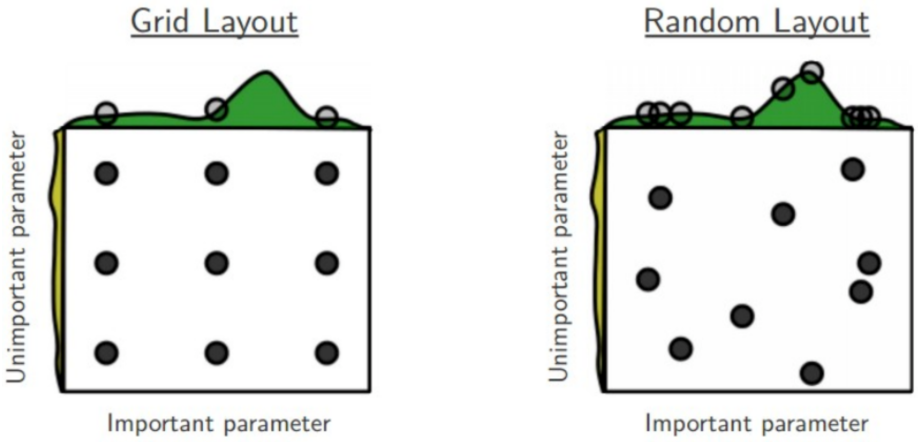


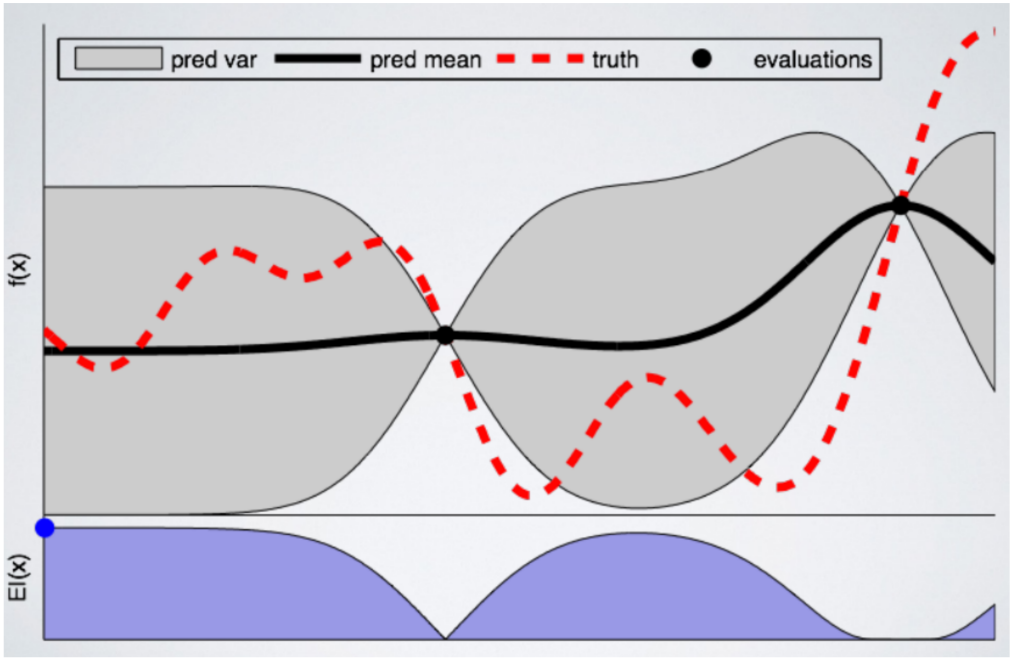
Figure 5.2. Configuration Space Layout of Grid Search and Random Search Approaches [11].

5.2.3 Random Search

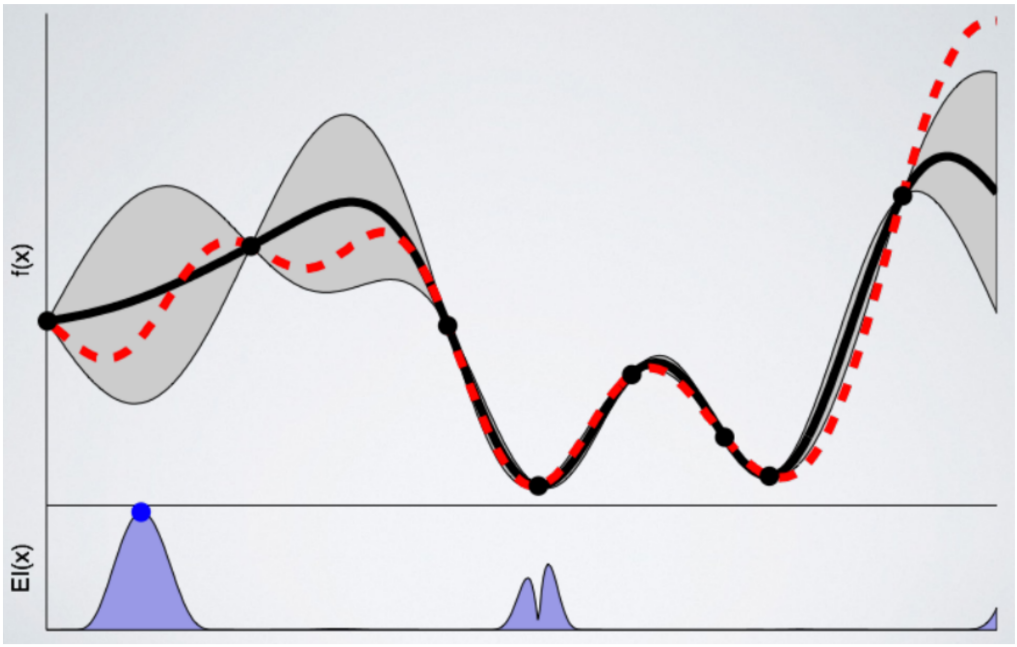
The main difference in this approach is the point is picked up randomly from configuration space. The graphical representation of the comparison between grid search and random search is shown in Fig. 5.2. From the above illustration, it is noticed that grid search uses only 3 values parallelly. Whereas, the random search evaluates total 9 models using 9 different values. The hyperparameters space exploration is wider in the random search than the grid search which helps find the best values in a lower number of iteration. Therefore, it is used when configuration space has high dimensionality.

5.2.4 Bayesian Optimization

This is an autonomous approach that adept the configuration space of hyperparameters at each iteration based on model evaluation from the last iteration. At each iteration, the surrogate model becomes more and more confident that yields the minimum training loss. The Gaussian process is used as the surrogate to predict the mapping from hyperparameter configuration to the region of interest, additionally, it



(a)



(b)

Figure 5.3. Bayesian Optimization Process for (a) 2 Trained Model, and (b) 8 Trained Models [12].

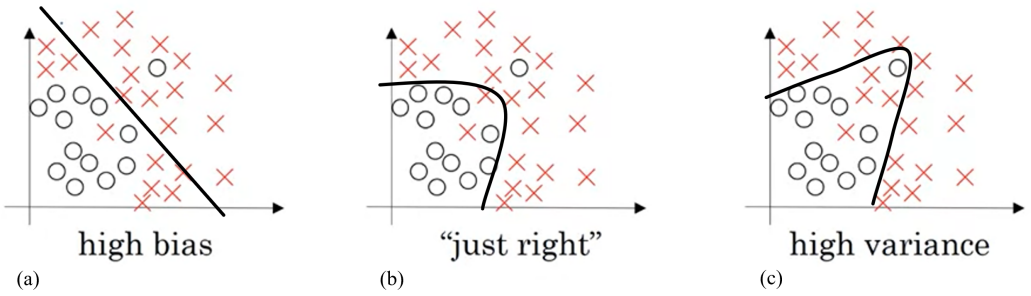


Figure 5.4. Classifier Behaviour for Various Bias and Variance Conditions [6].

also derives the range of uncertainty of weight and bias. The Bayesian optimization process after 3 and 8 model evaluation are shown in Fig. 5.3(a) and Fig. 5.3(b) respectively. The uncertainty is high in the areas where little information available. After each model evaluation, it adapts the new point (blue dot) on which the training will perform. As the number of trained models increases, the confidence of the surrogate will surge. However, this approach is only used for numerical hyperparameter, and it is impossible to stop earlier if the training progress is poor.

5.3 Bias and Variance Trade-off

Bias and Variance are the main sources of an estimation or prediction error. Bias is the error due to deviation of estimation from the true value of a parameter, on the other hand, variance is the error due to deviation of measurement from expectation. In other words in the context of deep learning, the bias is the error due to the difference between training error and ideal error (usually 0%). And the variance is the error due to the difference between testing and training accuracy. For instance, the bias will dominate the error if the training error is large, on the flip side the variance will be big if the difference between training and testing error is large as mentioned in table 5.1. In the third case, if the training error is high and the testing error is even

Training Error	Testing Error	Remarks
1 %	11 %	High Variance
15 %	16 %	High Bias
15 %	30 %	High Variance & Bias
0.5 %	1 %	Low Variance & Bias

Table 5.1. Effect of Variation in Train and Test Error on Bias and Variance

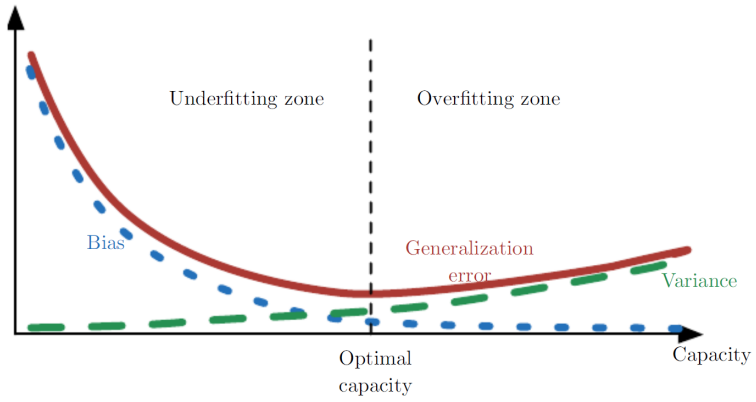


Figure 5.5. Relationship between Bias and Variance [13].

worst, the trained model has both high variance and high bias. A simple classifier's behavior is shown in Fig. 5.4. The highly linear classifier in Fig. 5.4 (a) has a high bias that under-fits the data, whereas the highly non-linear classifier in Fig. 5.4 (c) has a high variance that over-fits the data. However, the classifier in Fig. 5.4 (b) is reasonably fit to the data and it has just the right characteristic of non-linearity that prevents it from over-fitting.

The bias and variance are tightly connected and they are inversely proportional to each other as seen in Fig. 5.5. As per the illustration, the meaning of increase the capacity of the model is to reduce the bias and rise the variance. As the capacity is gradually rising, the model training moves from the under-fitting region to an optimal point where the generalized error is optimum. The capacity of a model can be increase by using a deeper network by adding more layers, as result, the overall

bias is considerably diminished. On the contrary, the model variance can be narrow down by using the larger dataset and use a similar source for train and test data which efficiently lessens the variance without affecting the bias.

5.4 List of Tunable Hyperparameters

There is a list of tunable hyperparameters that use to regularize the deep learning model; however, we only discuss the hyperparameters that are used in this study in the following subsections.

5.4.1 L2 Regularization

By regularization means to prevent the model from overfitting the data. For the object detection application in deep learning, there is a small amount of training dataset available based on the capacity (size) of a neural network. Therefore, overfitting of a network is a common problem in object detection. Such a problem can be used by regularizing the neural network.

Particularly, in the L2 regularization, the weight metric is penalized by the definition of L2 regularization parameter (λ). In the definition of the loss function, the additional term added that known as Frobenius norm or second norm as written in Eq. (5.1). By choosing a bigger value of λ prevents over-fitting, but selecting higher than the required value tends to increase the bias and results in under-fitting, hence the L2 regularization parameter needs to be tuned to prevent from over-fitting consequently taking care of bias.

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} L\left(a^{(i)}, y\right) + \frac{\lambda}{2m}\|w\|_2^2 \tag{5.1}$$

Where, $\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T \cdot w$, λ is a regularization parameter, J is a cost function, a is an activation from corresponding layer, m is the number of training data, L is a loss function, y is a true or desire output from ground truth data, and i is an iteration index.

$$\begin{aligned} \omega^{[l]} &= \omega^{[l]} - \alpha \left[(\text{from back-prop}) + \frac{\lambda}{m} w^{[l]} \right] \\ &= \omega^{[l]} - \frac{\alpha\lambda}{m} \cdot \omega^{[l]} - \alpha(\text{back-prop}) \\ &= [1 - \frac{\alpha\lambda}{m}] \cdot \omega^{[l]} - \alpha(\text{back-prop}) \end{aligned} \tag{5.2}$$

The primary intuition behind L2 regularization is explained by Eq.(5.2). By using regularization, the update step for weight metric in forward propagation is modified as seen in Eq.(5.2). From this equation, it has been seen that the weight metric is decay by a factor of $\frac{\alpha\lambda}{m}$, thus L2 regularization is also called weight decay.

5.4.2 Data Augmentation

Data augmentation is a method to improve network accuracy by randomly transforming the training data. Before training, the training dataset is pass through the data augmentation function to generate random transformations such as adding color jitter in HSV color space, flipping horizontally, and randomly scaling up to 10 or 10% as shown in Fig. 5.6. It is another way of increasing the training dataset.

5.4.3 Mini-batch Size

In deep learning, the size of the dataset is usually large, thus a single iteration in training takes a lot of time. Instead, the training data is divided into small sizes of mini-batches. By transforming training data into mini-batches to train the network, the speed is quite faster than the complete batch of data. However, the number of

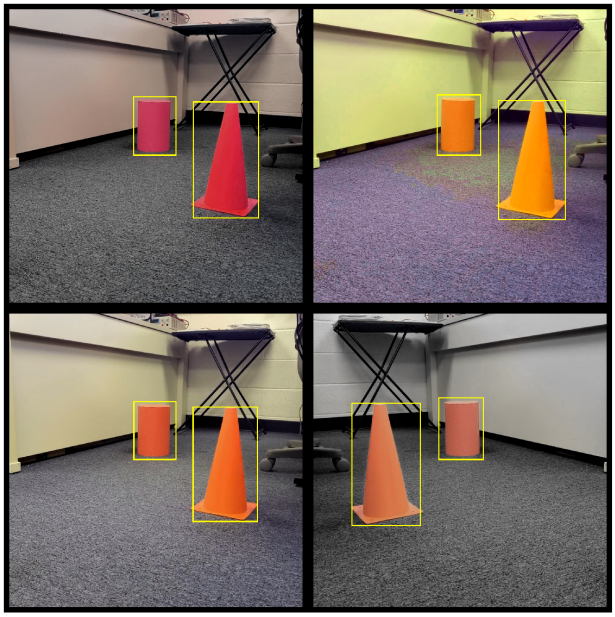


Figure 5.6. Training Data Augmentation.

training examples per mini-batch is a decisive factor in training accuracy. The small mini-batch size is extremely noisy since new data is passed at each iteration. On contrary, the large mini-batch size is extremely slow in convergence. Therefore, the mini-batch size is selected in a way that the learning progresses with enough speed and a considerable amount of noise.

5.4.4 Number of Epoch

The number of an epoch is a hyperparameter that defines the number of times the training algorithm passes through the complete dataset. Each epoch states the training batch has a chance to improve the internal training parameter. Traditionally, the number of epoch should be large enough that allows an algorithm to run until sufficient accuracy is achieved.

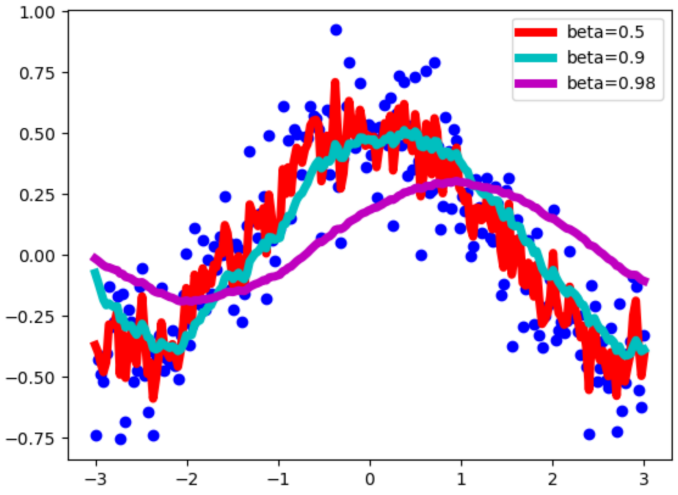


Figure 5.7. Exponentially Weighted Averages for Different Values of β [9].

5.4.5 Momentum

The momentum parameter is used in the gradient descent algorithm to speed up the process by utilizing the moving average. The velocity in the training process is defined in Eq.(5.3), which is an exponentially weighted average of last n velocities where n is the number of samples considers in evaluating average at each iteration. The value of the momentum parameter (β) is carefully selected by which the sequential plot of weighted average fits on noisy data. In Fig.5.7, the behavior of exponentially weighted average for different value of momentum parameter is illustrated. Since the lower value of β averages a lesser number of samples, the plot for $\beta = 5$ is much noisy.

$$V_t = \beta V_{t-1} + (1 - \beta) S_t \tag{5.3}$$

5.4.6 Learning Rate

Learning rate is a hyperparameter that represents the step size of the gradient in direction of convergence. This is an important parameter in training because the accuracy in training is greatly dependant on the amount learning rate. Usually, the learning rate is selected to be 0.001 for training small to medium size neural network, however, if the large size network needs to training on a complex dataset, the variation in learning rate is adopted as the training progresses. There are few approaches used to decay the learning rate online. One of them is discrete step decay, in which the larger value is selected initially and is reduced at every predefined number of iterations.

Chapter 6

Multi-sensor Data Fusion

6.1 Introduction

Multi-sensor data fusion is a multilevel, multifaceted process dealing with the detection, association, correlation, estimation, and combination of data and information from multiple sources to achieve refined state and identity estimation, and complete and timely assessments of situations and threat as define in [43]. The primary objective of sensor data fusion is to combine sensory data from multiple disparate sources such that the outcome is less uncertain than the individual sensory information.

When the information from more than one sensor is identical, a multi-sensor data fusion reduces the overall uncertainty of information, and if the information is not similar then it combines all information to make complex decisions that can not be possible by the use of a single sensor. In such a case, each sensor becomes complementary to the other.

6.2 Types of Multi-sensor Data Fusion

Based on the functional level of information abstraction, the data fusion is divided into three types as follows. 1. Feature Level, 2. Position Level, and 3. Attribute Level.

6.2.1 Feature Level Fusion

It is a fusion that performs directly on object features. The sensory system is responsible for on-board computation to detect features from the environment, therefore the overall computational load on the system is reduced. The correlated features further fuse to observe the trend of entity of interest for instance weather forecasting, organizational management, detect the fault in mechanical systems, and medical diagnosis.

6.2.2 Position Level Fusion

Position level fusion aims to perform fusion directly on the observation data of the sensor or estimation of states in time and space. It is the most common sensor fusion method that has been used in many applications. It takes information from homogeneous sources and performs fusion to increase certainty. This type of sensor fusion has to deal with big input spaces because raw information from more than one source creates huge information space. Thus, the overall performance of the framework slows down as the number of sources increases. If there is any malfunction in a sensor modality, then the whole system generates ambiguous output.

6.2.3 Attribute Level Fusion

It selects the hypothesis or attributes from a set of presumptions generated by individual decision nodes. It is a high-level extraction of information that already has been reprocessed through feature-level processing. Typically, this type of fusion is used in target classification and recognition tasks. The outcome of this framework is improved accuracy in decision-making. Based on the level of information abstraction, it is divided into three layers as follows: decision-making layer, feature layer, and

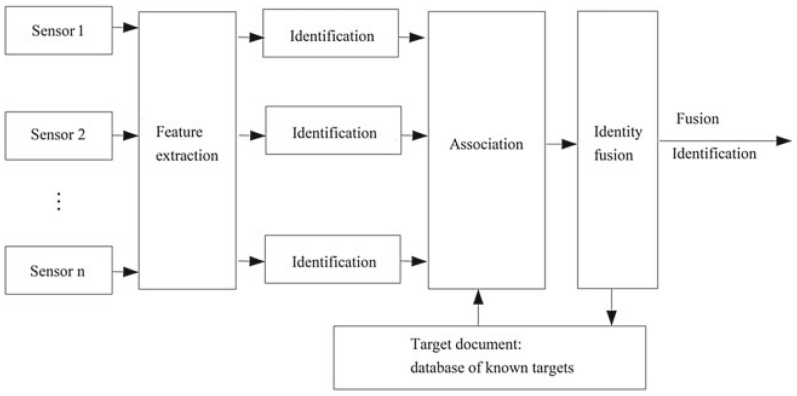


Figure 6.1. Attributes Identification Flow Chart [14].

data layer fusion. The common flow chart of target attributes identification is shown in Fig. 6.1.

6.3 Frameworks of Position Level Fusion

In position level multi-sensor data fusion, there are three types of fusion architectures such as centralized, distributed, and compound or hybrid structure which are explained in the following subsections.

6.3.1 Centralized Fusion Architecture

In the centralized fusion structure, the raw data from the discrete sources are directly fed into the fusion framework. Firstly, the time fusion is performed on the time sequence of target observation and then multi-sensor spatial fusion is performed at the same time. It is used for object tracking and state estimation. The flow chart of the centralized fusion structure is shown in Fig. 6.2.

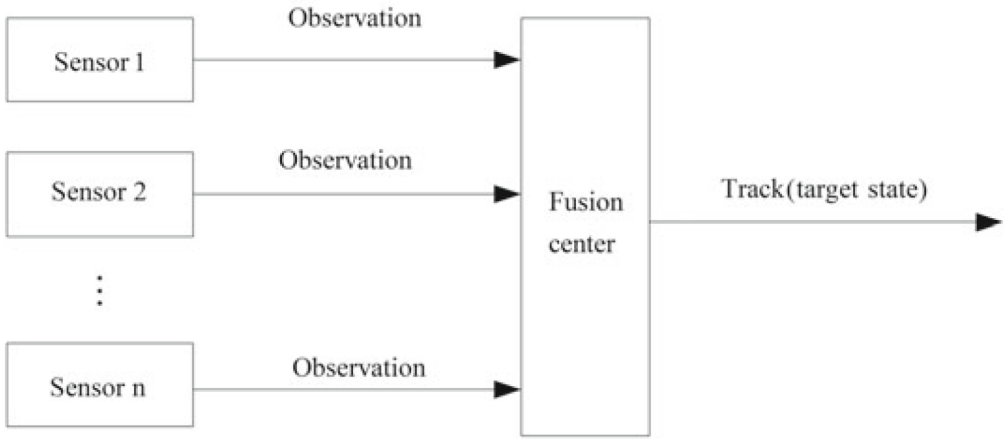


Figure 6.2. Centralized Fusion Architecture [14].

6.3.2 Distributed Fusion Architecture

In the distributed tracking system, two levels of tracking are performed. Each sensor has its tracker to perform local level tracking, and all these tracking estimations fed into a global tracker or central level tracker to synthesizes the local estimation into a joint target estimation. The advantages of such type of architecture are it requires low communication bandwidth, fast computation speed, reliable and continuity. The schematic of distributed fusion structure is shown in Fig. 6.3.

It is further divided into three types based on feedback from the central tracker such as distributed system without feedback, distributed system with feedback, and fully distributed. The first type is the most common type of architecture in which if any local tracker failure occurs it degrades the overall system performance. In the distributed system with feedback, the central node is fed back to each local node. The main advantage of such a structure is to detect failure or to supervise the local nodes. Furthermore, in the fully distributed structure, all the local nodes are connected with each other and the central node.

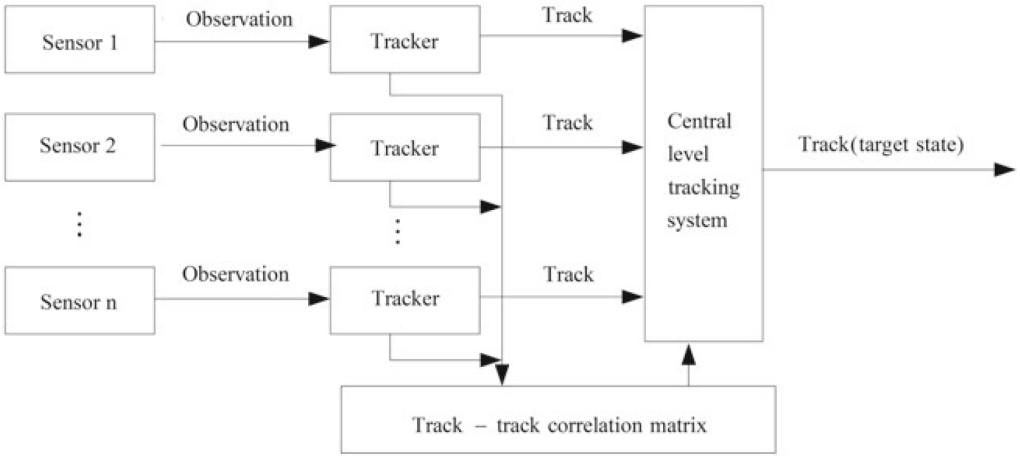


Figure 6.3. Distributed Fusion Architecture [14].

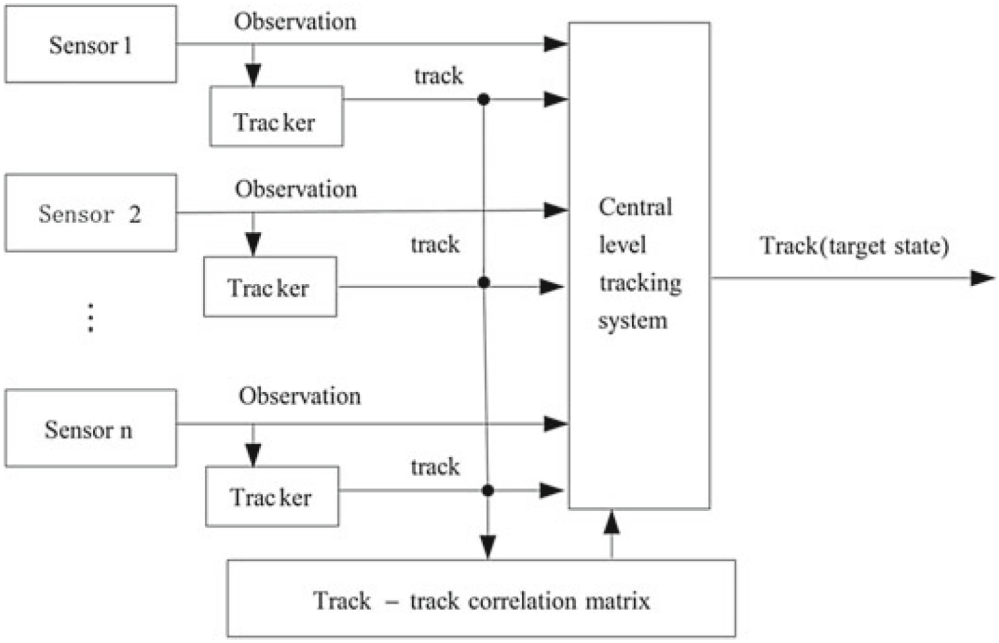


Figure 6.4. Compound Fusion Architecture [14].

6.3.3 Compound Fusion Architecture

Compound fusion structure is a combination of centralized and distributed fusion framework where the detection report from the sensor modality with tracking information from the local nodes are sent to the central node as shown in Fig. 6.4. Such type of structure requires an advanced processing logic because it synthesizes the detection report and tracking information in addition to optimizes combinations. Although it requires complex communication and long processing time, it is used in many applications such as cruise missile control, and radar-based guidance system.

6.4 Linear Kalman Filtering

The Kalman filter (KF) is a Bayesian filter that uses on Gaussian probability density function to estimate the target state. The target state is considered as a random variable. By defining the dynamic model and measurement model of the system, the Kalman filter describes the relationship between target state and measurement. Based on the minimum mean square error estimation criterion, the Kalman filter is an optimal filter. It performs the sequential calculations to predict the target state by using a dynamic model and further update it or make corrections in prediction by using measurements.

The implementation of a linear Kalman filter for the discrete-time dynamic system is as follows.

$$x_k = F_{k-1}x_{k-1} + G_{k-1}u_{k-1} + w_{k-1} \tag{6.1}$$

$$z_k = H_k x_k + v_k \tag{6.2}$$

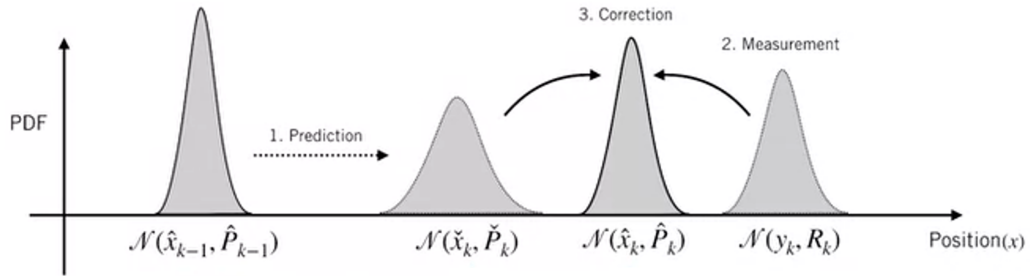


Figure 6.5. State and Co-variance Propagation in Prediction and Correction Steps [15].

Where, x_k is a state vector at time stamp k, F_{k-1} is a state transition metric, G_{k-1} is a control gain, u_{k-1} is a control input, w_{k-1} is a process noise at time $k-1$, z_k is a measurement vector from sensor at time k, H_k is a measurement metric, and v_k is a measurement noise.

The measurement noise v_k and process noise w_{k-1} are considered as a zero-mean uncorrelated white noise. The measurement co-variance R_k and process co-variance Q_k are defined as follow.

$$Q_k = Ew_i w_j^T = \begin{bmatrix} q_{xx} & 0 & 0 \\ 0 & q_{yy} & 0 \\ 0 & 0 & q_{zz} \end{bmatrix} R_k = Ev_i v_j^T = \begin{bmatrix} r_{xx} & 0 & 0 \\ 0 & r_{yy} & 0 \\ 0 & 0 & r_{zz} \end{bmatrix} \tag{6.3}$$

The state initialization x_0 is assumed to be follow the Gaussian distribution with mean $\bar{x_o}$ and co-variance P_0.

Prediction: In the prediction step, the state estimation of previous time stamp x_k is substituted in dynamic model to propagate the state forward at discrete time stamp $k+1$ as $x_{k+1|k}$. Similarly, the prediction co-variance is also propagated as $P_{k+1|k}$.

$$
\begin{aligned}
x_{k+1/k} &= F_k x_{k/k} + G_{k/k} u_k \\
P_{k+1/k} &= F_k P_{k/k} F_k^T + Q_k
\end{aligned}
\tag{6.4}
$$

Correction: In the correction step, estimated states are corrected by using measurements through feedback. It means the discrepancy between predicted measurement $x_{k+1|k}$ and actual measurement y_{k+1} is added to the state estimation after multiplying with feedback gain metric K_{k+1} (Kalman Gain) and the estimation covariance also corrected as mention in Eq.(6.6).

$$
K_{k+1} = P_{k+1/k} H_{k+1}^T \left(H_{k+1} P_{k+1/k} H_{k+1}^T + R_{k+1} \right)^{-1} \tag{6.5}
$$

$$
\begin{aligned}
x_{k+1/k+1} &= x_{k+1/k} + K_{k+1} \left(\tilde{z}_{k+1} - H_{k+1} x_{k+1/k} \right) \\
P_{k+1/k+1} &= \left(1 - k_{k+1} H_{k+1} \right) p_{k+1/k}
\end{aligned}
\tag{6.6}
$$

6.5 Multi-Sensor Data Fusion

In multi-sensor data fusion implementation, the sequential update of measurements from more than one sensor can be done in two ways based on sensor synchronization.

1: Synchronous Case In an asynchronous case, a measurement from each sensor available at each sampling time. In this case, the measurements from all sensors are stacked and simultaneous measurement update is carried out.

2: Asynchronous Case In this case, the measurements are available asynchronously at each timestamp. The prediction and update steps are carried out in the order in which the measurements are obtained. A flowchart for camera and LiDAR data fusion is illustrated in Fig. 6.6.

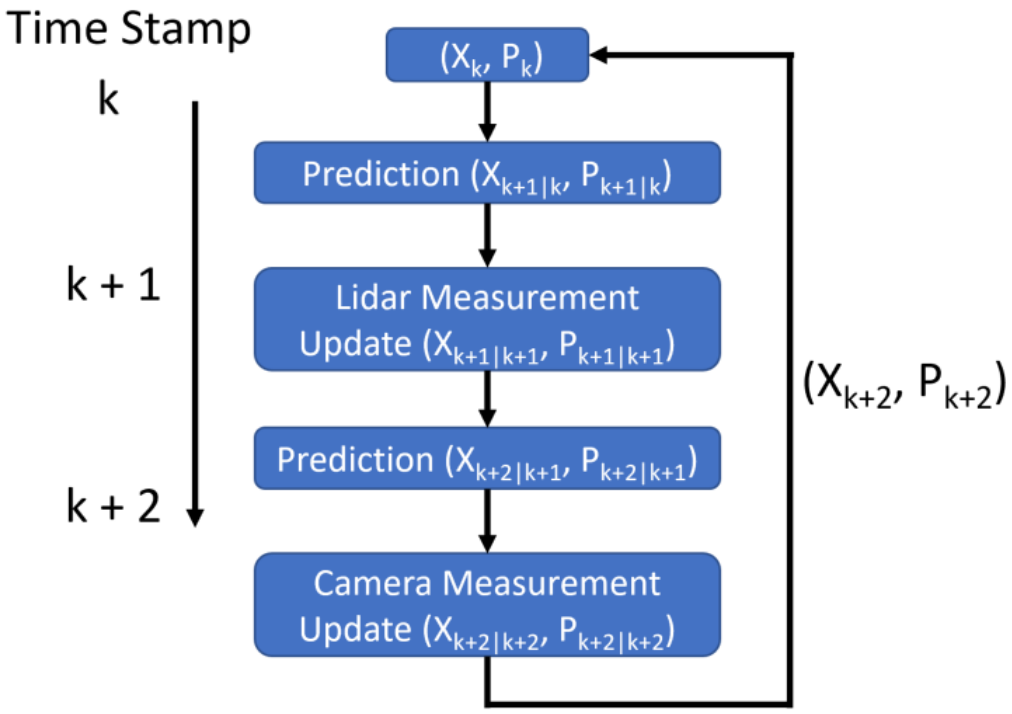


Figure 6.6. Flowchart of Multi-sensor Data Fusion for Asynchronous Case.

Chapter 7

Results, Discussion and Future Scope

7.1 Experimental Setup

The mobile robot platform used in this study is custom-tailored by fellow students at Aerospace System Laboratory at the University of Texas at Arlington. It is made from aluminum chassis with fours wheel driven by four gear motors at each wheel. It is equipped with an aluminum shock-absorbing suspension to be used in rough terrain. The rover is equipped with an Intel real sense L-515 LiDAR camera and a stereo camera with two Sony IMX 179 image sensors. Intel NUC is used for on-board processing. The mounting of these optical sensors and Intel NUC are 3D printed from Acrylonitrile Butadiene Styrene (ABS) material. The complete setup is illustrated in Fig.7.1.

7.2 YOLO-v3 Supervised Training

The YOLO-v3 network was trained on custom training data-set with total number of 10989 images. The training data is created to classify and detect four different classes of objects as follows cylinder, Traffic cone, pyramid, and box. Supervised training is performed on the structured dataset which was organise in the form of $y = f(x)$ in order to find the prediction function $\hat{y} = f(x)$ with minimal error $e = min(\hat{y} - y)$. The optimal values of the hyperparameter set for this training are given in table 7.1.

The ground truth dataset is divided into two batches, training and testing batch. From the empirical results, it has been suggested to use few hundreds of image data for the test batch. Since the size of available ground truth data is a total of 10989,

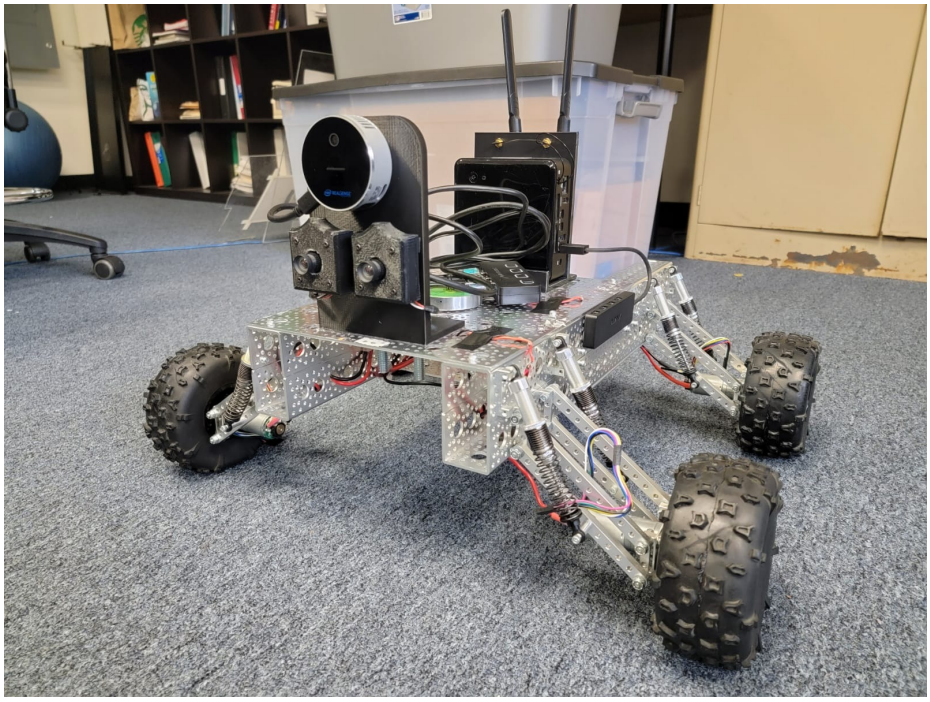


Figure 7.1. Wheeled Mobile Robot equipped with LiDAR and Stereo Camera.

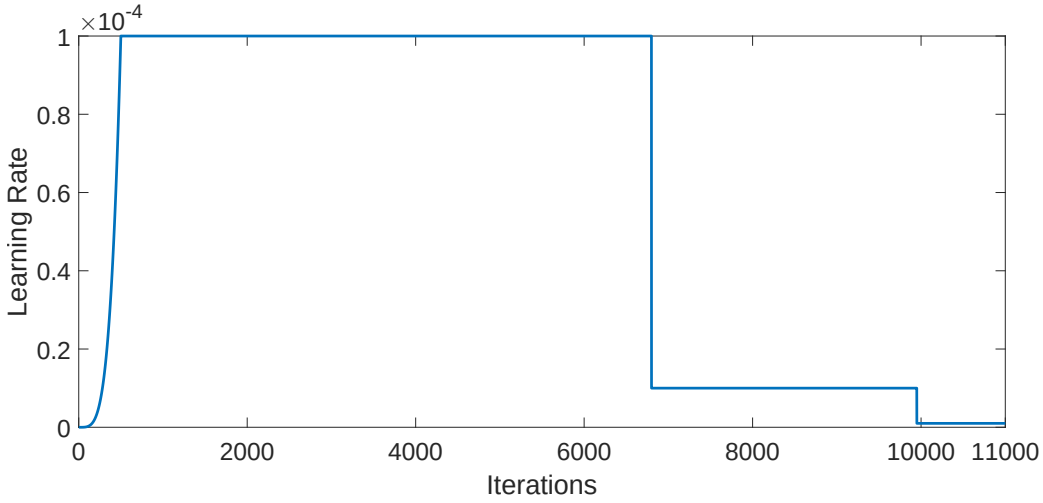


Figure 7.2. Learning Rate Variation During Training.

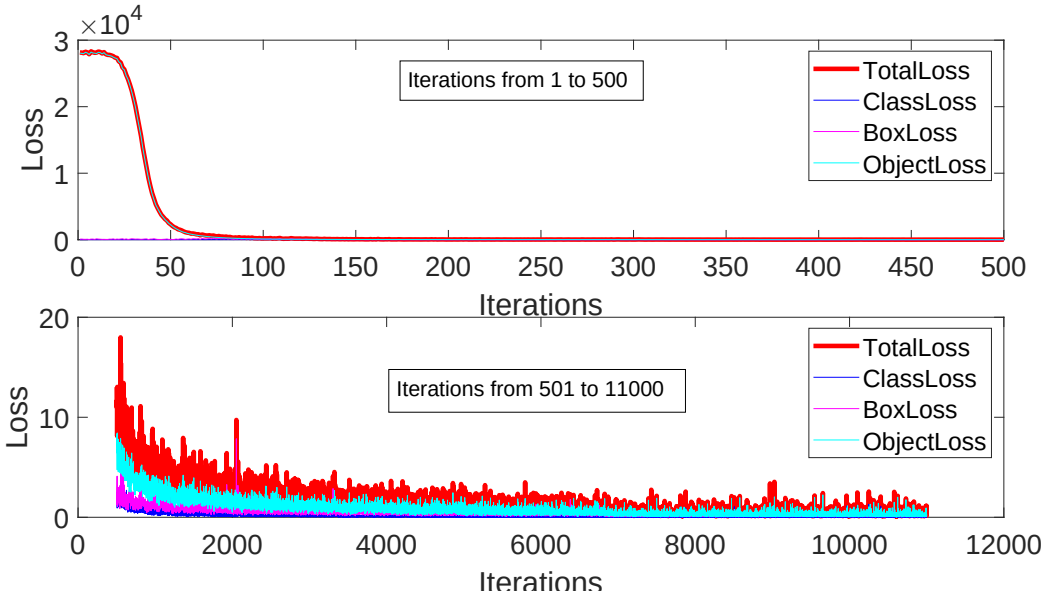


Figure 7.3. Loss Reduction During Training.

the size of the test batch is selected to be 2% of total data available and the rest of the data is allocated to the training batch.

Before the start of the training, the bounding boxes from the ground truth data are used as a reference to estimate the anchor boxes, using the k-means clustering algorithm, that precisely fit the training data. The mean IoU of the estimated anchor boxes is 0.8738, this is because of the 12 anchor boxes per detection head (Refer Section 4.5).

The training is performed in two parts. Initially, the learning rate is low in and is exponentially increased to the preset optimal learning rate till the number of iterations allocated for warming up phase as shown in Fig. 7.2. The warm-up period is set to prevent the primacy effect in training. In the initial phase of learning, the SGDM algorithm calculates the gradient based on the initial batches of data which is not accurate and may diverge from the direction to global optima. It can be observed

Hyperparameter Name	Optimal Value
Numbers of Iterations	11000
Learning Rate	0.0001
Warm-up Period	500 iterations
L2 Regularization Parameter	0.0005
Mini-batch Size	10
Train batch	98%
Test batch	2%
Mean IoU of Anchor Box Estimation	0.8738

Table 7.1. Optimal Value of Hyperparameter Set Considered in Training

from Fig. 7.3 that the loss function value is 30000 initially, and decreases to 5.46 in this period.

In the second part of training, the learning rate decay is predefined in which the learning rate reduces in discretized steps after a certain number of iterations as seen in Fig.7.2. When near to the end of learning or when the gradient is close to the global optimum, it is suggested from the empirical studies to take small steps toward the global minimum to prevent the gradient to swing around the global minimum. In this part, the network parameters are tuned so that the network can precisely classify and detect the objects.

As seen in Fig.7.3, the training is highly noisy because of the small mini-batch size. However, for the higher value of mini-batch, the training cannot be performed due to insufficient graphics memory of the system. Therefore, the optimum value of mini-batch size is 10 based on the availability of the computational resources. On the other hand, the training on CPU provides flexibility to select a mini-batch size of our choice, although it is expensive in terms of computational time. In this case, training on CPU may take 5-7 days approximately while on GPU it takes 7-8 hours.

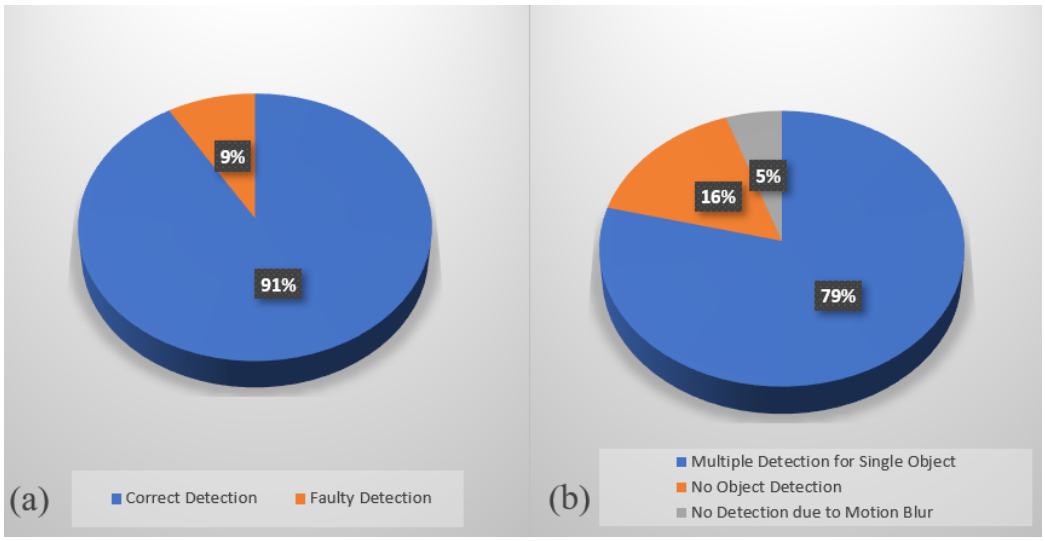


Figure 7.4. Pie Chart of Test Results (a) % of Faulty Detection and (b) % of Faulty Detection due to Various Reasons.

7.3 Neural Network Testing

Testing of the neural network is performed on the data which is never seen by the network during training. Therefore, 2% of the ground truth data was kept away for the network performance evaluation. In testing, the trained YOLO-v3 network correctly detects the objects from 201 test images out of 220 images. The pie-chart in Fig.7.4(a) illustrates the distribution of test data into correct and faulty detections; additionally, the pie-chart in Fig.7.4(b) shows the distribution of sources of error for faulty detections. The major reason for defective detection is multiple detections for single objects. Few other reasons for defective detections could be the network not detecting the object and motion blur. The correct detection results are shown in Fig.7.7 with their performance parameters in table 7.3, and faulty detection examples are shown in Fig.7.8.

There is a small difference between testing and training accuracy as shown in Table 7.2. It represents the network has a small variance during the training.

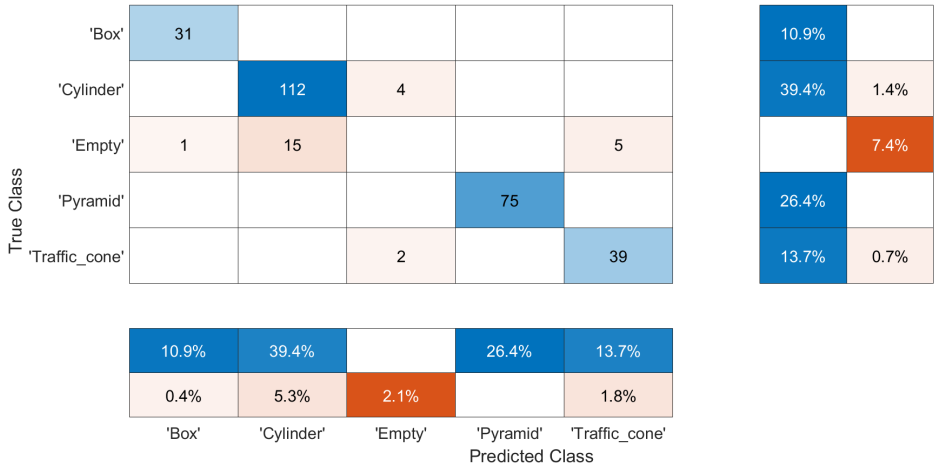


Figure 7.5. Confusion Chart of Classification on Test Batch.

Therefore, it is inferred that the network over-fits on some outliers. However, the difference between test and train accuracy is not too large so such outliers do not degrade the network performance. Additionally, the mean IoU of the test data is 0.81, which shows the ability to accurately predict the bounding box to localize the objects within the frame. The overall precision for each class of object is shown in Fig. 7.6. In the test period, the network seems to be robust, and the average processing time is 59 ms per frame.

The confusion chart is shown in Fig.7.5 for the classification result of the test batch. The network can identify 91% of the objects. The empty label on prediction axis represents that the object has a label in ground truth data but was not detected by the network; on the flip side, the empty label on the true classes axis states that the object does not have a label in ground truth data because it is partially visible in the frame but it was detected by the network. However, it can be noted that the trained network has never done the incorrect classification during the testing.

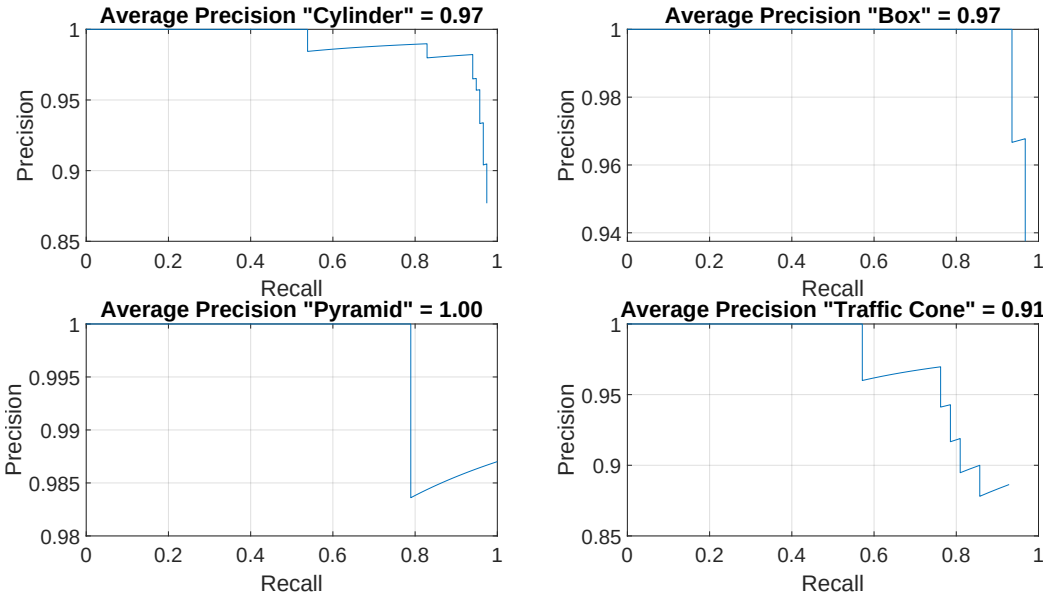


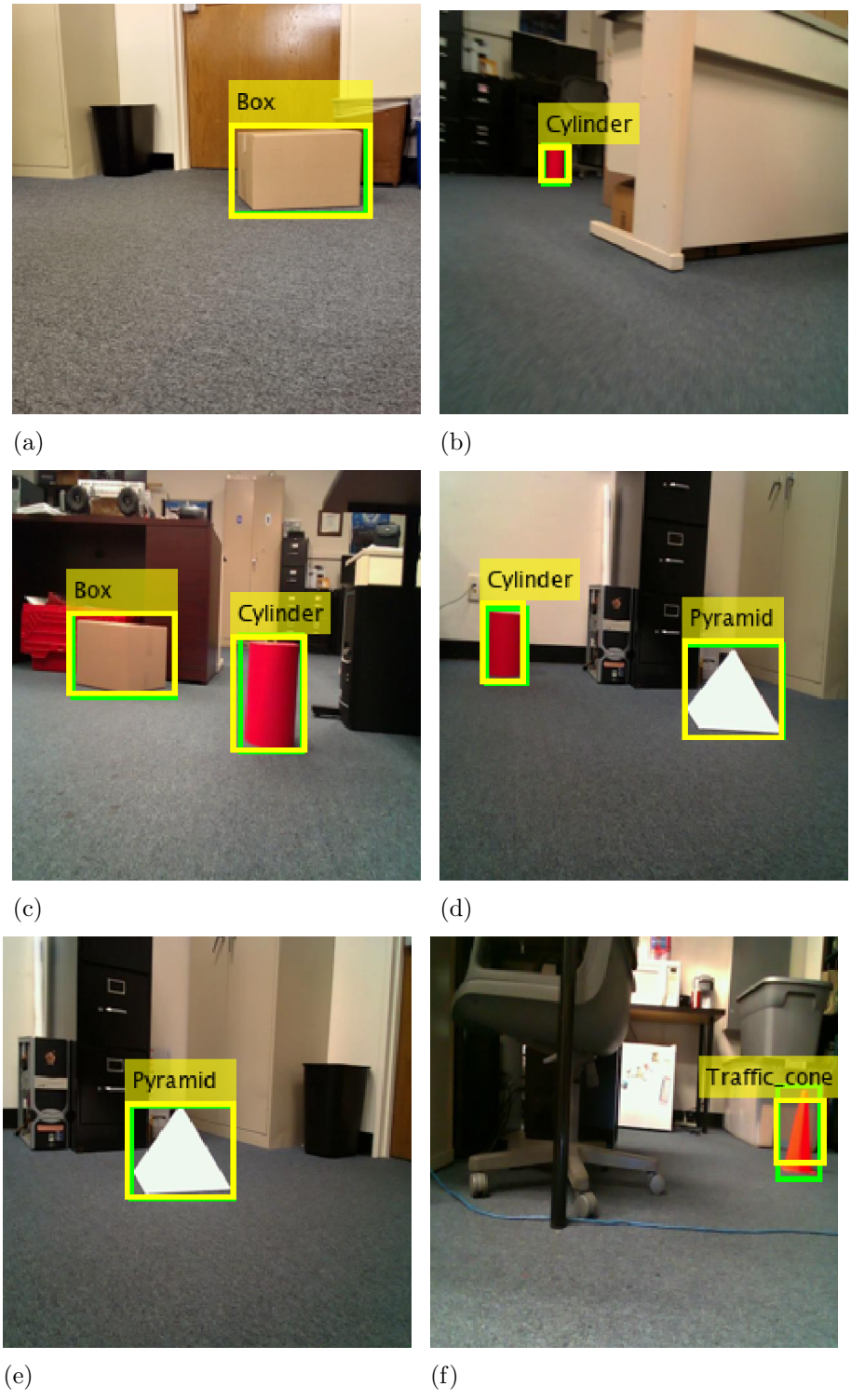
Figure 7.6. Overall Precision of Test Batch.

Parameter	Worth
Average Detection Time	59 ms
mean IoU	0.8157
Training Accuracy	99.08%
Testing Accuracy	96.25%

Table 7.2. Performance Parameters of Testing

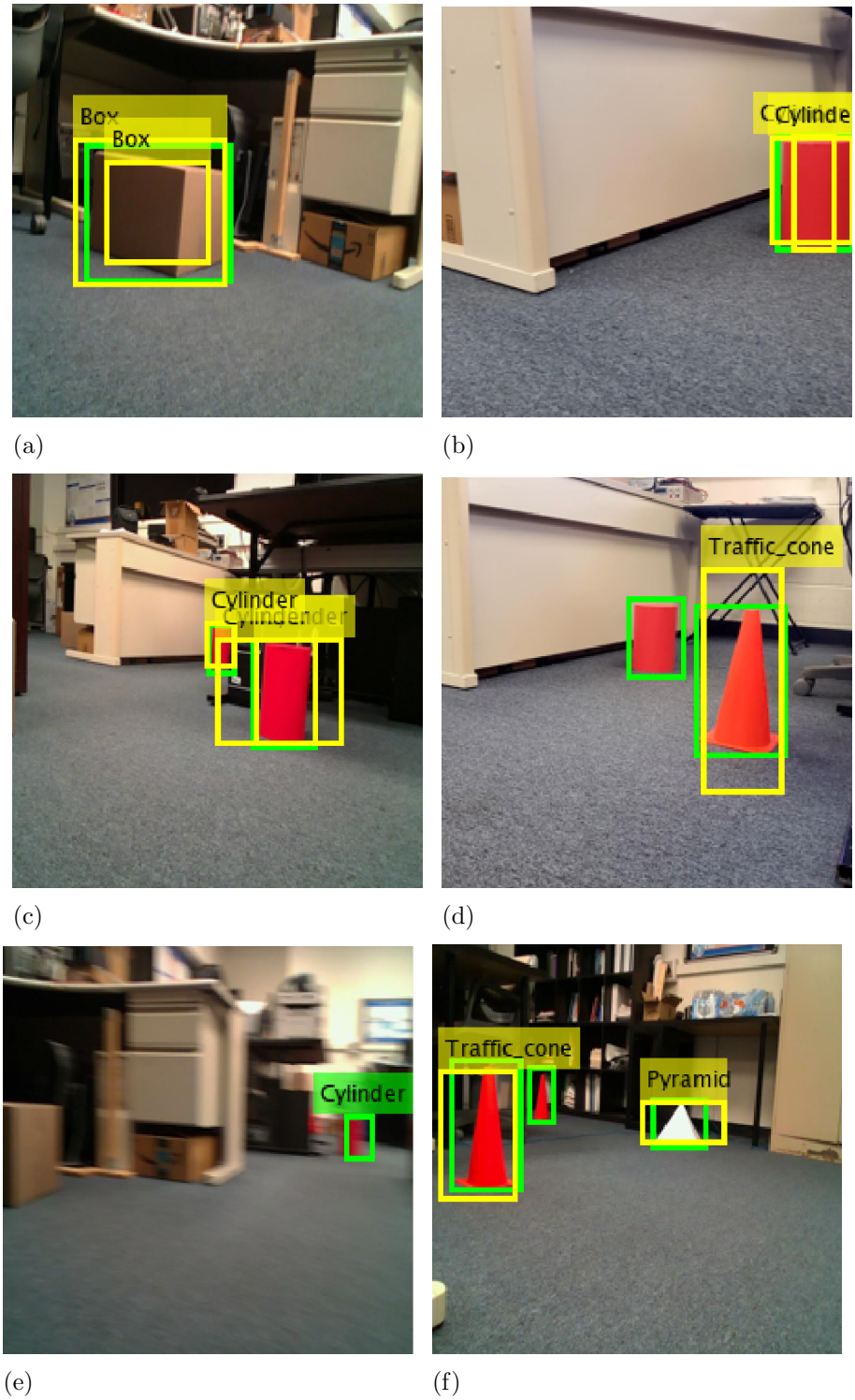
Image Tag	Object	Overlap Ratio	Precision	time
(a)	Box	0.95	97%	91 ms
(b)	Cylinder	0.76	96%	75 ms
(c)	Cylinder	0.85	89%	120 ms
(c)	Box	0.92	99%	120 ms
(d)	Cylinder	0.77	98%	58 ms
(d)	Pyramid	0.94	99%	58 ms
(e)	Pyramid	0.93	99%	120 ms
(f)	Traffic Cone	0.62	99%	66 ms

Table 7.3. Performance Parameters of Detections in Fig.7.7



(a) (b) (c) (d) (e) (f)

Figure 7.7. True Positive Detections of Test Batch.



(a) (b)

(c) (d)

(e) (f)

Figure 7.8. Faulty Detections of Test Batch (a),(b), and (c) Multiple Detection for Single Object, (d) and (f) No Detection, and (e) No Detection due to Motion Blur.

7.4 Real-time Object Detection Results and Position Estimation

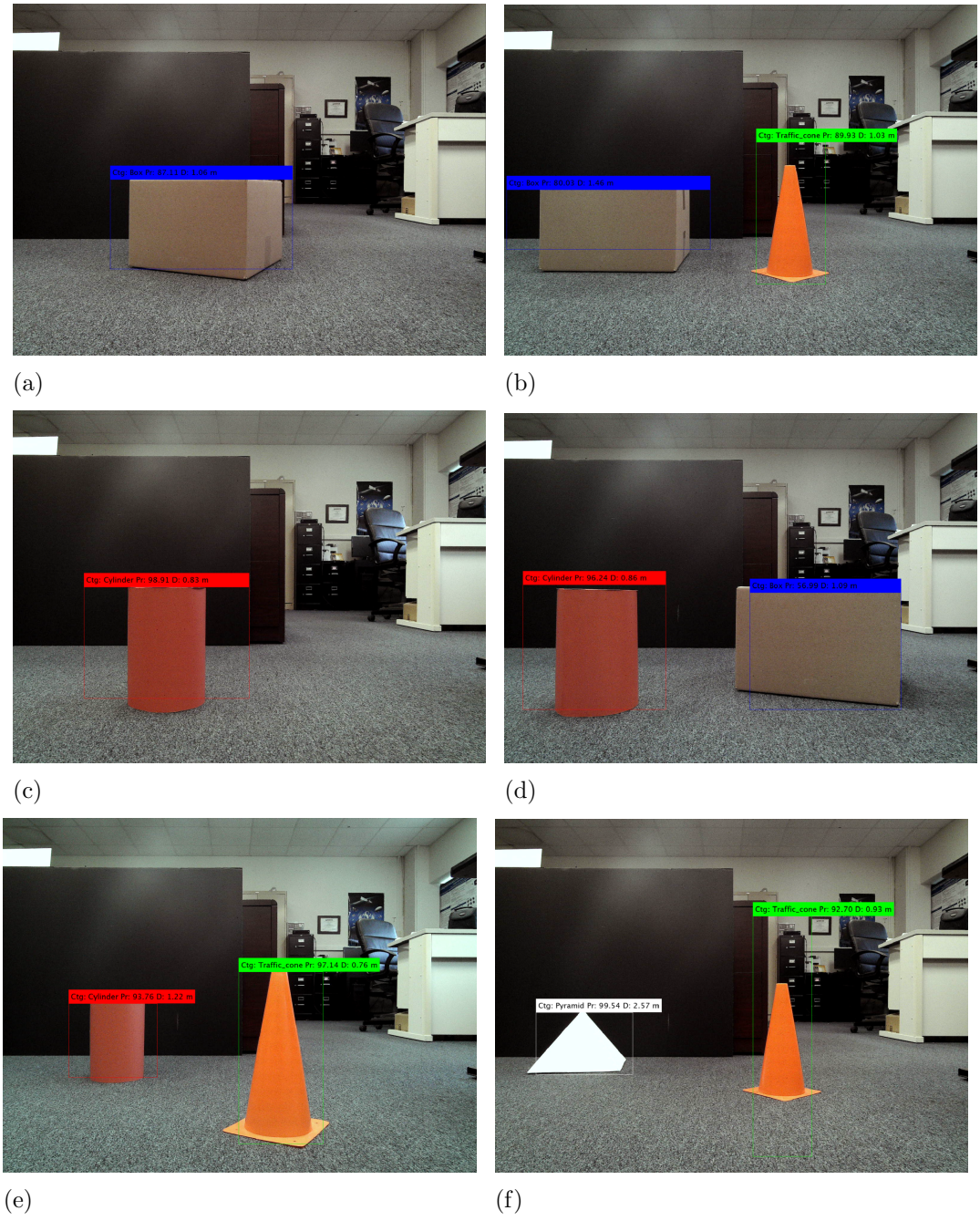


Figure 7.9. Real time Object Detection with Position Estimation.

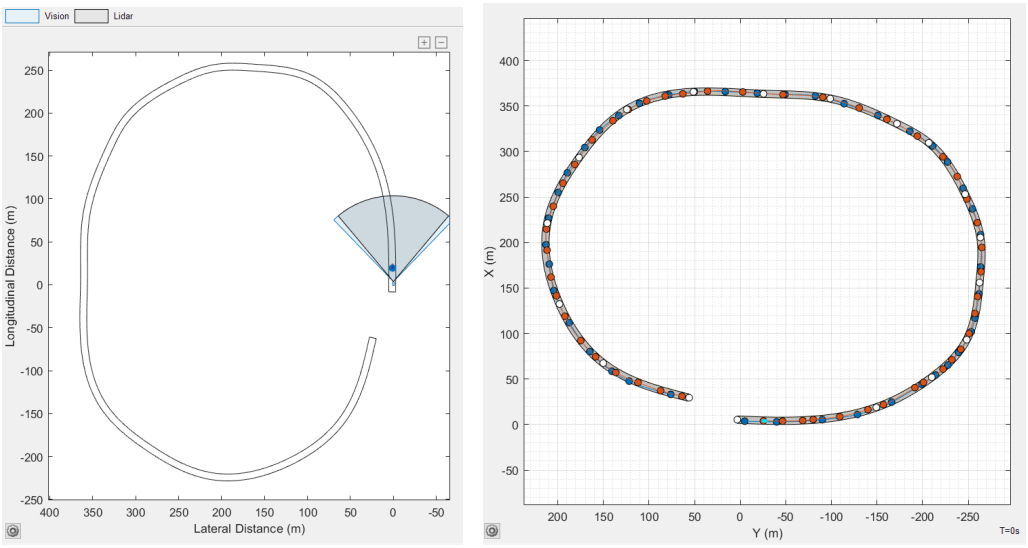
7.5 Simulation Results of Sensor Fusion

The simulation environment is created in MATLAB 2020b using the driving scene designer tool in order to evaluate the sensor fusion implementation. The essential parameters are defined as given in table 7.4. The lane, way-points, ego vehicle, object vehicle, and sensors definition are shown in Fig. 7.10. The camera and LiDAR sensors are mounted on the ego vehicle and uncertainty in sensor measurements follows the white Gaussian noise. Since this sensor provides the measurement data w.r.t body frame of the ego vehicle, it does not require performing extrinsic calibration of a sensor to find the transformation matrix. Sensor's intrinsic calibration is performed from the measurement data and their standard measurement error is given in table 7.5. The sensor covariance for LiDAR and camera which are derived from the calibration results are as follows.

$$R_{LiDAR} = \begin{bmatrix} 0.2102 & 0 & 0 \\ 0 & 0.0093 & 0 \\ 0 & 0 & 0.0001 \end{bmatrix} \tag{7.1}$$

$$R_{Camera} = \begin{bmatrix} 1.5336 & 0 & 0 \\ 0 & 1.3365 & 0 \\ 0 & 0 & 0.0001 \end{bmatrix} \tag{7.2}$$

Sensor fusion is implemented for the simulation environment using the linear Kalman filter. Dynamic model of the system is considered as a linear constant acceleration model. In order to characterize the performance of the sensor fusion, the estimated error is compared with sensor measurement error in table 7.5 and time history is shown in Fig. 7.15. It is inferred that the estimation error is improved by 56% compared to camera measurement error and reduction in variance after Camera-LiDAR fusion as shown in the residue error plot w.r.t in Fig. 7.15.



(a) (b)

Figure 7.10. Driving Scene in Simulation (a) Bird's Eye View of Scene (b) Way points Definition.

Moreover, the states which cannot be measured by sensors can be estimated through sensor fusion such as velocity and acceleration. In Fig. 7.11, the time history of the estimated position, velocity, and acceleration is illustrated and compared with the available ground truth data.

Scene Parameter	**Worth**
Ego Vehicle Speed	30 m/s (67 mph)
Object Vehicle Speed	30 m/s (67 mph)
Camera Azimuthal Limits	[-43 43] (deg)
Camera Elevation Limits	[-90 90] (deg)
LiDAR Azimuthal Limits	[-40 40] (deg)
LiDAR Elevation Limits	[-90 90] (deg)

Table 7.4. Driving Scene Parameter Definitions

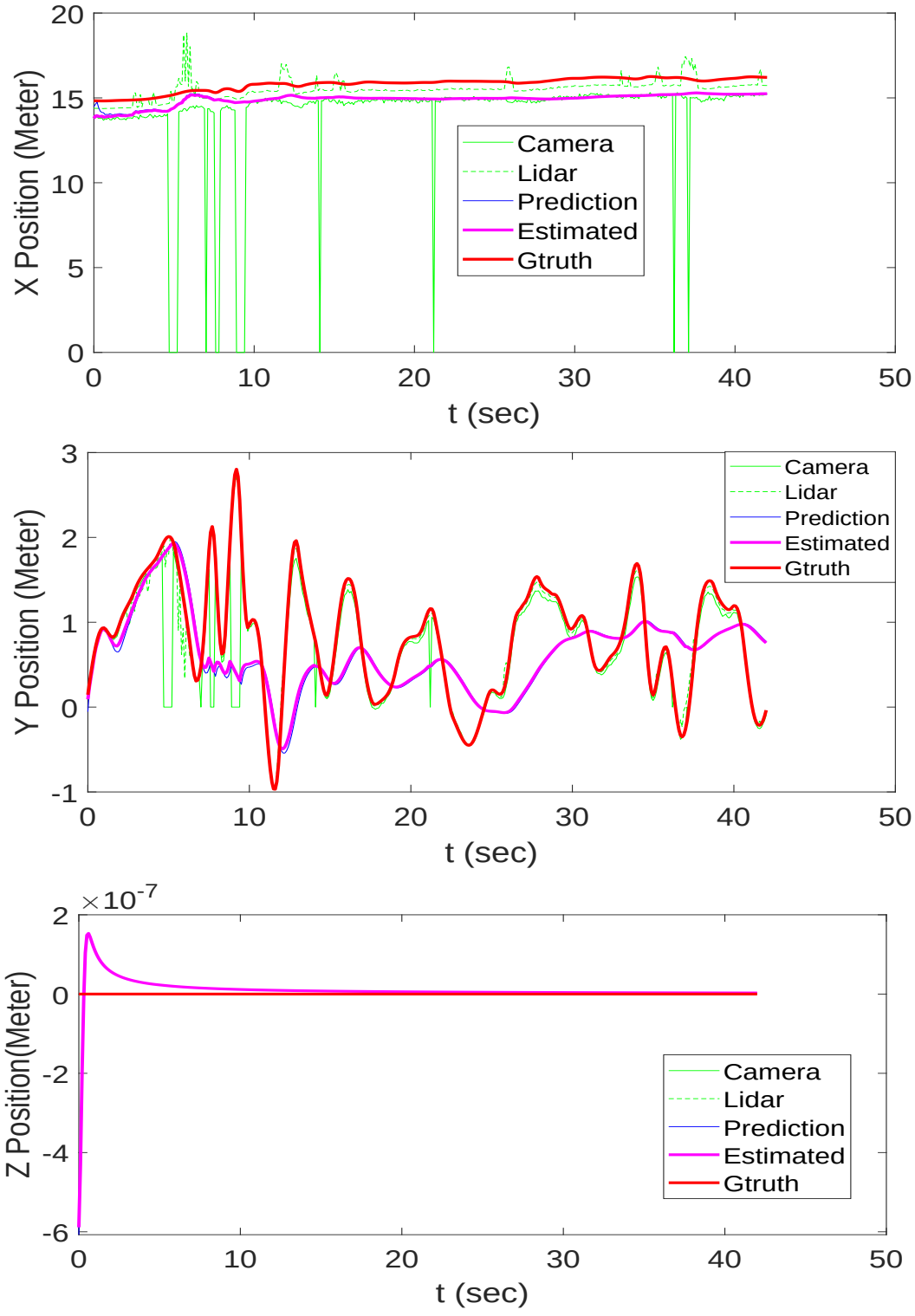


Figure 7.11. Position Estimation of Object in Simulation.

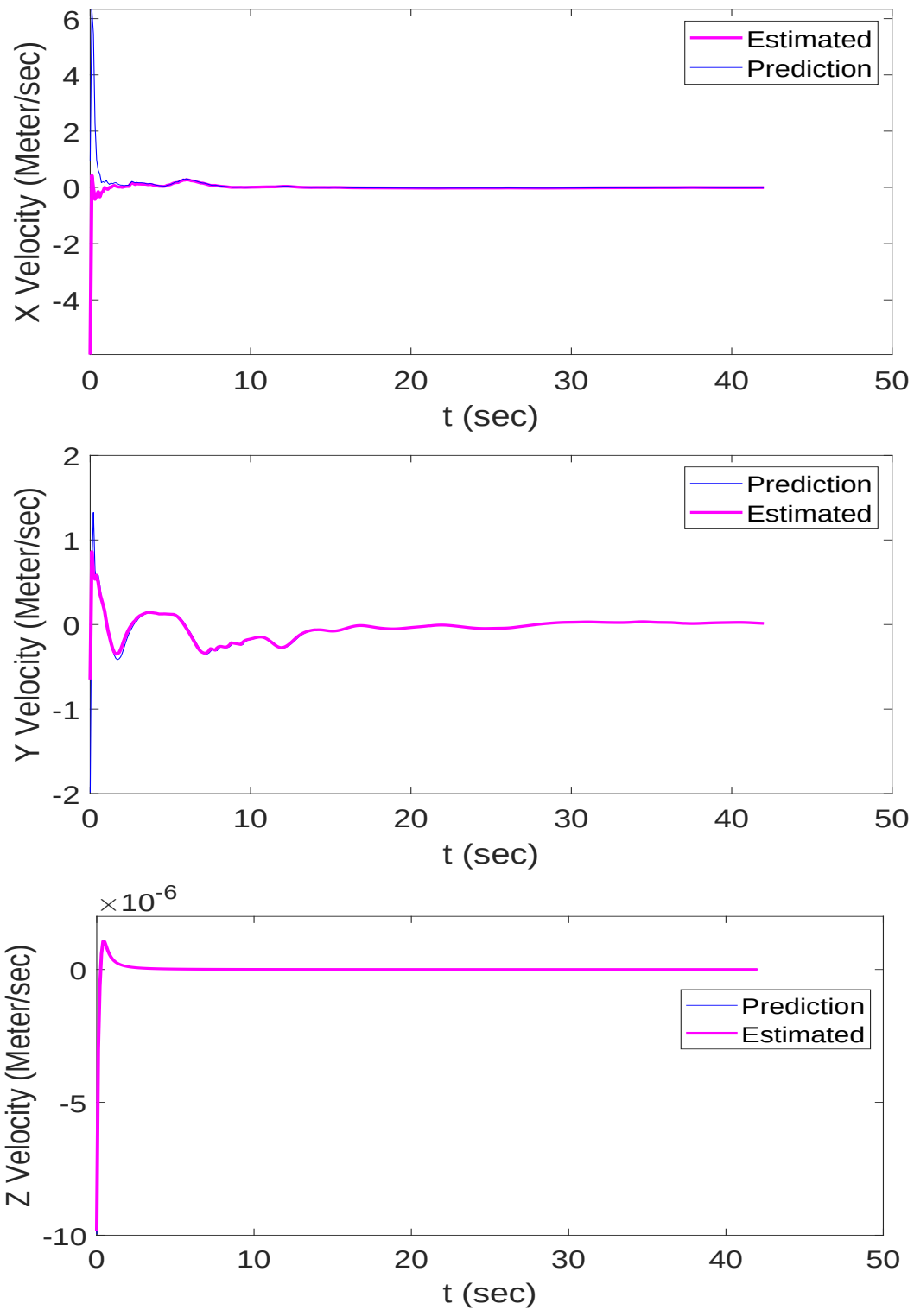


Figure 7.12. Velocity Estimation of Object in Simulation.

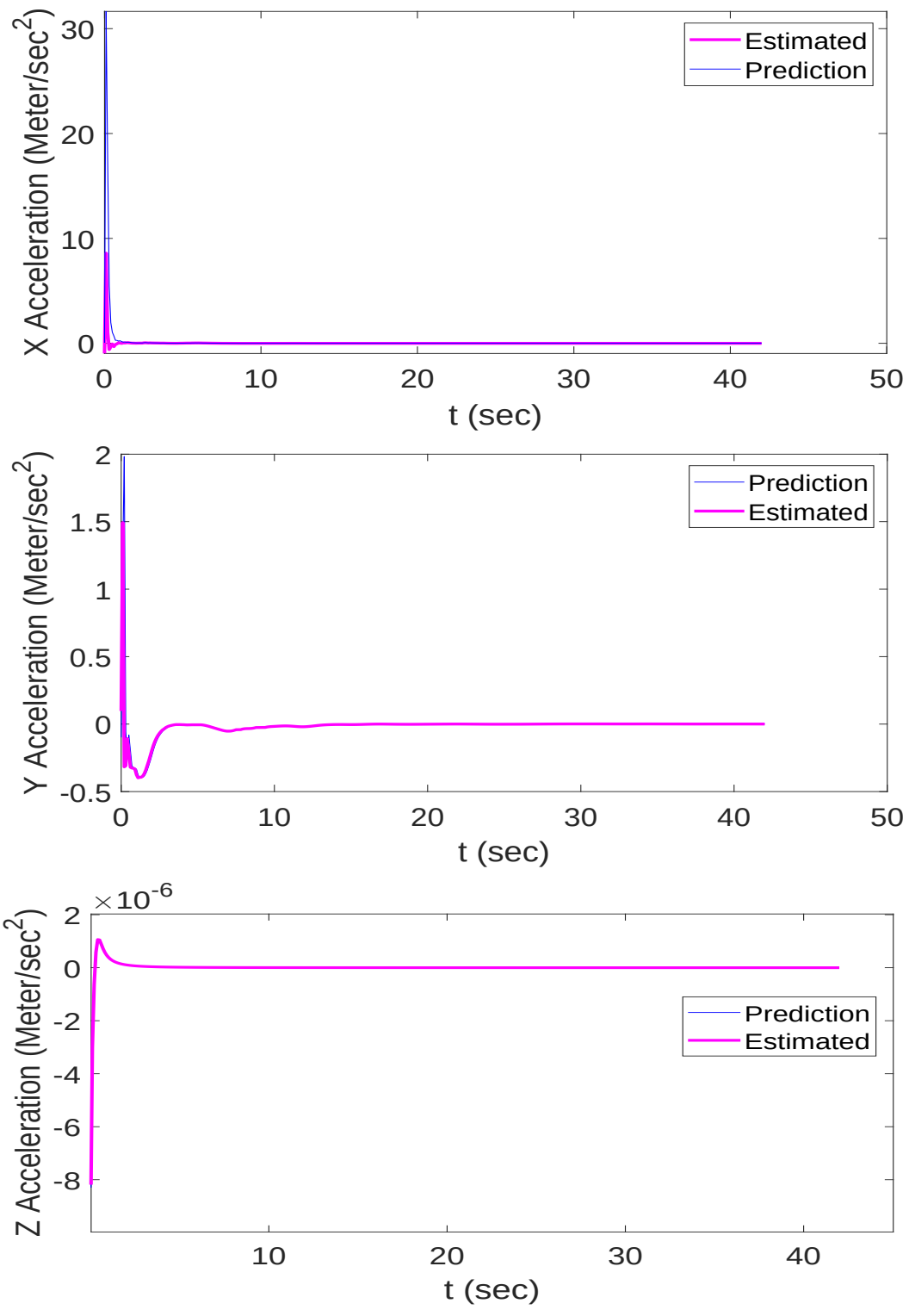


Figure 7.13. Acceleration Estimation of Object in Simulation.

Description	Camera	LiDAR	Estimation
X Error	0.95 m	0.29 m	0.40 m
Y Error	0.20 m	0.033 m	0.14 m
X Error	0.0001 m	0.0001 m	0.00001 m
Overall Error	0.97 m	0.29 m	0.42 m

Table 7.5. Sensor Measurement and Residue Error in Driving Scene

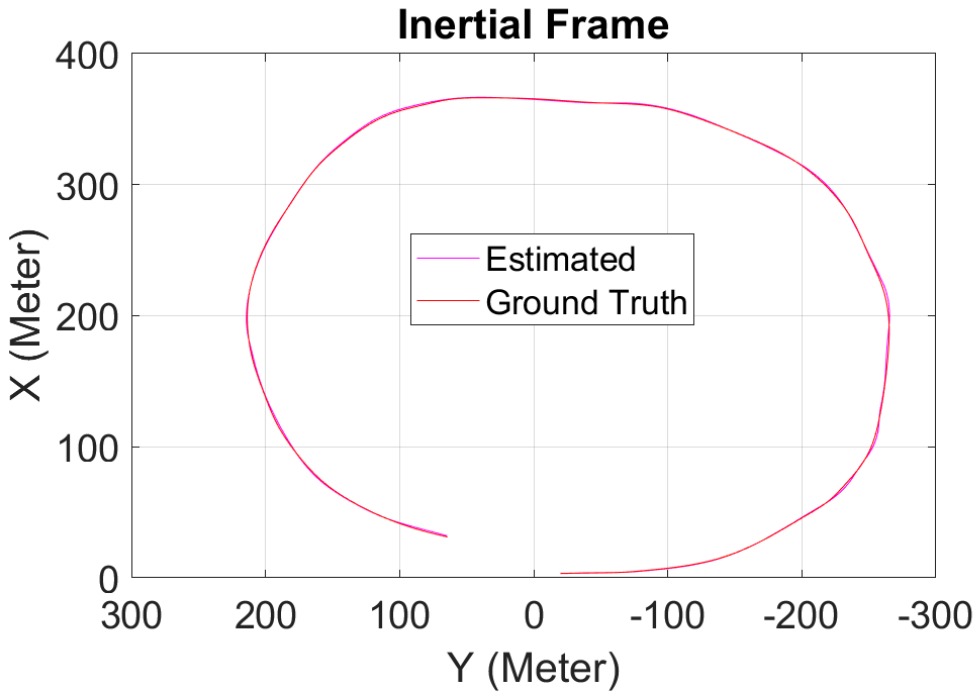


Figure 7.14. Trajectory Estimation of a Object w.r.t Inertial Frame.

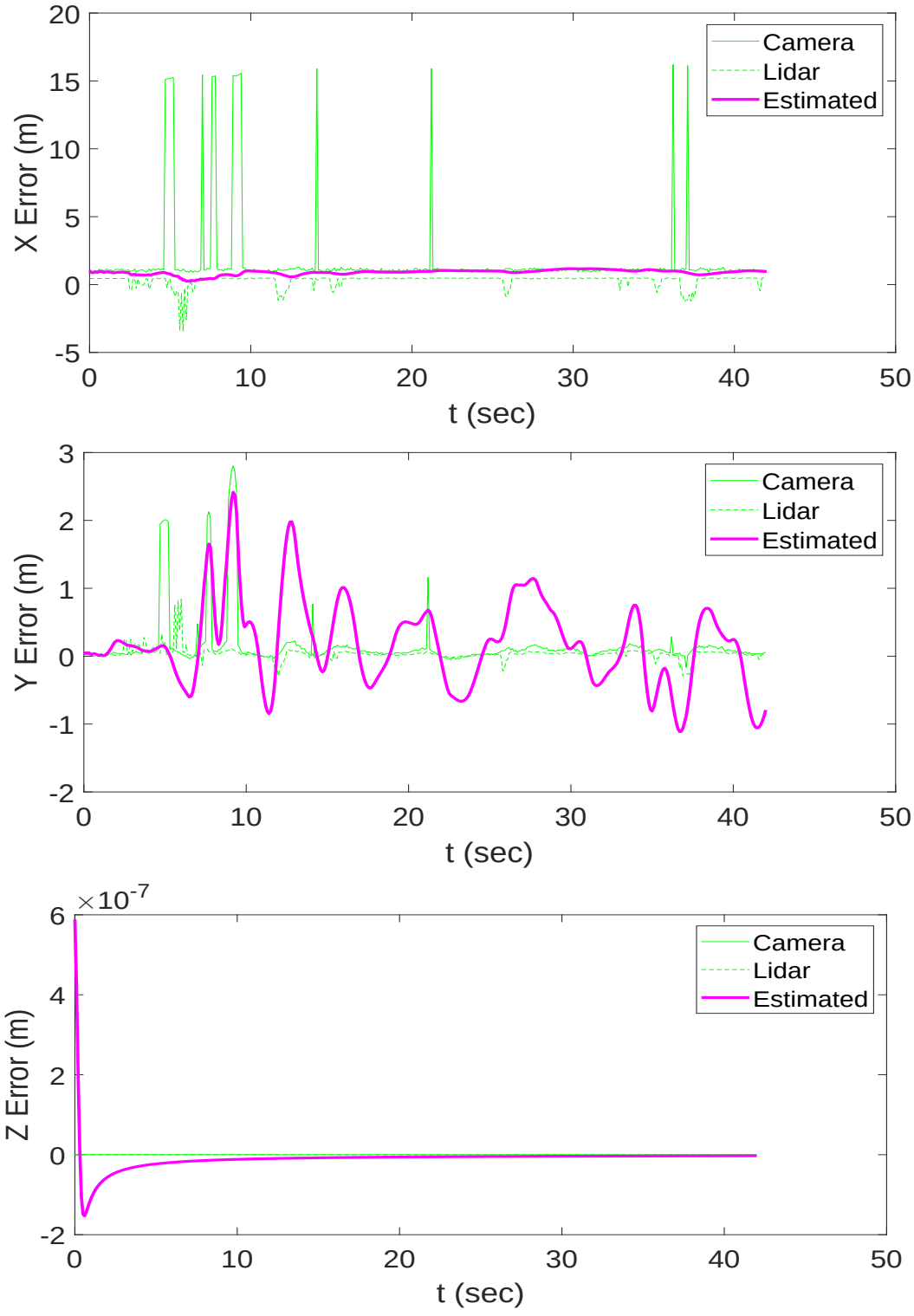


Figure 7.15. Sensor Measurement and Estimation Error Plot.

7.6 Sensor Fusion Implementation on Hardware

The real-time object estimation is implemented by fusing information from two optical sensors namely LiDAR and stereo camera. From the sensor intrinsic calibration, the sensor measurement covariance is derived for each sensor which is mention in Eq. (7.3) and (7.4). The Linear Kalman filter is used to estimate the position of an object which we have discussed in the section 6.4. Since the image label from the camera is used to predict the label from LiDAR data, both sensors are synchronized to generate measurement data at the same time. The data acquisition frequency for both synchronized sensors is 41 Hz. The NVIDIA GeForce GTX 1050 GPU is used to run the real-time object detection and position estimation framework. On this hardware, the average timestamp of 1.44 sec is achieved, unlike CPU based system that takes approximately 8 sec to completely process a single frame.

The sensor fusion framework is used to estimate the position of an object detected by the YOLO-v3 network. The algorithm was tested for two objects Box and Traffic cone. Their position and velocity plots are shown in Fig. 7.16 and 7.18. Moreover, the real-time object position estimation results are illustrated in Fig. 7.9, and the label annotated on the bounding box shows the following information: Object category (Ctg), precision in detection (Pr), Estimated Distance from the vehicle (D), Estimated velocity w.r.t vehicle (V), and Estimated Acceleration w.r.t vehicle (a).

$$R_{LiDAR} = \begin{bmatrix} 5.84e-05 & 0 & 0 \\ 0 & 2.73e-04 & 0 \\ 0 & 0 & 3.917e-06 \end{bmatrix} \tag{7.3}$$

$$R_{Camera} = \begin{bmatrix} 2.59e-05 & 0 & 0 \\ 0 & 9.76e-06 & 0 \\ 0 & 0 & 4.46e-04 \end{bmatrix} \tag{7.4}$$

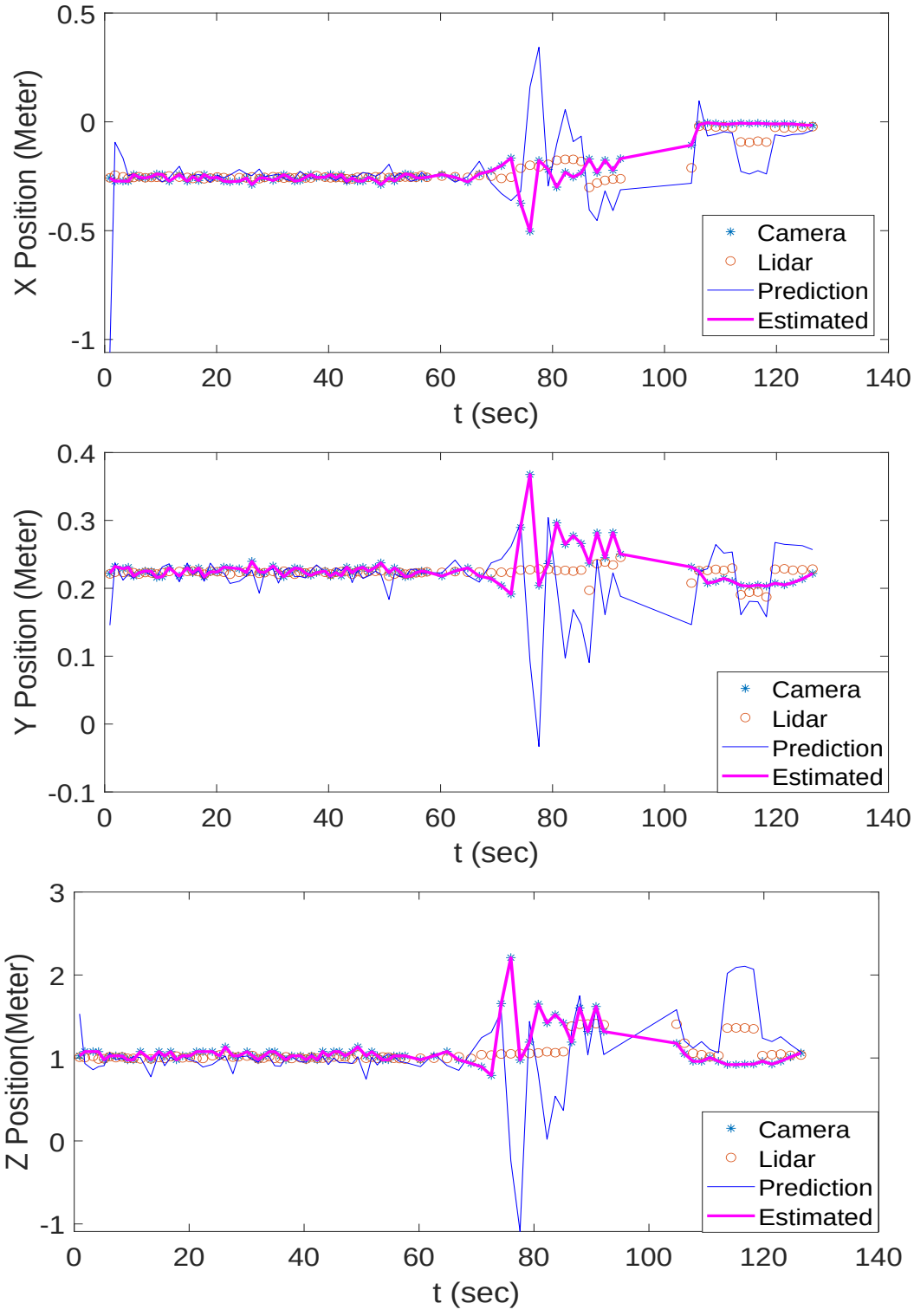


Figure 7.16. Real time Position Estimation of a Object 'Box'.

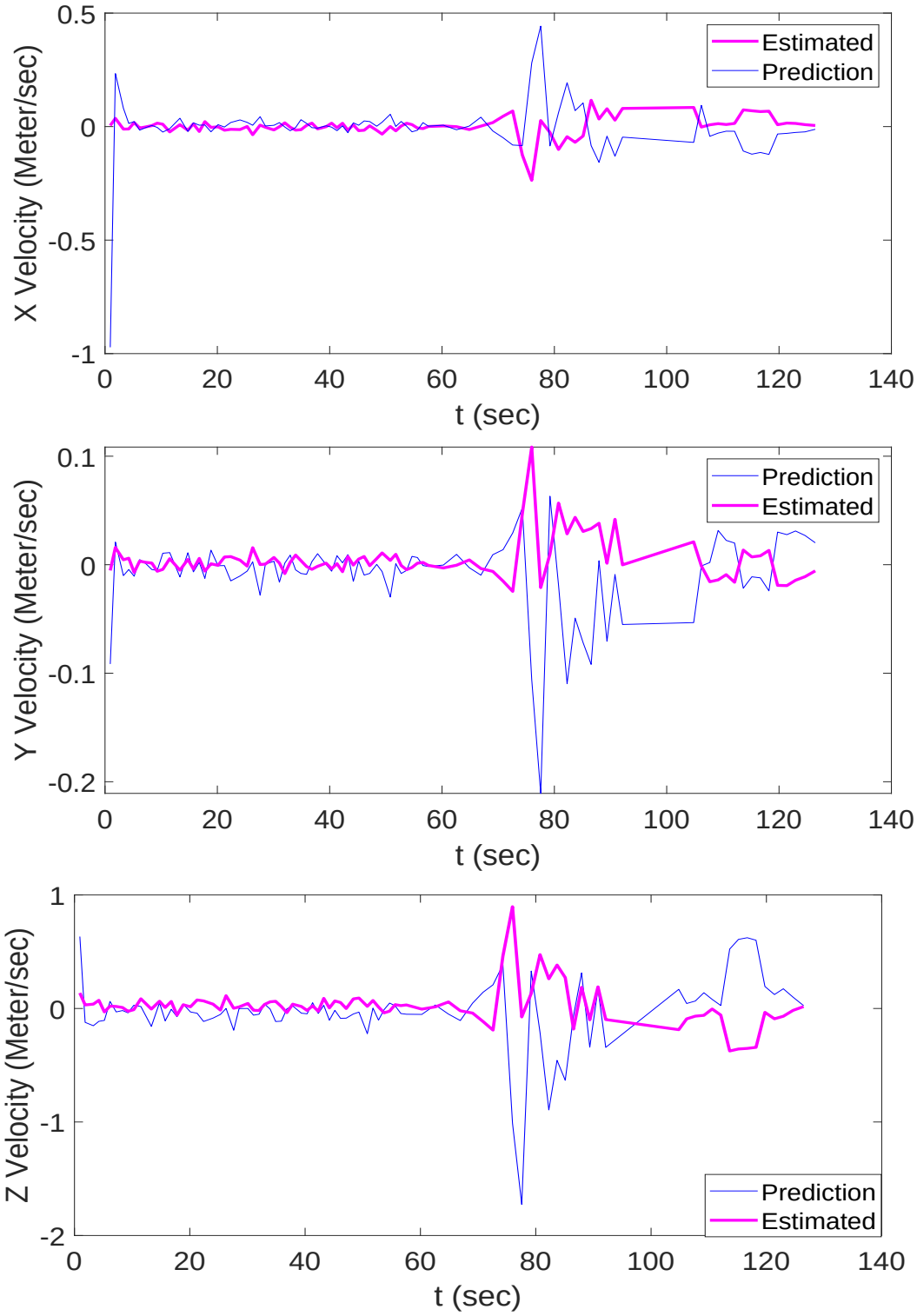


Figure 7.17. Real time Velocity Estimation of a Object 'Box'.

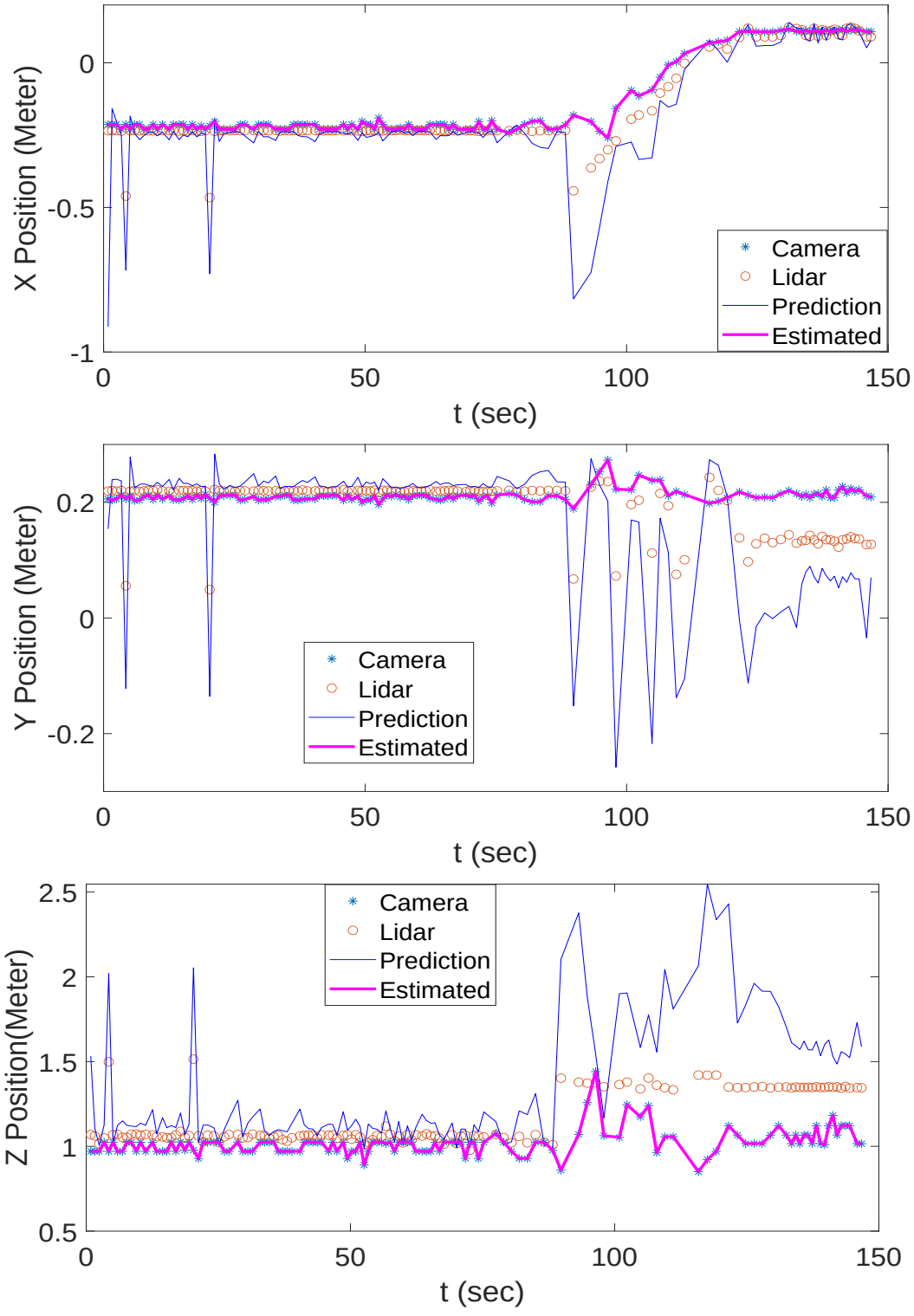


Figure 7.18. Real time Position Estimation of a Object 'Traffic cone'.

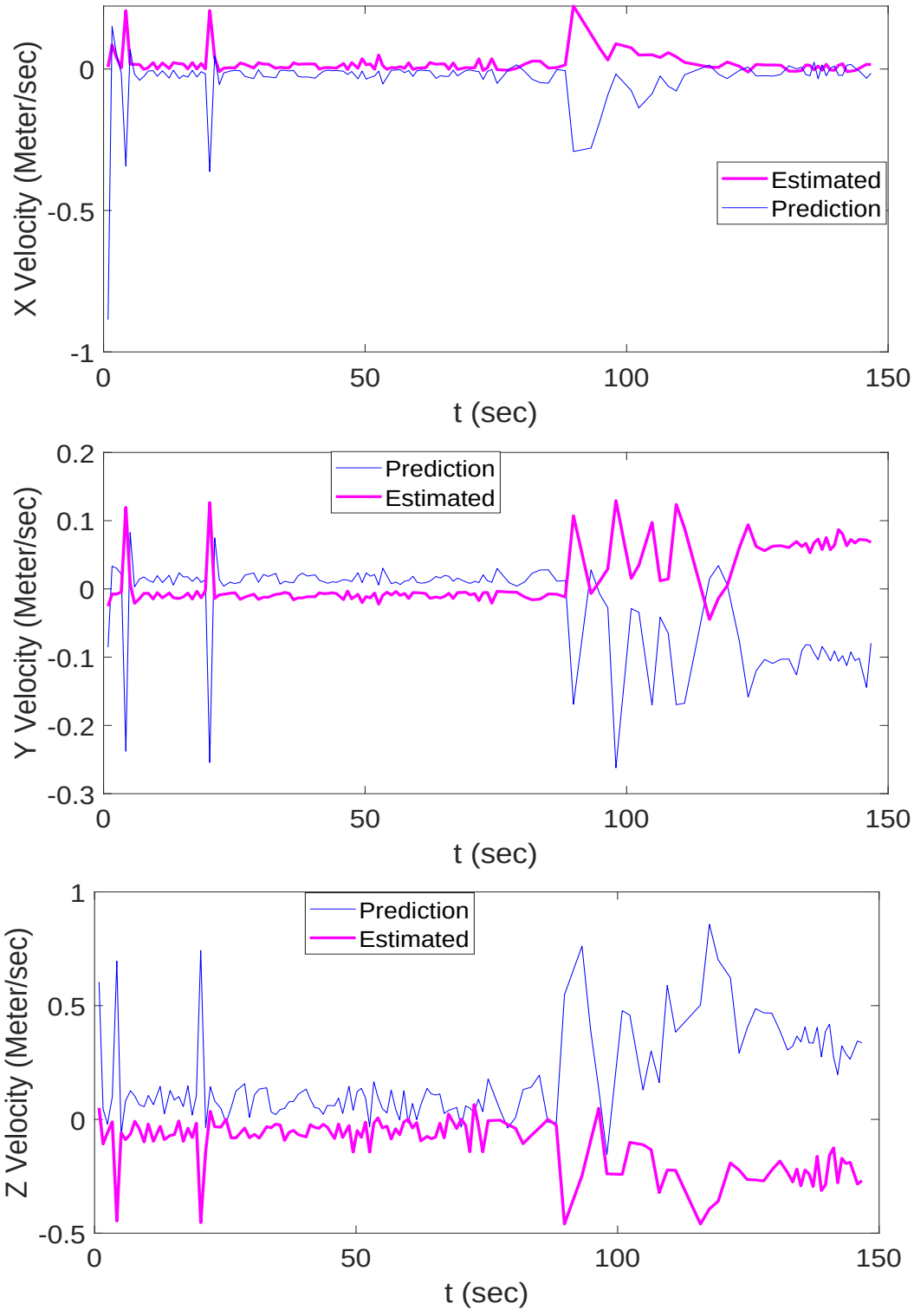
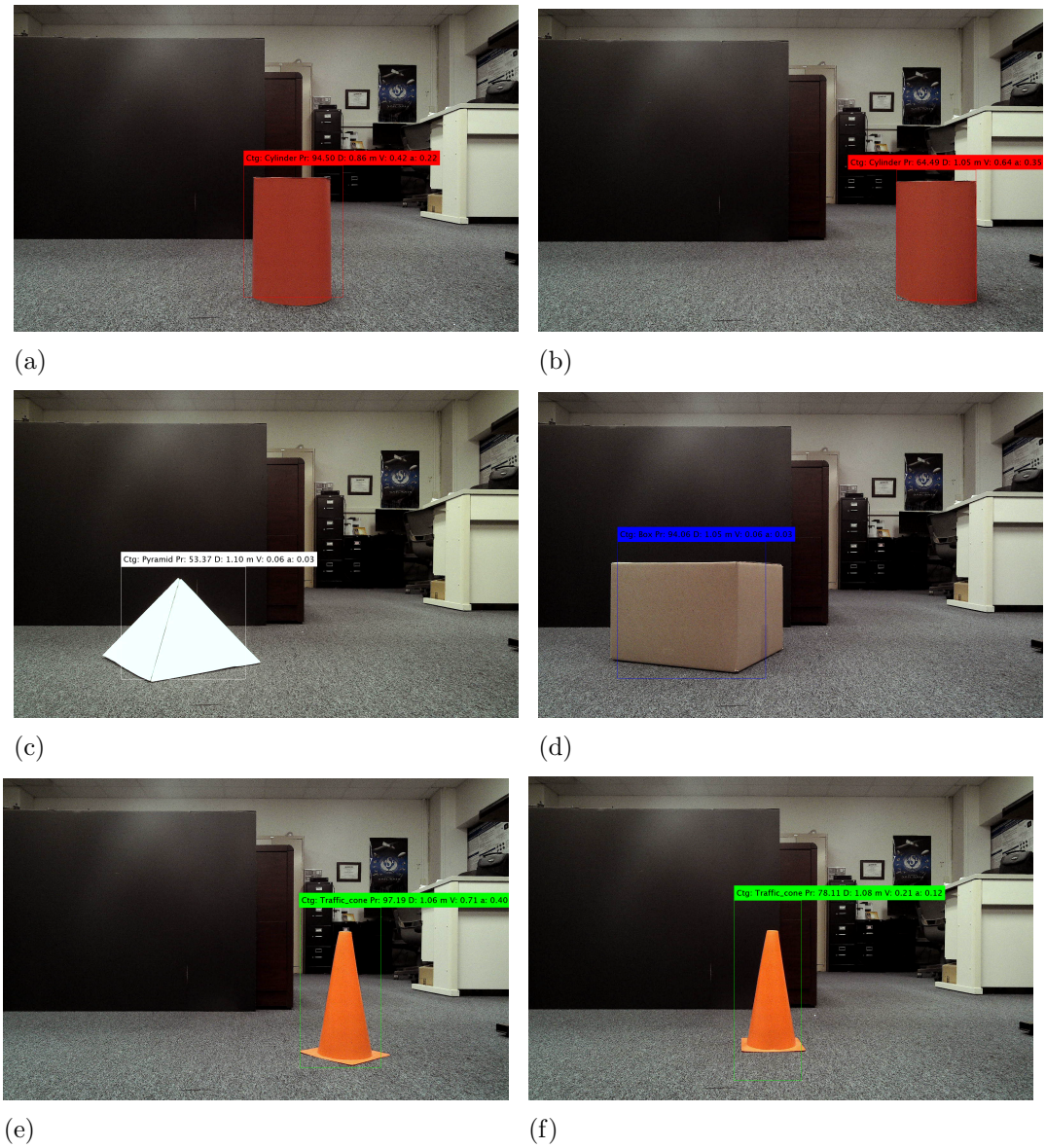


Figure 7.19. Real time Velocity Estimation of a Object 'Traffic cone'.



(a) (b)

(c) (d)

(e) (f)

Figure 7.20. Real time Object Detection and Position Estimation of Various Object.

7.7 Conclusion

It was shown in this work that the YOLO-v3 unified algorithm performs better and faster than most of the modern object detection neural networks. An intrinsic and extrinsic calibration of LiDAR and stereo camera was carried out to compute the measurement and transformation error of camera and LiDAR data. This was shown to be 0.3 m for the camera to LiDAR and LiDAR to camera transformation and -0.004 m and -0.021 m for camera and LiDAR transformation.

A labeled training data-set of 11000 images was created to train YOLO-v3 deep neural network so that it can detect four basic shapes: cylinder, cone, pyramid, and box. The data-set of 11000 images was randomly divided so that 98% of the data-set can be used for training and the other 2% can be used for validation of the trained neural network. The hyper-parameters like L2 regularization, minimum batch size, number of epoch, etc. were finely tuned to train the neural network faster with precision. The labeled data-set for LiDAR was generated by using the image label and creating the bounding boxes on LiDAR data based on the known depth of the obstacles. Finally, a sensor fusion algorithm was implemented by using a linear Kalman filter to fuse the data coming from LiDAR and the stereo camera.

7.8 Future Work

This study has covered a broad aspect of the field of computer vision, deep learning, statistics, and practical implementation. To cover greater depth in each mentioned field is difficult due to time constrains; however, this study lays down a ground for further advanced research. Some suggestions for possible future work are listed as follows.

- Develop a more sophisticated multi-object tracking algorithm; for instance JPDA (Joint Probabilistic Data Association).
- Further this study can be extended to implement the collision avoidance, and SLAM (Simultaneous Localization and Mapping).
- The neural network can be trained to detect more complex objects that might be encountered in the real world.
- The YOLO-v3 network can be further tuned to improve test accuracy.
- By use of the LiDAR with a stronger laser that can be used in ambient conditions, this research work can be extended to autonomous driving or advanced driving assistance system.
- The LiDAR and camera calibrations can be automated and made more accurate to improve the overall system accuracy in the future.

References

[1] G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision in C++ with the OpenCV Library, 2nd ed. O'Reilly Media, Inc., 2013.

[2] https://www.mathworks.com/help/vision/ug/camera-calibration.html.

[3] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," arXiv preprint arXiv:1804.02767, 2018.

[4] https://www.mathworks.com/help/lidar/ug/detect-vehicles-in-lidar-using-image-labels.html.

[5] H. Ma, Y. Liu, Y. Ren, and J. Yu, "Detection of collapsed buildings in post-earthquake remote sensing images based on the improved yolov3," Remote Sensing, vol. 12, no. 1, 2020.

[6] https://www.coursera.org/learn/neural-networks-deep-learning/home/welcome.

[7] https://computersciencewiki.org/index.php/Max-pooling_/_Pooling.

[8] K. Kolev, "Convexity in image-based 3d surface reconstruction," 2012.

[9] https://towardsdatascience.com/stochastic-gradient-descent-with-momentum-a84097641a5d.

[10] J. Redmon and A. Farhadi, "Yolo9000: Better, faster, stronger," in 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 6517–6525.

[11] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," J. Mach. Learn. Res., vol. 13, no. null, p. 281–305, Feb. 2012.

[12] https://blog.floydhub.com/guide-to-hyperparameters-search-for-deep-learning-models.

[13] I. Goodfellow, Y. Bengio, A. Courville, and Y. Bengio, Deep learning. MIT press Cambridge, 2016, vol. 1, no. 2.

[14] L. Jiang and Y. Xia, Multisensor Fusion Estimation Theory and Application, 01 2021.

[15] https://www.coursera.org/learn/state-estimation-localization-self-driving-cars/home/week/2.

[16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in Advances in Neural Information Processing Systems 25, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2012, pp. 1097–1105.

[17] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," 2014.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," Lecture Notes in Computer Science, p. 346–361, 2014. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-10578-9_23

[19] R. Girshick, "Fast r-cnn," in 2015 IEEE International Conference on Computer Vision (ICCV), 2015, pp. 1440–1448.

[20] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun, "Overfeat: Integrated recognition, localization and detection using convolutional networks," 2014.

[21] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28. Curran Associates, Inc.,

2015. [Online]. Available: https://proceedings.neurips.cc/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf

[22] J. Uijlings, K. Sande, T. Gevers, and A. Smeulders, "Selective search for object recognition," International Journal of Computer Vision, vol. 104, pp. 154–171, 09 2013.

[23] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," 2016.

[24] X. Chen and A. Gupta, "An implementation of faster rcnn with study for region sampling," 2017.

[25] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 779–788.

[26] J. Sang, Z. Wu, P. Guo, H. Hu, H. Xiang, Q. Zhang, and B. Cai, "An improved yolov2 for vehicle detection," Sensors, vol. 18, no. 12, p. 4272, 2018.

[27] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," 2014.

[28] J. Van Brummelen, M. O'Brien, D. Gruyer, and H. Najjaran, "Autonomous vehicle perception: The technology of today and tomorrow," Transportation research part C: emerging technologies, vol. 89, pp. 384–406, 2018.

[29] G. A. Kumar, J. H. Lee, J. Hwang, J. Park, S. H. Youn, and S. Kwon, "Lidar and camera fusion approach for object distance estimation in self-driving vehicles," Symmetry, vol. 12, no. 2, 2020.

[30] R. Labayrade, C. Royere, D. Gruyer, and D. Aubert, "Cooperative fusion for multi-obstacles detection with use of stereovision and laser scanner," Autonomous Robots, vol. 19, no. 2, pp. 117–140, 2005.

[31] J. Shi, W. Wang, X. Wang, H. Sun, X. Lan, J. Xin, and N. Zheng, "Leveraging spatio-temporal evidence and independent vision channel to improve multi-sensor fusion for vehicle environmental perception," in 2018 IEEE Intelligent Vehicles Symposium (IV). IEEE, 2018, pp. 591–596.

[32] G. Zhao, X. Xiao, J. Yuan, and G. W. Ng, "Fusion of 3d-lidar and camera data for scene parsing," Journal of Visual Communication and Image Representation, vol. 25, no. 1, pp. 165–183, 2014.

[33] J. Li, "Fusion of lidar 3d points cloud with 2d digital camera image," Oakland University: Rochester, MI, USA, 2015.

[34] J. Li, X. He, and J. Li, "2d lidar and camera fusion in 3d modeling of indoor environment," in 2015 National Aerospace and Electronics Conference (NAECON). IEEE, 2015, pp. 379–383.

[35] A. Mastin, J. Kepner, and J. Fisher, "Automatic registration of lidar and optical images of urban scenes," in 2009 IEEE conference on computer vision and pattern recognition. IEEE, 2009, pp. 2639–2646.

[36] X. Gong, Y. Lin, and J. Liu, "3d lidar-camera extrinsic calibration using an arbitrary trihedron," Sensors, vol. 13, no. 2, pp. 1902–1918, 2013.

[37] H. Alismail, L. D. Baker, and B. Browning, "Automatic calibration of a range sensor and camera system," in 2012 Second International Conference on 3D Imaging, Modeling, Processing, Visualization & Transmission. IEEE, 2012, pp. 286–292.

[38] Y. Park, S. Yun, C. S. Won, K. Cho, K. Um, and S. Sim, "Calibration between color camera and 3d lidar instruments with a polygonal planar board," Sensors, vol. 14, no. 3, pp. 5333–5353, 2014.

[39] A.-I. García-Moreno, J.-J. Gonzalez-Barbosa, F.-J. Ornelas-Rodriguez, J. B. Hurtado-Ramos, and M.-N. Primo-Fuentes, "Lidar and panoramic camera ex-

trinsic calibration approach using a pattern plane," in Mexican Conference on Pattern Recognition. Springer, 2013, pp. 104–113.

[40] S. Verma, J. S. Berrio, S. Worrall, and E. Nebot, "Automatic extrinsic calibration between a camera and a 3d lidar using 3d point and plane correspondences," in 2019 IEEE Intelligent Transportation Systems Conference (ITSC). IEEE, 2019, pp. 3906–3912.

[41] A. Geiger, F. Moosmann, Ö. Car, and B. Schuster, "Automatic camera and range sensor calibration using a single shot," in 2012 IEEE International Conference on Robotics and Automation. IEEE, 2012, pp. 3936–3943.

[42] P. De Bièvre, "The 2012 international vocabulary of metrology: "vim"," Accreditation and Quality Assurance, vol. 17, 04 2012.

[43] E. L. Waltz, J. Llinas, and F. E. White, Multisensor Data Fusion. Artech House, Inc., 1990.

Biographical Statement

Kamalkumar B. Mehta is currently staying in Arlington, TX. He is belonging from Bakor a small village in Gujarat, India. He earned his bachelor's degree in Aeronautical engineering from Gujarat Technological University (GTU), Ahmadabad, India in 2017. Upon completion of his baccalaureate degree, he enrolled in a master of science (M.S) degree in Aerospace engineering at the University of Texas at Arlington (UTA) in 2018. His coursework mainly focused on Guidance, navigation, and control of aerospace vehicles, Unmanned vehicle systems, deep learning, and optimal state estimation.